Oracle Database 12*c*: Data Guard Administration

Student Guide – Volume I

D79232GC10

Edition 1.0

February 2014

D85496

ORACLE

Author

Mark Fuller

Technical Contributors and Reviewers

Christopher Andrews
Maria Billings
Trevor Bowen
Harald van Breederode
Larry Carpenter
Al Flournoy
Joe Fong
Gerlinde Frenzen
Joel Goodman
Uwe Hesse
Nitin Karkhanis
Donna Keesling
Sean Kim
Jerry Lee,
Stephen Moriarty
Juan Quezada Nunez
Veerabhadra Rao Putrevu
Puneet Sangar
Thorsten Senft
Ira Singer
Branislav Valny

Editors

Daniel Milne
Raj Kumar
Anwesha Ray

Publishers

Joseph Fernandez
Veena Narasimhan

Contents

1

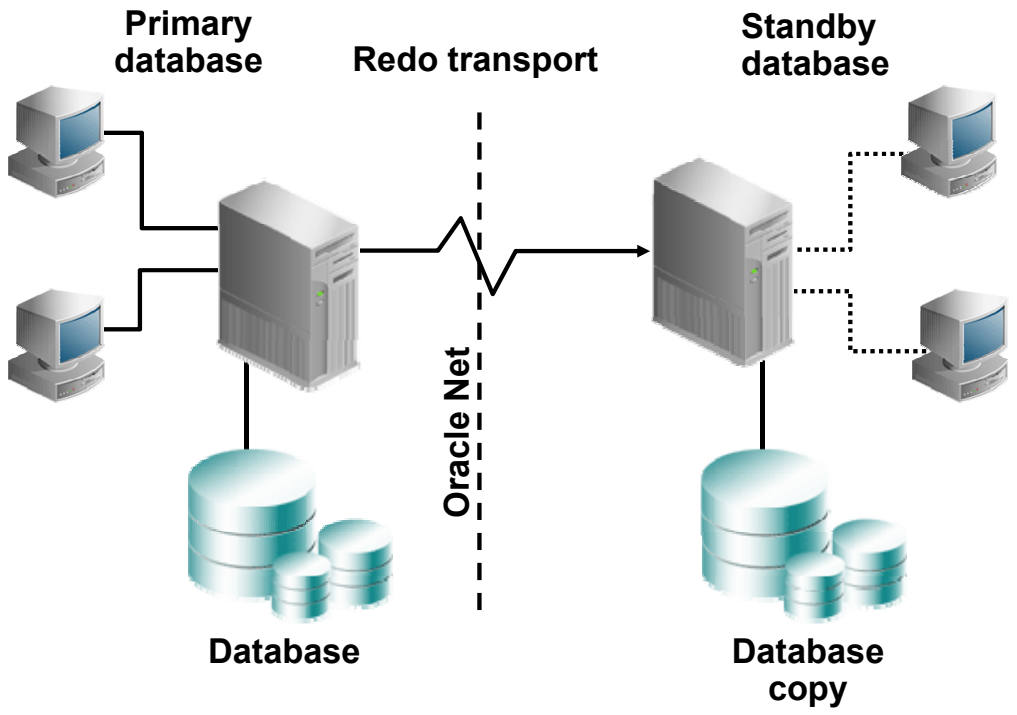
Introduction to Oracle Data Guard

Objectives

After completing this lesson, you should be able to:

- Describe the basic components of Oracle Data Guard
- Explain the differences between physical and logical standby databases
- Explain the benefits of implementing Oracle Data Guard

What Is Oracle Data Guard?



Oracle Data Guard is a central component of an integrated Oracle Database High Availability (HA) solution set that helps organizations ensure business continuity by minimizing the various kinds of planned and unplanned down time that can affect their businesses.

Oracle Data Guard is a management, monitoring, and automation software infrastructure that works with a production database and one or more standby databases to protect your data against failures, errors, and corruptions that might otherwise destroy your database. It protects critical data by providing facilities to automate the creation, management, and monitoring of the databases and other components in a Data Guard configuration. It automates the process of maintaining a copy of an Oracle production database (called a *standby database*) that can be used if the production database is taken offline for routine maintenance or becomes damaged.

In a Data Guard configuration, a production database is referred to as a *primary database*. A *standby database* is a synchronized copy of the primary database. By using a backup copy of the primary database, you can create from 1 to 30 standby databases. The standby databases, together with the primary database, make up a Data Guard configuration.

All Data Guard standby databases can enable up-to-date read access to the standby database while redo being received from the primary database is applied. This makes all standby databases excellent candidates for relieving the primary database of the overhead of supporting read-only queries and reporting.

Types of Standby Databases

- Physical standby database:
  - Is identical to the primary database on a block-for-block basis
  - Is synchronized with the primary database through application of redo data received from the primary database
  - Can be used concurrently for data protection and reporting
- Logical standby database
  - Shares the same schema definition
  - Is kept synchronized with the primary database by transforming the data in the redo received from the primary database into SQL statements and then executing the SQL statements
  - Can be used concurrently for data protection, reporting, and database upgrades

Physical Standby Database

A physical standby database is physically identical to the primary database, with on-disk database structures that are identical to the primary database on a block-for-block basis. The physical standby database is updated by performing recovery using redo data that is received from the primary database. Oracle Database12*c* enables a physical standby database to receive and apply redo while it is open in read-only mode.

Logical Standby Database

A logical standby database contains the same logical information (unless configured to skip certain objects) as the production database, although the physical organization and structure of the data can be different. The logical standby database is kept synchronized with the primary database by transforming the data in the redo received from the primary database into SQL statements and then executing the SQL statements on the standby database. This is done with the use of LogMiner technology on the redo data received from the primary database. The tables in a logical standby database can be used simultaneously for recovery and for other tasks such as reporting, summations, and queries.

Note: For more information about LogMiner, see *Using LogMiner to Analyze Redo Log Files* in *Oracle Database Utilities12c Release 1 (12.1)* documentation.

Types of Standby Databases

- Snapshot standby database:
  - Is a fully updatable standby database
  - Is created by converting a physical standby database
  - Can be used for updates, but those updates are discarded before the snapshot standby database is converted back into a physical standby database
  - Can be used for testing

Snapshot Standby Database

A snapshot standby database is a database that is created by converting a physical standby database into a snapshot standby database. The snapshot standby database receives redo from the primary database, but does not apply the redo data until it is converted back into a physical standby database. The snapshot standby database can be used for updates, but those updates are discarded before the snapshot standby database is converted back into a physical standby database. The snapshot standby database is appropriate when you require a temporary, updatable version of a physical standby database.

Types of Data Guard Services

Data Guard provides three types of services:

- Redo transport services
- Apply services
  - Redo Apply
  - SQL Apply
- Role management services



ORACLE

The following types of services are available with Data Guard:

- **Redo transport services:** Control the automated transmittal of redo information from the primary database to one or more standby databases or archival destinations.
- **Apply services:** Control when and how data is applied to the standby database.
  - **Redo Apply:** Technology used for physical standby databases. Redo data is applied on the standby database by using Oracle media recovery.
  - **SQL Apply:** Technology used for logical standby databases. The received redo data is first transformed into SQL statements, and then the generated SQL statements are executed on the logical standby database.
- **Role management services:** A database operates in one of two mutually exclusive roles: primary or standby. Role management services operate in conjunction with redo transport services and apply services to change these roles dynamically as a planned transition (called a *switchover operation*) or as a result of database failure due to a failover operation.

Role Transitions: Switchover and Failover

Oracle Data Guard supports two role-transition operations:

- Switchover
  - Planned role reversal
  - Used for OS or hardware maintenance
- Failover
  - Unplanned role reversal
  - Emergency use
  - Zero or minimal data loss (depending on choice of data-protection mode)
  - Can be initiated automatically when fast-start failover is enabled

ORACLE

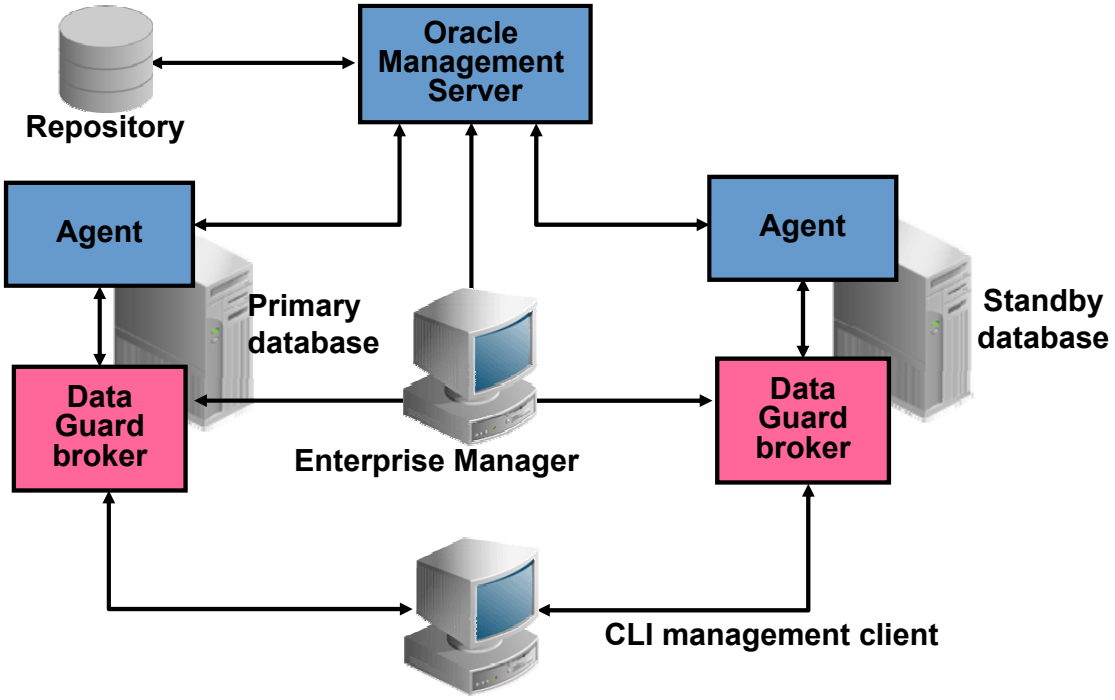
Data Guard enables you to change the role of a database dynamically by issuing SQL statements or by using either of the Data Guard broker's interfaces. Data Guard supports two role-transition operations:

- **Switchover:** The switchover feature enables you to switch the role of the primary database to one of the available standby databases. The chosen standby database becomes the primary database, and the original primary database then becomes a standby database.
- **Failover:** You invoke a failover operation when a catastrophic failure occurs on the primary database. During a failover operation, the failed primary database is removed from the Data Guard environment, and a standby database assumes the primary database role. You invoke the failover operation on the standby database that you want to fail over to the primary role. You can also enable fast-start failover, which allows Data Guard to automatically and quickly fail over to a previously chosen synchronized standby database.
  Note: Fast-start failover is discussed in detail in the lesson titled "Enabling Fast-Start Failover."

Databases that are disabled after a role transition are not removed from the broker configuration, but they are disabled in the sense that the databases are no longer managed by the broker. To reenable broker management of these databases, you must reinstate or re-create the databases.

Note: See the lesson titled “Performing Role Transitions” for detailed information.

Oracle Data Guard Broker Framework



The Oracle Data Guard broker is a distributed management framework that automates and centralizes the creation, maintenance, and monitoring of Data Guard configurations. After creating the Data Guard configuration, the broker monitors the activity, health, and availability of all systems in the configuration.

The benefits of using Oracle Data Guard broker include:

- Enhanced disaster protection
- Higher availability and scalability with Oracle Real Application Clusters (Oracle RAC) Databases
- Automated creation of a Data Guard configuration
- Easy configuration of additional standby databases
- Simplified, centralized, and extended management
- Simplified switchover and failover operations
- Fast Application Notification (FAN) after failovers
- Built-in monitoring and alert and control mechanisms
- Transparency to the application

Choosing an Interface for Administering a Data Guard Configuration

- Data Guard broker configuration:
  - DGMGRL command-line interface
  - Enterprise Manager Cloud Control
  - SQL commands to query data dictionary views
- Non–Data Guard broker configuration:
  - SQL commands

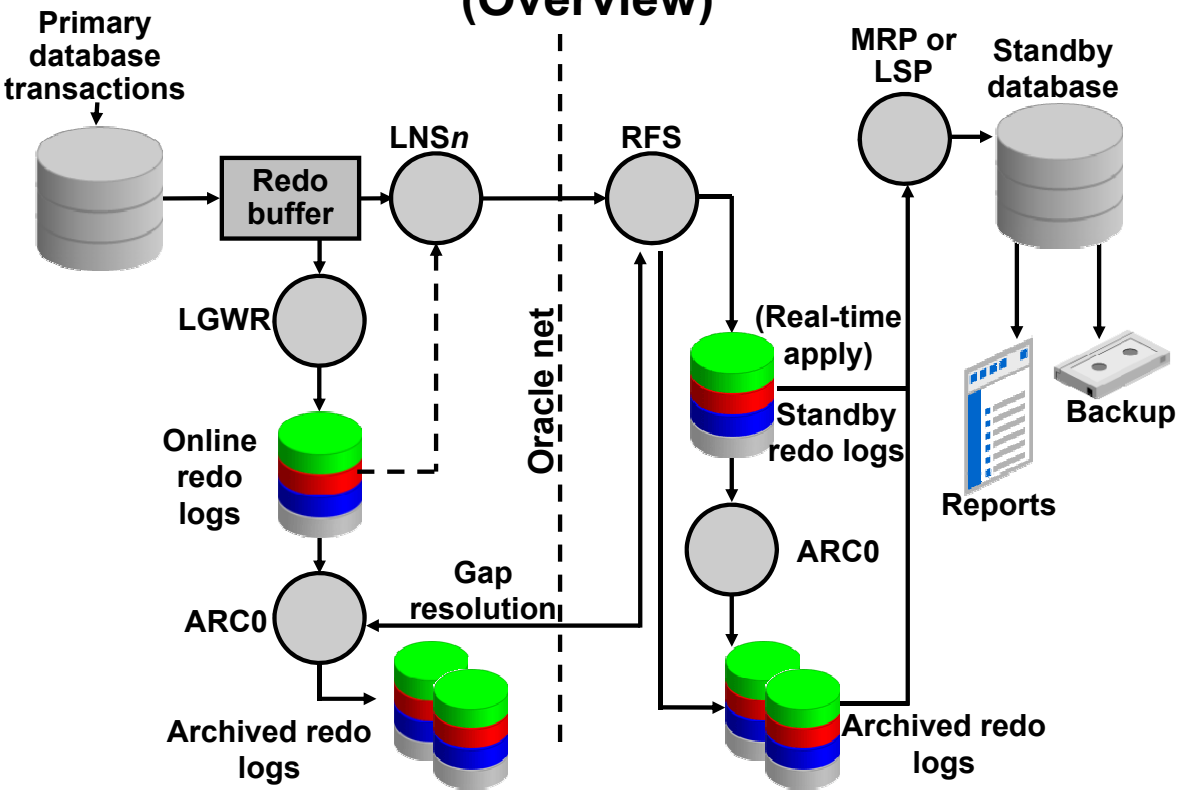
You can use Oracle Enterprise Manager Cloud Control or the Data Guard broker's own specialized command-line interface (DGMGRL) to take advantage of the broker's management capabilities.

Enterprise Manager Cloud Control provides a web-based interface that combines with the broker's centralized management and monitoring capabilities so that you can easily view, monitor, and administer primary and standby databases in a Data Guard configuration.

You can also use DGMGRL to control and monitor a Data Guard configuration. You can perform most of the activities that are required to manage and monitor the databases in the configuration from the DGMGRL prompt or in scripts.

If you do not create a Data Guard broker configuration, you can manage your standby databases by using SQL commands.

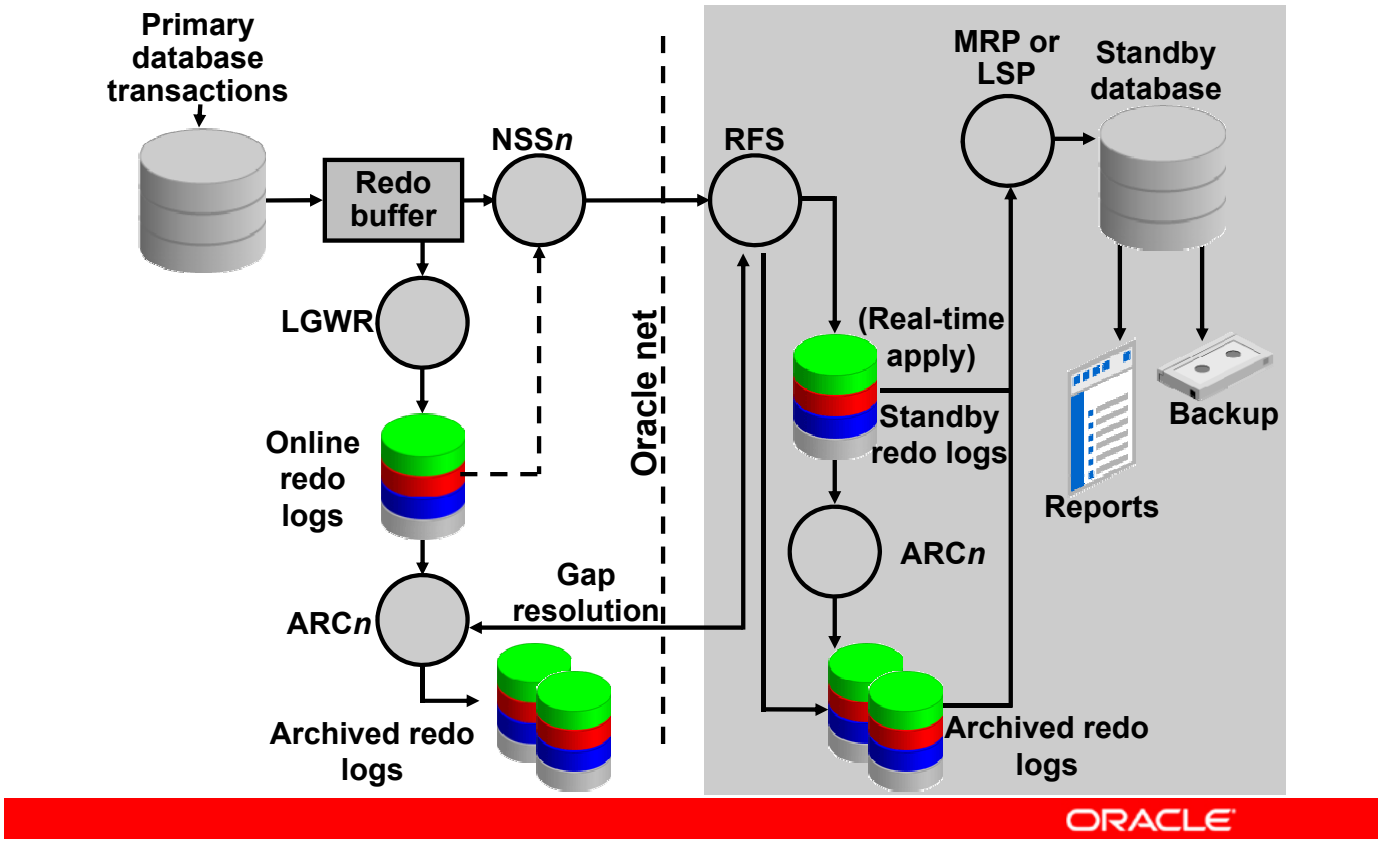
Oracle Data Guard: Architecture (Overview)



Oracle Data Guard leverages the existing database redo-generation architecture to keep the standby databases in the configuration synchronized with the primary database. By using the existing architecture, Oracle Data Guard minimizes its impact on the primary database.

Oracle Data Guard uses several processes to achieve the automation that is necessary for disaster recovery and high availability. Some of these processes support Oracle Database in general, and other processes are specific to a Data Guard environment.

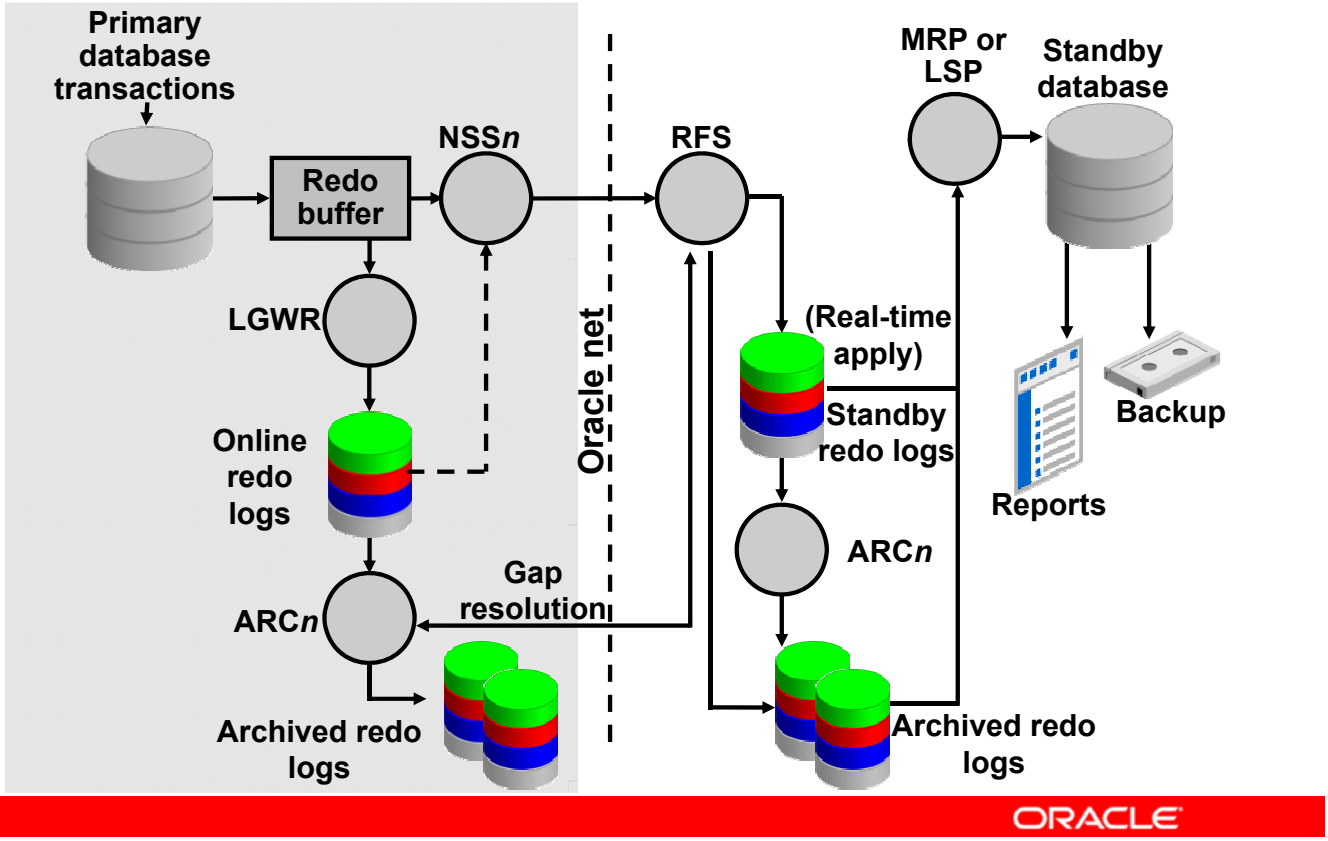
Primary Database Processes



On the primary database, Data Guard uses the following processes:

- **Log writer (LGWR):** LGWR collects transaction redo information and updates the online redo logs. For each synchronous (SYNC) standby database, LGWR passes the redo to an NSS (Network Server SYNC) process, which ships the redo directly to the remote file server (RFS) process on the standby database. LGWR waits for confirmation from the NSS process before acknowledging the commit. For asynchronous (ASYNC) standby databases, independent redo transport slave processes (TTnn) read the redo from either the redo log buffer in memory or the online redo log file, and then ship the redo to its standby database. Other than starting the asynchronous TTnn processes, LGWR has no interaction with any asynchronous standby destination.
- **Archiver (ARC*n*):** The ARC*n* process creates a copy of the online redo log files locally for use in a primary database recovery operation. ARC*n* is also responsible for shipping redo data to an RFS process at a standby database and for proactively detecting and resolving gaps on all standby databases. There can be 30 archiver processes. The default value is four.

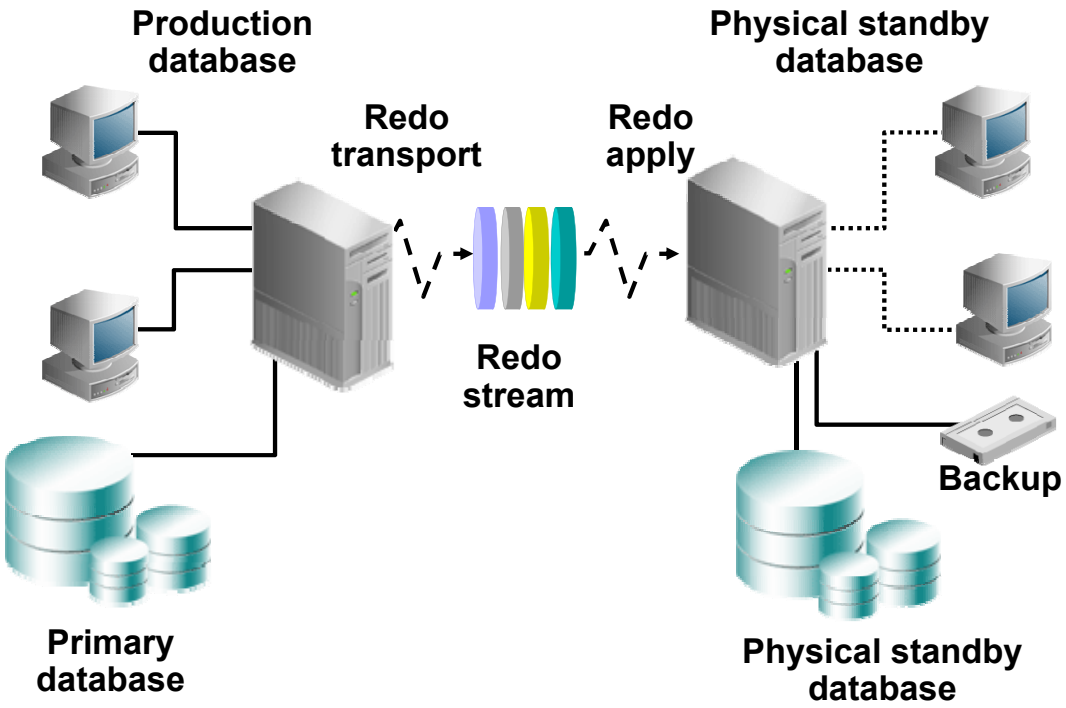
Standby Database Processes



On the standby database, Data Guard uses the following processes:

- **Remote file server (RFS):** RFS receives redo information from the primary database and can write the redo into standby redo logs or directly to archived redo logs. Each NSS*n* and ARC*n* process from the primary database has its own RFS process.
  Note: The use of standby redo logs is discussed in more detail in the lesson titled "Creating a Physical Standby Database by Using SQL and RMAN Commands."
- **Archiver (ARC*n*):** The ARC*n* process archives the standby redo logs.
- **Managed recovery (MRP):** For physical standby databases only, MRP applies archived redo log information to the physical standby database. If you start managed recovery with the `ALTER DATABASE RECOVER MANAGED STANDBY DATABASE` SQL statement, this foreground session performs the recovery. If you use the optional `DISCONNECT [FROM SESSION]` clause, the MRP background process starts. If you use the Data Guard broker to manage your standby databases, the broker always starts the MRP background process for a physical standby database.
- **Logical standby (LSP):** For logical standby databases only, LSP controls the application of archived redo log information to the logical standby database.

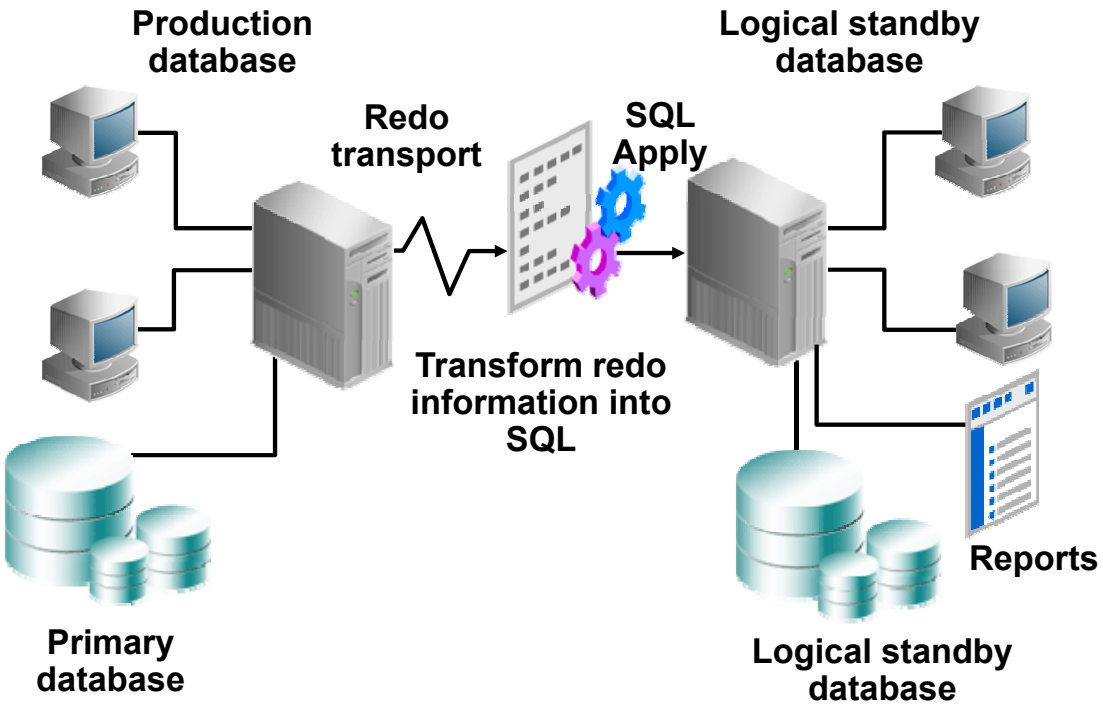
Physical Standby Database: Redo Apply Architecture



The Data Guard physical standby Redo Apply architecture consists of:

- A production (primary) database, which is linked to one or more standby databases (up to 30) that are identical copies of the production database. The limit of 30 standby databases is imposed by the `LOG_ARCHIVE_DEST_n` parameter. In Oracle Database 12*c* Release 1 (12.1), the maximum number of destinations is 31. One is used as the local archive destination, leaving the other 30 for uses such as the Far Sync or standby databases.
- The standby database, which is updated by redo that is automatically shipped from the primary database. The redo can be shipped as it is generated or archived on the primary database. Redo is applied to each standby database by using Oracle media recovery. During planned down time, you can perform a switchover to a standby database. When a failure occurs, you can perform a failover to one of the standby databases. The physical standby database can also be used to back up the primary database.

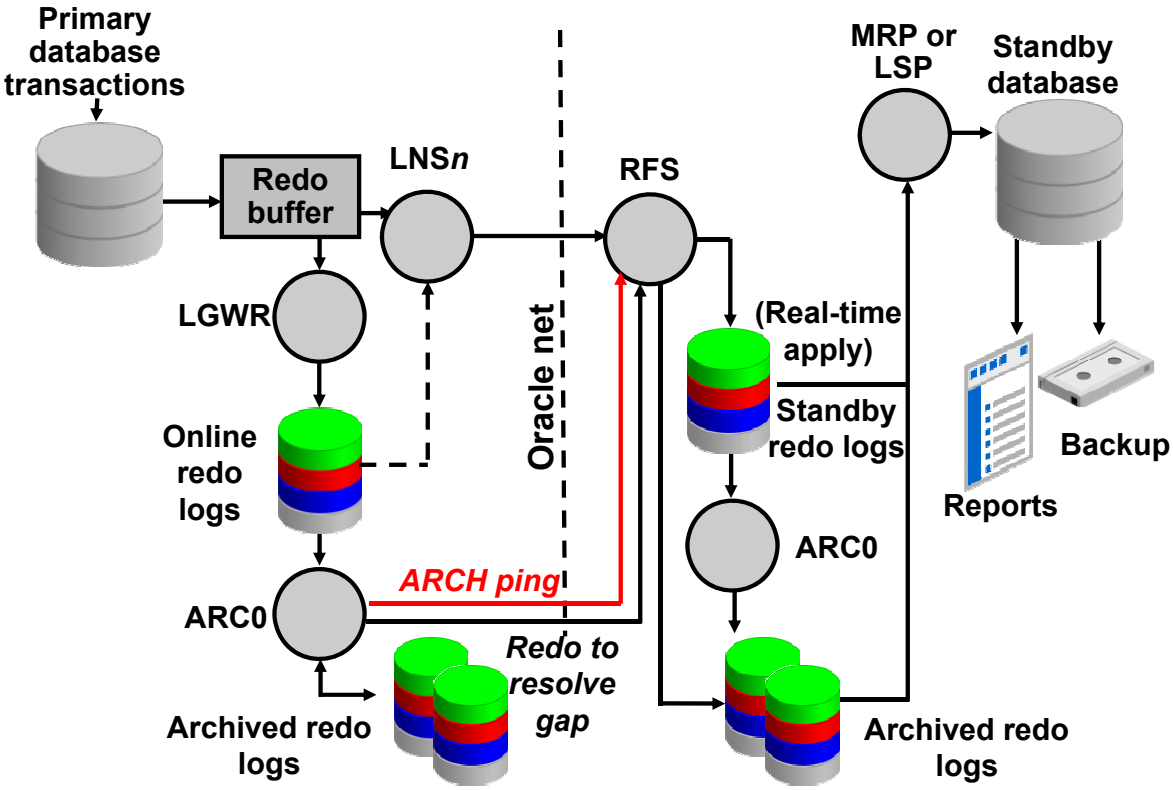
Logical Standby Database: SQL Apply Architecture



In a logical standby database configuration, Data Guard SQL Apply uses redo information shipped from the primary system. However, instead of using media recovery to apply changes (as in the physical standby database configuration), the redo data is transformed into equivalent SQL statements by using LogMiner technology. These SQL statements are then applied to the logical standby database. The logical standby database is open in read/write mode and is available for reporting capabilities. By opening the logical standby database in read/write mode, additional reporting structures such as indexes or materialized views can be created that do not exist in the primary database.

A logical standby database can be used to perform rolling database upgrades, thereby minimizing down time when upgrading to new database patch sets or full database releases.

Automatic Gap Detection and Resolution



If connectivity is lost between the primary database and one or more standby databases, redo data that is being generated on the primary database cannot be sent to those standby databases. When a connection is reestablished, Data Guard automatically detects that there are missing archived redo log files (referred to as a *gap*), and then automatically transmits the missing archived redo log files to the standby databases by using the ARC*n* processes. The standby databases are synchronized with the primary database without manual intervention by the DBA.

Data Protection Modes

Select the mode to balance cost, availability, performance, and data protection:

- Maximum protection
- Maximum availability
- Maximum performance

Data Guard provides three high-level modes of data protection that you can configure to balance cost, availability, performance, and transaction protection. You can configure the Data Guard environment to maximize data protection, availability, or performance.

Maximum Protection

This protection mode guarantees that no data loss occurs if the primary database fails. For this level of protection, the redo data that is needed to recover each transaction must be written to both the local online redo log and the standby redo log (used to store redo data received from another database) on at least one standby database before the transaction commits. To ensure that data loss does not occur, the primary database shuts down if a fault prevents it from writing its redo stream to at least one remote standby redo log.

Maximum Availability

This protection mode provides the highest possible level of data protection without compromising the availability of the primary database. As with maximum protection mode, a transaction does not commit until the redo needed to recover that transaction is written to the local online redo log and to at least one remote standby redo log. Unlike maximum protection mode, the primary database does not shut down if a fault prevents it from writing its redo stream to a remote standby redo log. Instead, the primary database operates in an unsynchronized mode until the fault is corrected and all the gaps in the redo log files are resolved. When all the gaps are resolved and the primary database is synchronized with the standby database, the primary database automatically resumes operating in maximum availability mode.

This mode guarantees that no data loss occurs if the primary database fails, but only if a second fault does not prevent a complete set of redo data from being sent from the primary database to at least one standby database.

Maximum Performance (Default)

The default protection mode provides the highest possible level of data protection without affecting the performance of the primary database. This is accomplished by allowing a transaction to commit as soon as the redo data needed to recover that transaction is written to the local online redo log. The primary database's redo data stream is also written to all ASYNC standby databases and is written asynchronously with respect to the commitment of the transactions that create the redo data.

Data Guard Operational Requirements: Hardware and Operating System

Primary database systems and standby database systems may have different:

- CPU architectures
- Operating systems
- Operating system binaries (32-bit or 64-bit)
- Oracle Database binaries (32-bit or 64-bit)

Many restrictions exist.

Hardware and Operating System Requirements

These are the requirements for Data Guard:

- The hardware for the primary and standby database systems can be different. For example, the number of CPUs, the memory size, and the storage configuration can differ.
- The operating systems for both databases and operating system binaries can be different.

If the primary and standby databases are both on the same server, you must ensure that the operating system enables you to mount two databases with the same name on the same system simultaneously. Certain parameters must be specified to support this configuration, as discussed in the lesson titled “Creating a Physical Standby Database by Using SQL and RMAN Commands.”

Refer to My Oracle Support notes 413484.1, 395982.1, and 414043.1 for additional information.

Data Guard Operational Requirements: Oracle Database Software

- The same release of Oracle Database Enterprise Edition must be installed for all databases except when you perform a rolling database upgrade by using a logical standby database.
- If any database uses ASM or OMF, all databases should use the same combination.

ORACLE

- You must install the same release of Oracle Database Enterprise Edition for the primary database and all standby databases in your Data Guard configuration. Oracle Data Guard is available only as a feature of Oracle Database Enterprise Edition; it is not available with Oracle Database Standard Edition.

  Note: See the documentation titled *Oracle Data Guard Concepts and Administration* for information about simulating a standby database environment when using Oracle Database Standard Edition.
- If you use Oracle Automatic Storage Management (ASM) and Oracle Managed Files (OMF) in a Data Guard configuration, you should use ASM and OMF symmetrically on the primary and standby database. If any database in your Data Guard configuration uses ASM, OMF, or both, then every database in the configuration should use the same combination (that is, ASM, OMF, or both).

  Note: An exception to this guideline is if you are using Data Guard as a technique to migrate to ASM and/or OMF.

Benefits of Implementing Oracle Data Guard

Oracle Data Guard provides the following benefits:

- Continuous service during disasters or crippling data failures
- Complete data protection against corruption and data loss
- Elimination of idle standby systems
- Flexible configuration of your system to meet requirements for business protection and recovery
- Centralized management

ORACLE

- **Continuous service:** With the use of switchover and failover between systems, your business does not need to halt because of a disaster at one location.
- **Complete data protection:** Data Guard guarantees that there is no data loss and provides a safeguard against data corruption and user errors. Redo data is validated when applied to the standby database.
- **Elimination of idle standby systems:** Standby databases can be used for reporting and ad hoc queries in addition to providing a safeguard for disaster recovery. You can also use the physical standby database to off-load backups of the primary database.
- **Flexible configurations:** You can use Data Guard to configure the system to your needs by using the protection modes and several tunable parameters.
- **Centralized management:** You can use Enterprise Manager Cloud Control to manage all configurations in your enterprise.

Quiz

Which of the following are types of standby databases?

a. Physical
b. Primary
c. Logical
d. Snapshot

Answer: a, c, d

Quiz

What is the maximum number of standby databases supported by the Data Guard Broker?

a. 10
b. 20
c. 30
d. 40

Answer: c

Summary

In this lesson, you should have learned how to:

- Describe the basic components of Oracle Data Guard
- Explain the differences between physical and logical standby databases
- Explain the benefits of creating a Data Guard environment

Practice 1: Overview

This practice covers the following topics:

- Introducing the lab environment
- Starting and stopping a virtual machine
- Starting and stopping the Oracle Management Server
- Starting and stopping the Oracle Agent
- Accessing Oracle Enterprise Manager

2

Creating a Physical Standby Database by Using Enterprise Manager Cloud Control

Objectives

After completing this lesson, you should be able to use Oracle Enterprise Manager Cloud Control 12*c* to:

- Create a Data Guard broker configuration
- Create a physical standby database
- Verify a Data Guard configuration
- Edit database properties related to Data Guard
- Test a Data Guard configuration

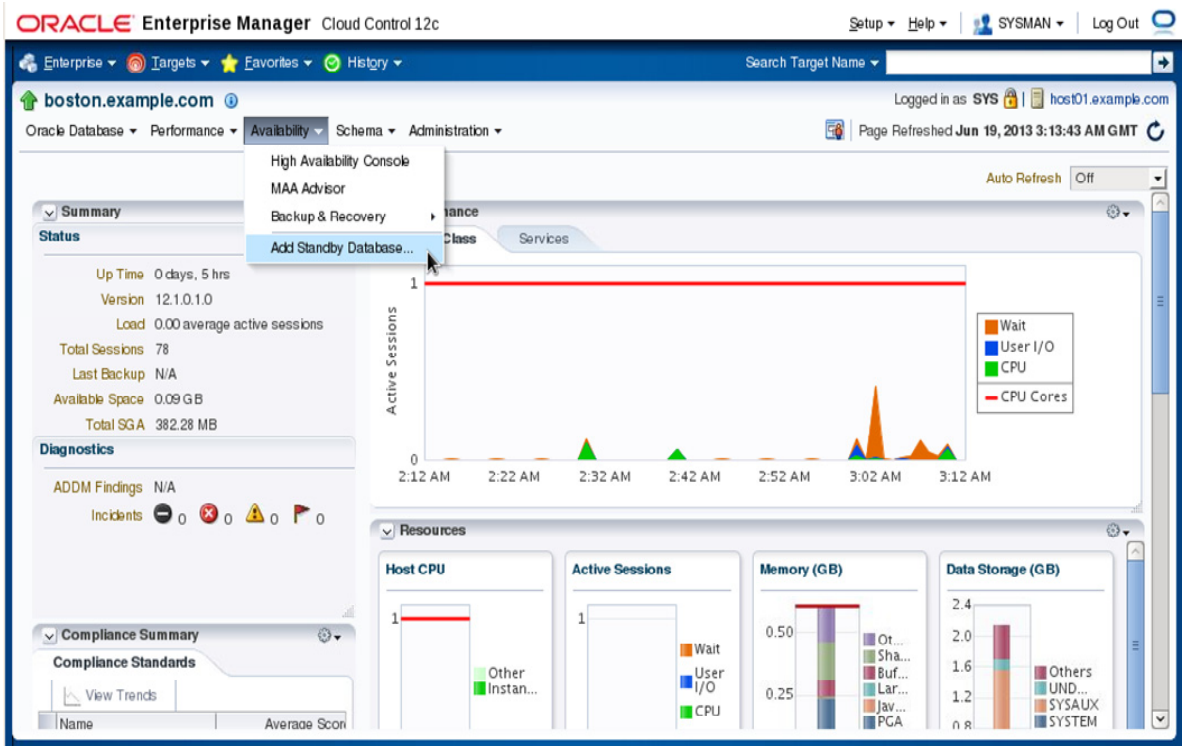
Prerequisites for Using Oracle Enterprise Manager Cloud Control 12*c*

To use Oracle Enterprise Manager Cloud Control 12*c* for Data Guard, the following prerequisite steps must be performed:

- Install Oracle Enterprise Manager Cloud Control 12*c*
- Install agents on each host in the Data Guard environment
- Install Oracle Database software binaries on each host in the Data Guard environment
- Deploy the Oracle Database plug-in to each remote agent
- Verify the agent status on each host
- Define preferred credentials for each host
- Create or identify the primary database

Oracle Enterprise Manager Cloud Control 12*c* can automate much of the process of creating both physical and logical standby databases. Before using Oracle Enterprise Manager for a Data Guard environment, some prerequisite steps must be performed. At a minimum, Oracle Enterprise Manager should be installed and configured on a server host. For each additional server that is to participate in a Data Guard environment, the software binaries for both the Oracle Database and the Oracle Enterprise Manager agent must be installed. These are installed into distinct Oracle Home locations. A default installation of the Oracle Enterprise Manager agent does not install plug-ins to the agent. Deploy the latest version of the Oracle Database plug-in to each agent and verify the status of the agents. Create a basic security configuration by defining named credentials for normal and privileged users for the host target type. Define individual target credentials for each host by referencing the named credentials.

Accessing the Add Standby Database Wizard



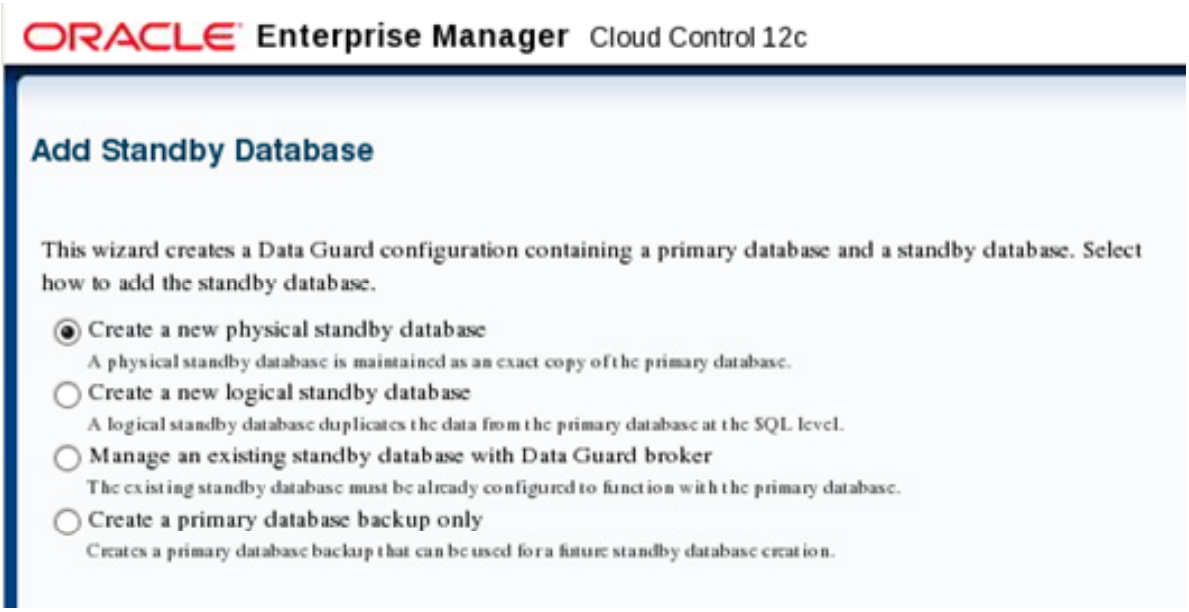
The screenshot in the slide shows the home page for the primary database, with the top level menu structure containing the following items: Oracle Database, Performance, Availability, Schema, and Administration.

To access the Add Standby Database wizard, navigate to the primary database target and select Add Standby Database from the Availability menu. The Add Standby Database menu item is the only Data Guard option on the Availability menu if a Data Guard configuration does not already exist. After a Data Guard configuration is defined, the Availability menu will also contain menu items for Data Guard Administration, Data Guard Performance, and Verify Data Guard Configuration.

Before you invoke the Add Standby Database wizard, verify that the primary database instance was started with a server parameter file (SPFILE) and that archiving is enabled.

Note: Actual webpages may differ with each version of Oracle Enterprise Manager. The screenshots used in this lesson were taken with Oracle Enterprise Manager Cloud Control 12*c* Release 2 with patch set update (PSU) two installed (12.1.0.2.2).

Accessing the Add Standby Database Wizard



The above webpage snippet shows the Add Standby Database wizard after logging in for the first time. It provides the following options:

- Create a new physical standby database
- Create a new logical standby database
- Manage an existing standby database with Data Guard broker
- Create a primary database backup only

If you are not connected to the primary database when you select Add Standby Database from the Availability menu, a Database Login window appears. You will then be able to select from an existing named credential, or you can create a new credential. You must be connected to the primary database with `SYSDBA` credentials to use the Add Standby Database wizard.

Creating a New Standby Database

The standby database creation process performs the following:

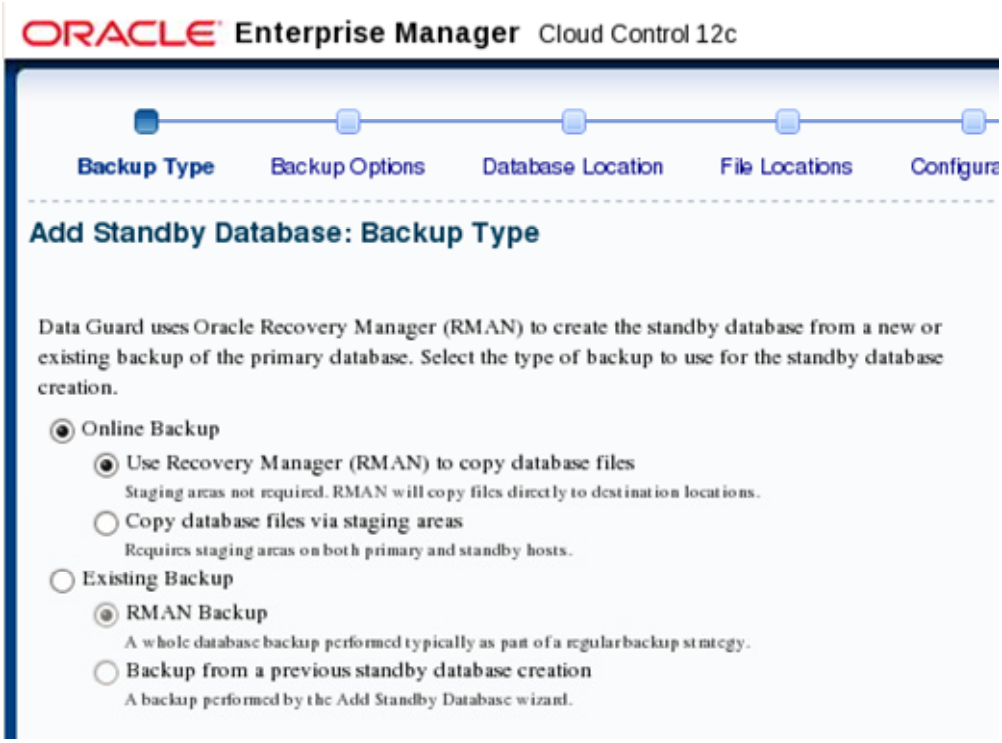
- Performs an online backup (or optionally uses an existing backup) of the primary database control file, datafiles, and archived redo log files
- Transfers the backup pieces from the primary host to the standby host
- Creates other needed files (for example, initialization and password files) on the standby host
- Restores the control files, datafiles, and archived redo log files to the specified locations on the standby host
- Adds online redo logs and other files to the standby database as needed
- Configures the recovered database as a physical or logical

ORACLE

The Add Standby Database wizard can create both physical and logical standby databases. In addition to the task listed in the slide, the wizard will prepare the primary database to support Data Guard operations including additional tasks such as:

- Creating standby redo logs on the primary if needed
- Starting the Data Guard broker process
- Creating the Data Guard broker configuration files
- Adding primary and standby database to the Data Guard broker configuration
- Enabling the Data Guard broker configuration
- Adjusting initialization parameters such as `LOG_ARCHIVE_CONFIG`, `LOG_ARCHIVE_DEST_`*n*, and `STANDBY_FILE_MANAGEMENT`.

Creating a Physical Standby Database
Step 1: Backup Type



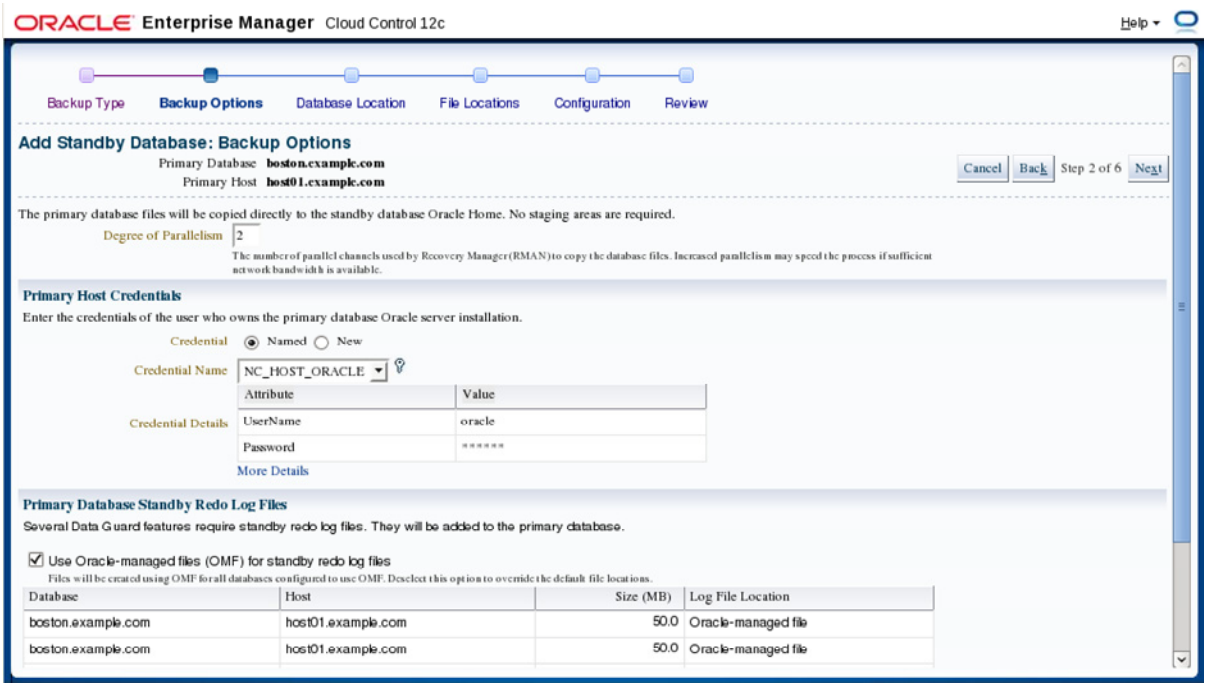
The screenshot in the slide shows step one of six steps in creating a physical standby database. Step one defines the type of backup to be used for creating the standby database—either one that you create by performing an online backup, or an existing backup. Data Guard uses Oracle Recovery Manager (RMAN) to perform the backup.

The online backup option can be performed using a direct copy approach that does not require staging areas, or it can utilize staging directories. The staging directories can be retained for future standby database creations, or they can be deleted after the standby database is created. A backup performed into a staging area can be compressed to reduce file size and transfer time, but it may also slow down datafile backup and restoration.

The existing backup option requires a level 0 (full) or level 1 (incremental) RMAN backup that is typically performed as part of a regular backup strategy. You will be prompted for both the existing RMAN backup location and a new staging area location on the primary database host if you select this option.

Note: The actual Enterprise Manager webpages will vary depending on the options selected. For example, if the Add Standby Database wizard is used to create a logical standby database, this step would also show all of the tables that are unsupported by SQL Apply.

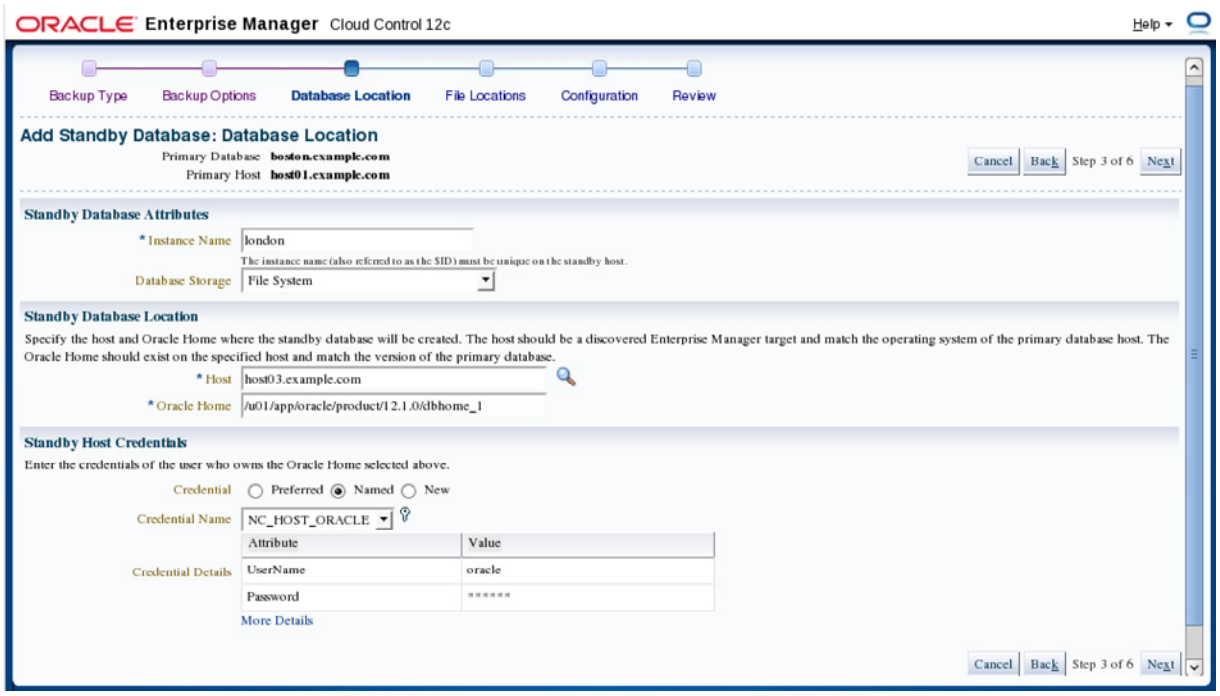
Creating a Physical Standby Database
Step 2: Backup Options



The screenshot in the slide shows step two of six steps in creating a physical standby database. Step two is titled “Backup Options.” For an online direct backup, the degree of parallelism is specified as a backup option. In this step, you must supply the primary host credentials, using either an existing named credential or by creating a new credential. If the primary database does not currently have standby redo logs, this step will also require that they be created. Oracle Enterprise manager will automatically determine the correct size and number of groups to create for standby redo logs. You can select the option to use Oracle-managed files (OMF) for the standby redo log files or you can explicitly name and locate the standby redo logs that will be created on the primary database.

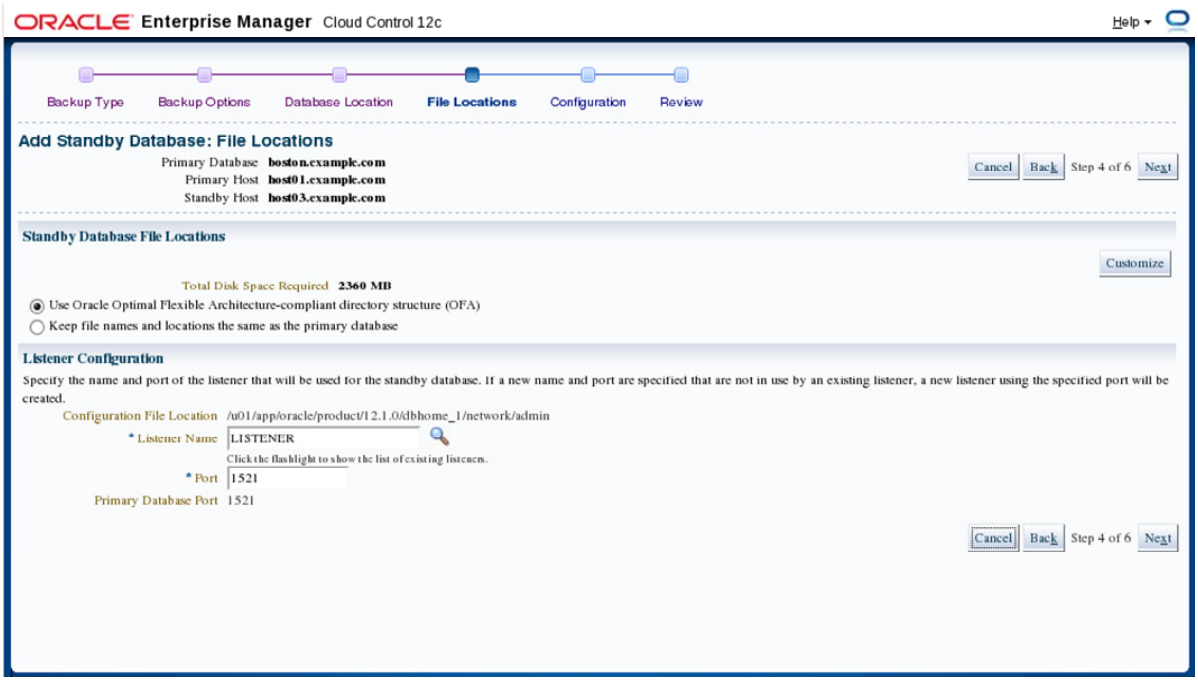
For backup types other than online direct backups, you supply the staging area locations, existing RMAN backup locations, backup compression features, and whether to retain the backups for future use depending on which backup method was chosen.

Creating a Physical Standby Database
Step 3: Database Location



The screenshot in the slide shows step three of six steps in creating a physical standby database. Step three is titled “Database Location.” This step contains three sections. In the Standby Database Attributes section, you define the instance name of the standby database and whether the standby database will use file system storage or Automatic Storage Management (ASM). In the Standby Database Location section, you define the host name for the server to which the standby database will be transferred and the existing Oracle home software location on that server. In the Standby Host Credentials section, you specify host credentials to be used on the standby server machine (you can use existing named credentials or you may create new credentials).

Creating a Physical Standby Database
Step 4: File Locations



ORACLE

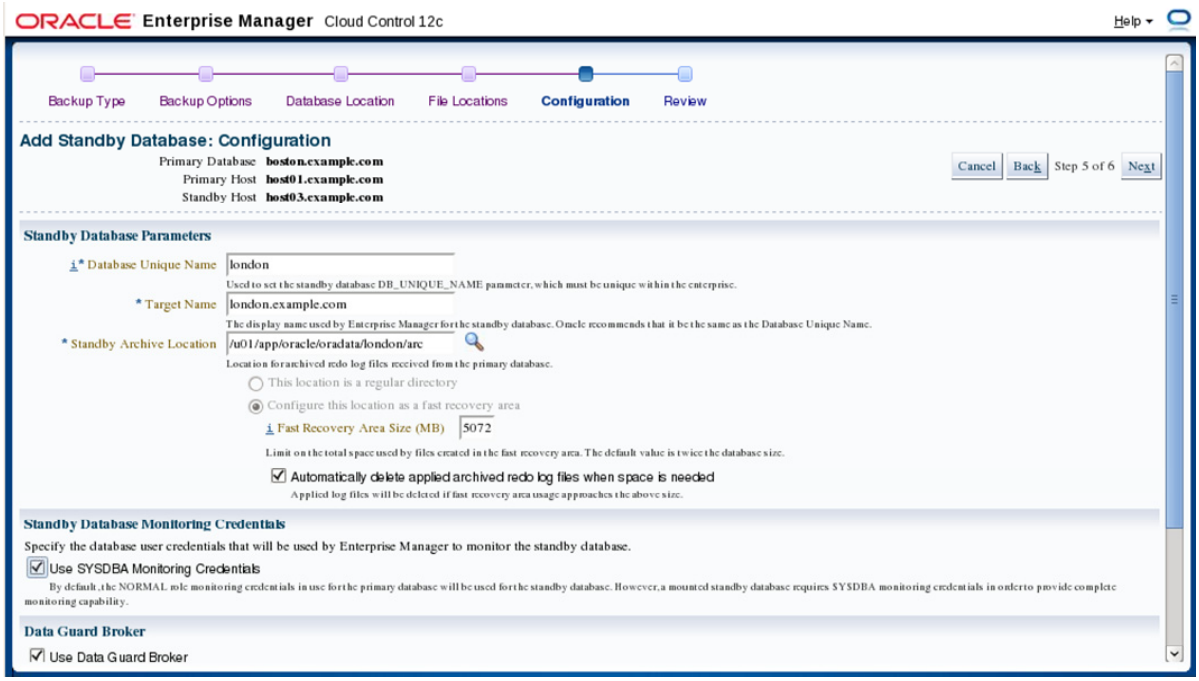
The screenshot in the slide shows step four of six steps in creating a physical standby database. Step four is titled “File Locations.” This step contains two sections, Standby Database File Locations and Listener Configuration. If a file system was selected in step three as the database storage location, the Standby Database File Location section of this window allows you to keep the same files names and locations as the primary database uses, or allows you to create a new directory structure compliant with Oracle Optimal Flexible Architecture (OFA). If ASM was chosen in step three as the database storage location, Oracle Enterprise Manager will prompt you to log in to the ASM instance to determine available ASM disk groups and allow you to specify which ASM disk groups to use for storage. With either option selected, a Customize button in this section allows you to view all individual datafiles, tempfiles, logfiles, control files, directory objects, and external files, and to define specific names and locations.

If staging directories had been selected earlier, an additional section, Standby Host Backup File Access, appears. This allows the specific directory on the standby host machine to be defined along with the file transfer method, either HTTP/S or FTP. There is also an option for the standby database to access the files directly from a shared directory such as an NFS mount.

In the Listener Configuration section of this step, you specify the listener name and port number for the listener. A new listener will be created if the specified listener name and port does not already exist.

Note: When the Next button is clicked, a warning message will appear if any files for external tables located on the primary database host already exist on the standby database host. These files will be overwritten by the Add Standby Database wizard.

Creating a Physical Standby Database
Step 5: Configuration



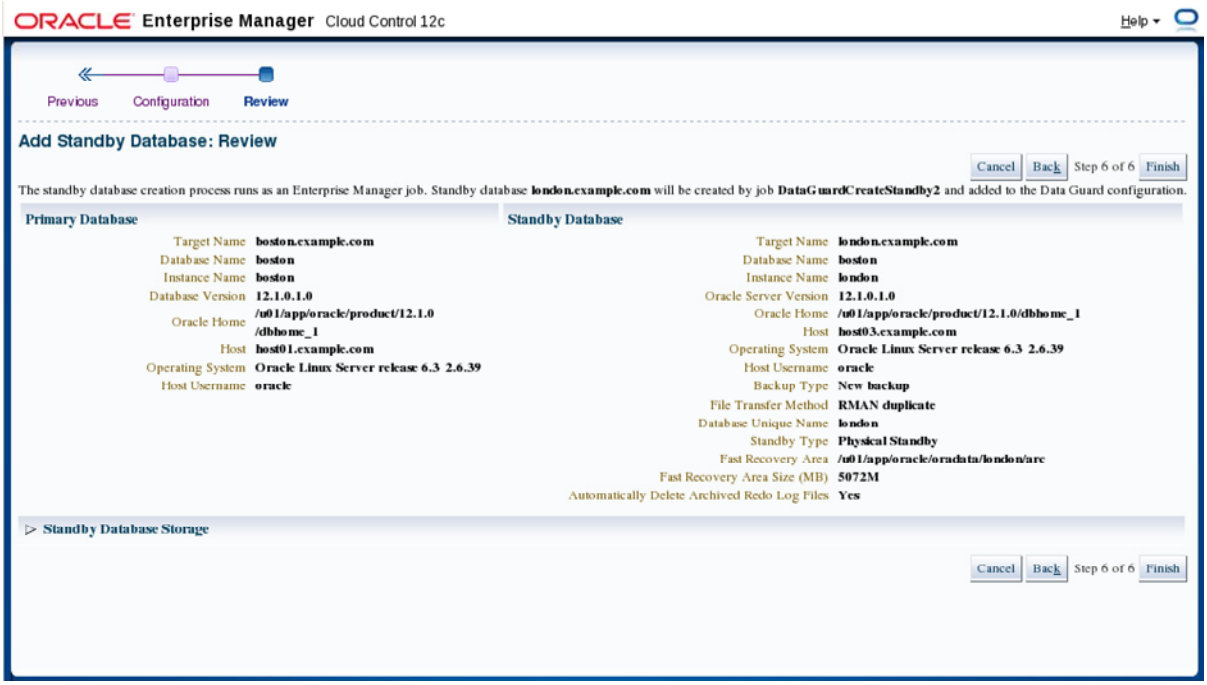
The screenshot in the slide shows step five of six steps in creating a physical standby database. Step five is titled “Configuration.” This step contains four sections. The first section, Standby Database Parameters, prompts for the standby `DB_NAME` parameter (for logical standby databases only), standby `DB_UNIQUE_NAME` parameter, `STANDBY_ARCHIVE_DEST` parameter, and `FAST_RECOVERY_AREA_SIZE` parameter. You can also select to delete applied archive redo logs when space is needed.

The Standby Database Monitoring Credentials section of this step allows you to override normal default monitoring credentials and specify `SYSDBA` credentials for monitoring. This is required for a mounted standby database.

The Data Guard Broker section is only displayed if no existing broker configuration is found. It allows you to delete the broker configuration after the standby database is created if desired. If there is an existing broker configuration, this section is hidden.

The Data Guard Connect Identifiers section allows you to specify connect identifiers for both the primary database and standby database. You can use Enterprise Manager connect descriptors that are explicitly coded into the `LOG_ARCHIVE_DEST_`*n* parameters, or specify an existing net service name that can be used by all databases. If a connect identifier for the primary database has already been created previously for other standby databases, you will not be prompted for it again here.

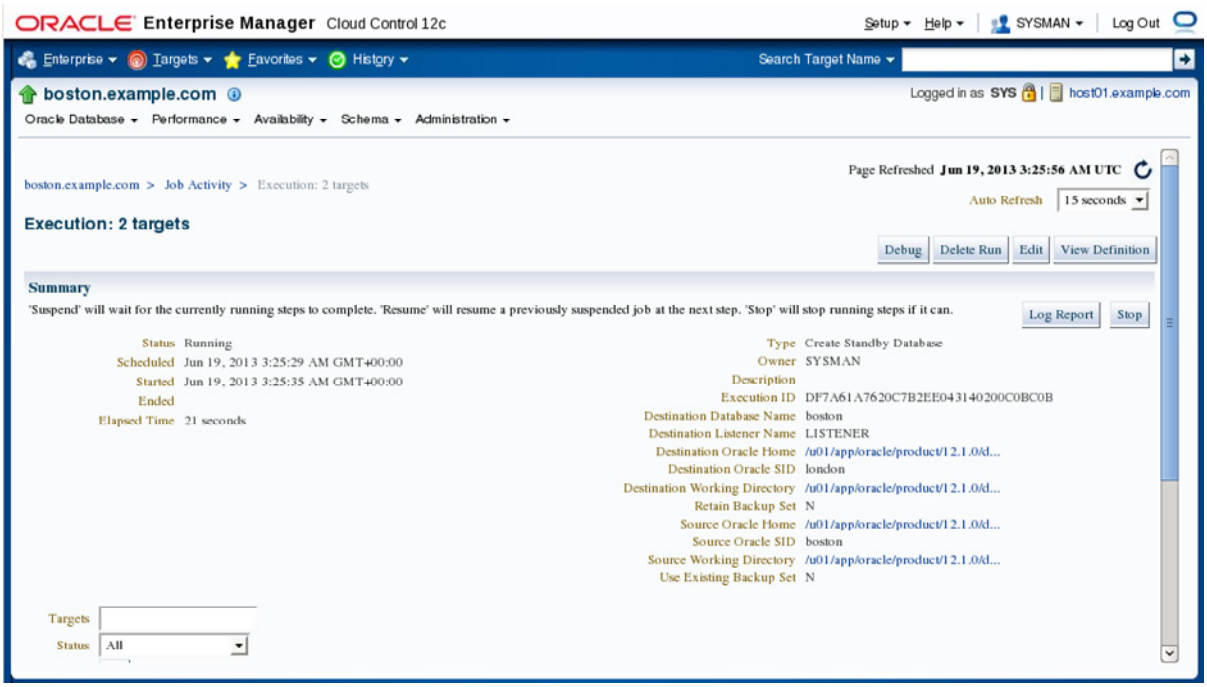
Creating a Physical Standby Database
Step 6: Review



The screenshot in the slide shows step six of six steps in creating a physical standby database. Step six is titled “Review” and it provides a recap of many of the settings entered on the previous five steps. The Standby Database Storage section can be expanded to show all individual datafiles, tempfiles, logfiles, control files, directory objects, and external files that will be created on the standby database host. If everything is satisfactory, click Finish to create a job to perform the tasks.

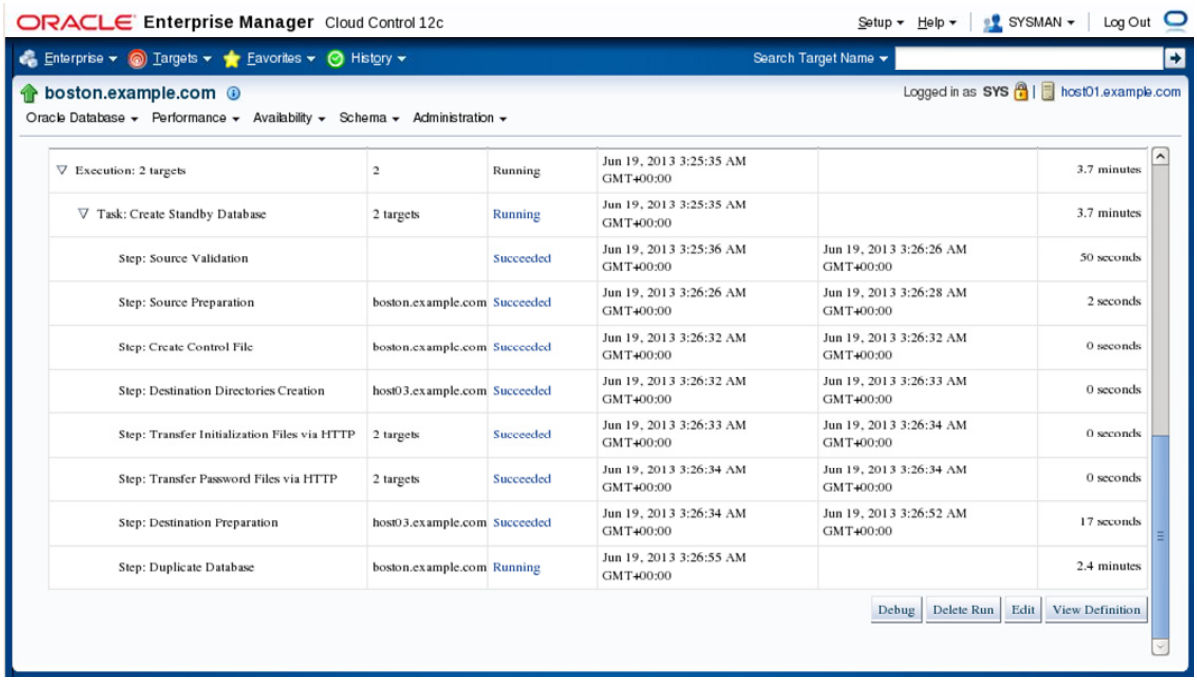
Note: There is no option to schedule the creation job to run at a future time. It will be an immediate job submitted when the Finish button is clicked.

Creating a Physical Standby Database
Job Activity



When you click the Finish button in step six of the Add Standby Database wizard, an Oracle Enterprise Manager job is created and an information dialog window appears that provides a URL link to view the job. The screenshot in the slide shows the job activity window for the job that was created. The default refresh mode for this window is manual refresh. Change the auto refresh option to one of the available time frequency choices to see changes happen while they are in progress or manually click the refresh icon when desired. The Log Report button will show the output log for all SQL*Plus, RMAN, and Listener Control Utility (`lsnrctl`) utility commands used. The Debug button will provide more details in the logs, including the actual RMAN script that was used to duplicate the primary database.

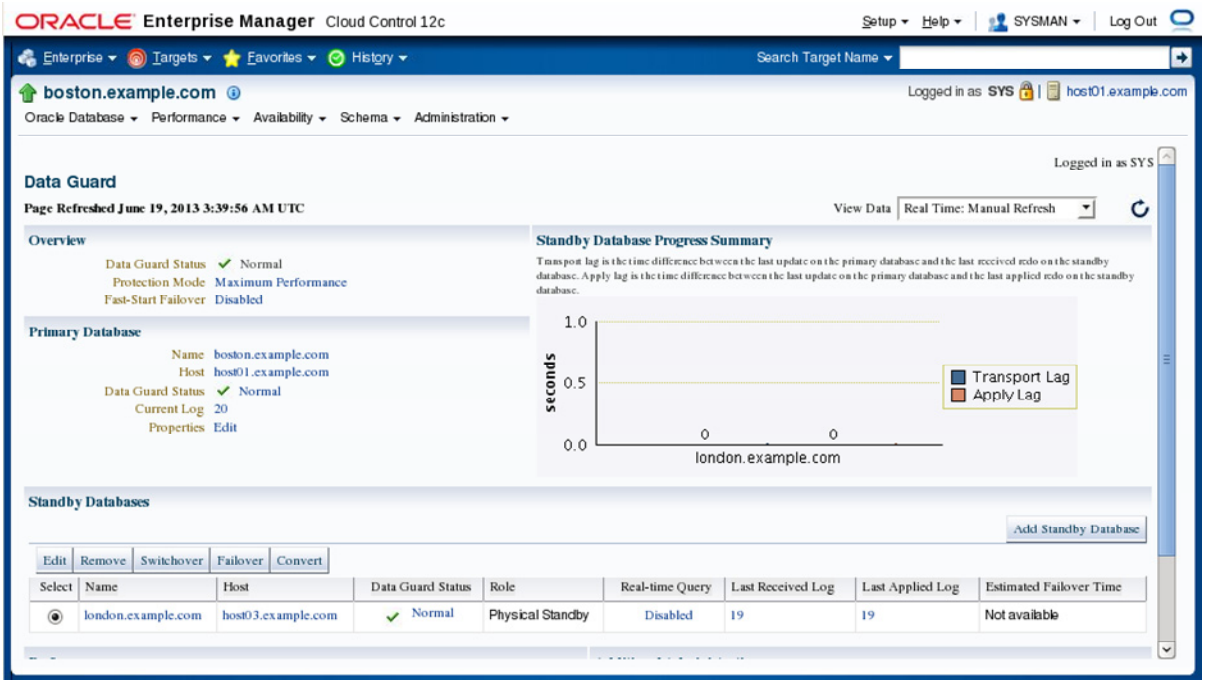
Creating a Physical Standby Database
Job Task Details



ORACLE

The screenshot in the slide shows the job activity window. The scroll bar on the right side of the window has been moved down to show individual job steps of the Create Standby Database job. Each row or job step shows the target machine that the job step was run on, the status of the job step, the start time of the job step, the end time of the job step, and a calculated elapsed time for the job step.

Data Guard Administration Page



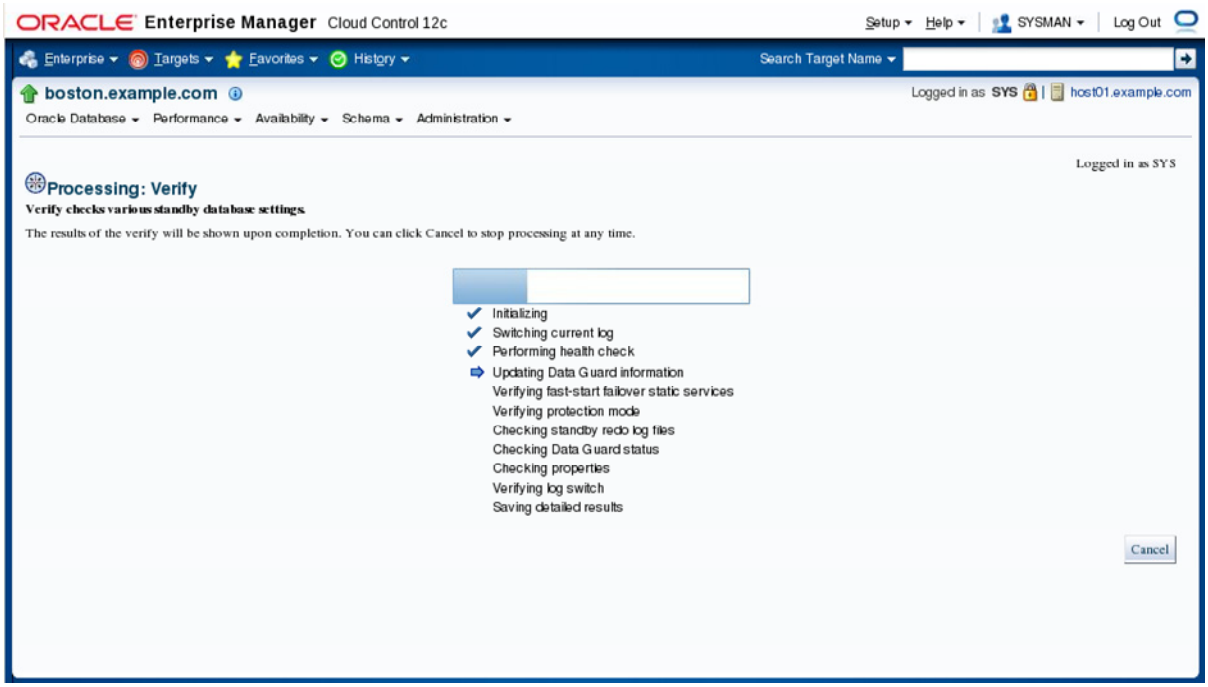
ORACLE

After a Data Guard configuration has been defined, the Availability menu contains both Add Standby Database and Data Guard Administration items. Clicking either one while on the primary database home page opens the Data Guard administration page.

The screenshot in the slide shows the Data Guard administration page. The Overview section on this page displays the Data Guard status for the configuration, the protection mode, and whether Fast-Start Failover has been enabled. The Primary Database section of this page shows the hostname of the primary database, current log sequence number, and an option to edit primary database properties. The Standby Databases section of this page shows all standby databases that are configured for this environment along with properties about each database, such as name, host machine on which it resides, status, role, whether real-time query is enabled, last received redo log, last applied redo log, and an estimated failover time for that particular standby database.

A chart on the Data Guard Administration page graphs both transport lag and apply lag for each standby database. Transport lag is the time difference between the last update on the primary database and the last received redo on the standby database. Apply lag is the time difference between the last update on the primary database and the last applied redo on the standby database. An Add Standby Database button on this page is used to create additional standby databases, and links at the bottom navigate to performance monitoring and verification.

Verifying Configuration



The screenshot in the slide shows the progress indicator of the Verify Configuration administrative task that can be invoked from the Data Guard administration page. The verify configuration task performs a series of validation checks on the Data Guard configuration, including a health check of each database and agent. The Verify Configuration operation:

- Determines the current data protection mode settings, including the current redo transport mode settings for each database and whether the standby redo logs are configured properly. If standby redo logs are needed for a database, a message indicates this on the Detailed Results page. You can then add the standby redo logs.
- Validates each database for the current status
- Performs a log switch on the primary database (for non-RAC databases) and verifies that the log was applied on each standby database
- Checks the agent status for each database. The verify process executes a SQL*Plus job on the agent if credentials are available. If credentials are not available to run the job, the agent is pinged instead. If errors occur, a message appears on the Results page.
- Displays the results of the Verify Configuration operation (including errors)

Note: You can cancel the Verify Configuration operation at any time by clicking Cancel.

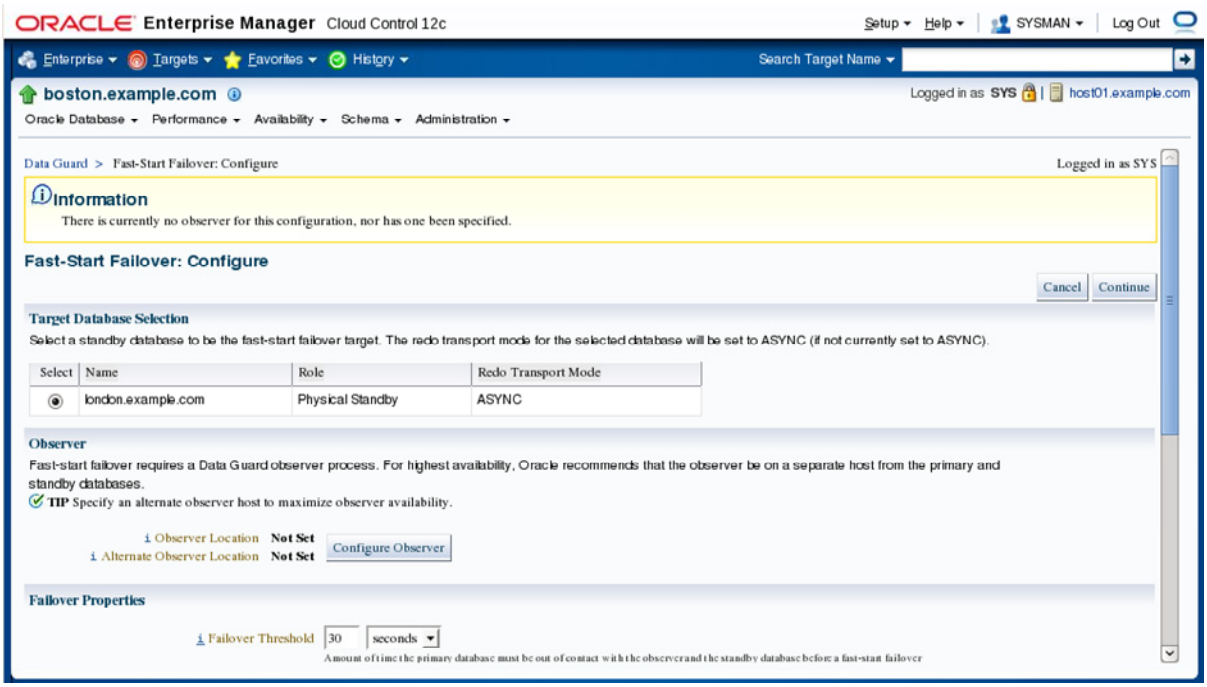
Performing Routine Maintenance

Use Enterprise Manager Cloud Control 12*c* to:

- Configure Fast-Start Failover
- Change the properties of a primary database
- Change the properties of a standby database
- Run a test application to generate a workload
- Manage apply services
- Set the redo transport mode

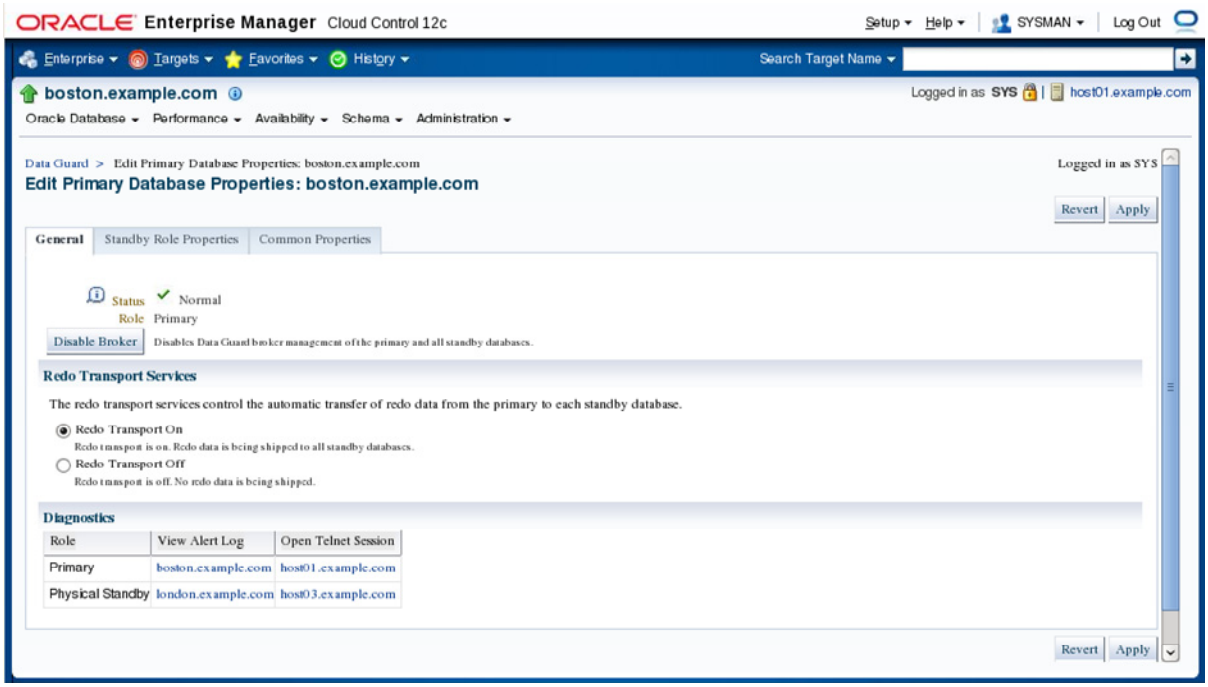
You can use Enterprise Manager Cloud Control 12*c* to maintain your broker configuration. Each task is described in detail in the next few slides.

Configuring Fast-Start Failover



Enterprise Manager Cloud Control 12*c* can manage the Fast-Start Failover feature for Data Guard. This feature will be discussed in more detail in lesson titled “Enabling Fast Start Failover.” The screenshot in the slide shows the Configure page for Fast-Start Failover. It allows you to specify which standby database Fast-Start failure will use as the failover target, which host and alternate host will be used for the Data Guard observer process, failover properties, and primary database properties related to the Fast-Start Failover configuration.

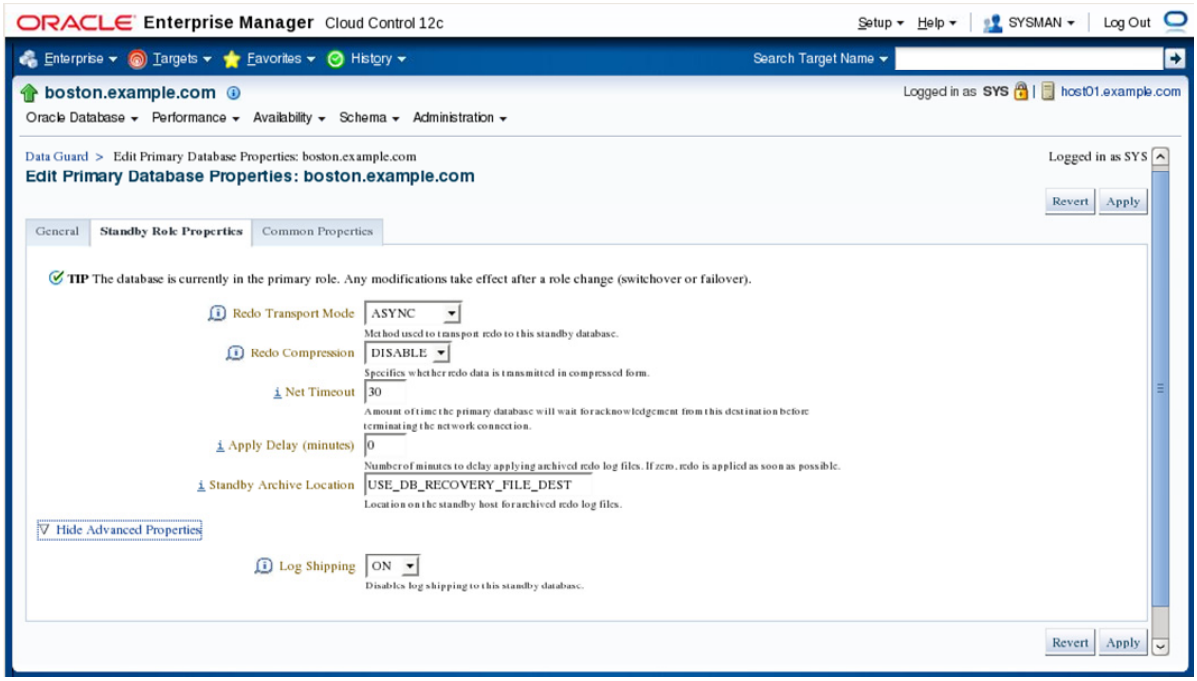
Editing Primary Database Properties: General Properties



ORACLE

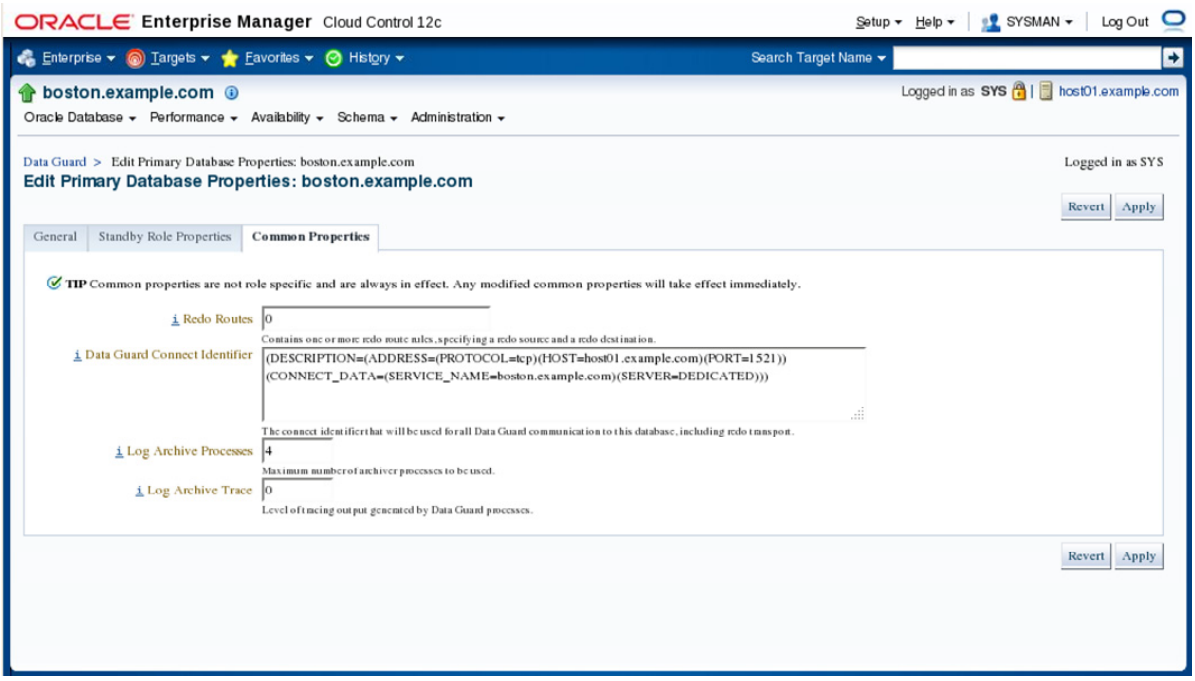
Enterprise Manager Cloud Control 12*c* can manage primary database properties related to the Data Guard configuration. In order to access the Edit Primary Database Properties page, navigate to the Data Guard administration home page of the primary database and click the Edit link beside the properties label of the primary database section. The screenshot in the slide shows the General tab for editing primary database properties. There is also a Standby Role Properties tab and a Common Properties tab. The General tab allows the broker to be disabled or enabled, redo transport to be started or stopped, and allows viewing of the alert log of the primary database and standby database.

Editing Primary Database Properties: Standby Role Properties



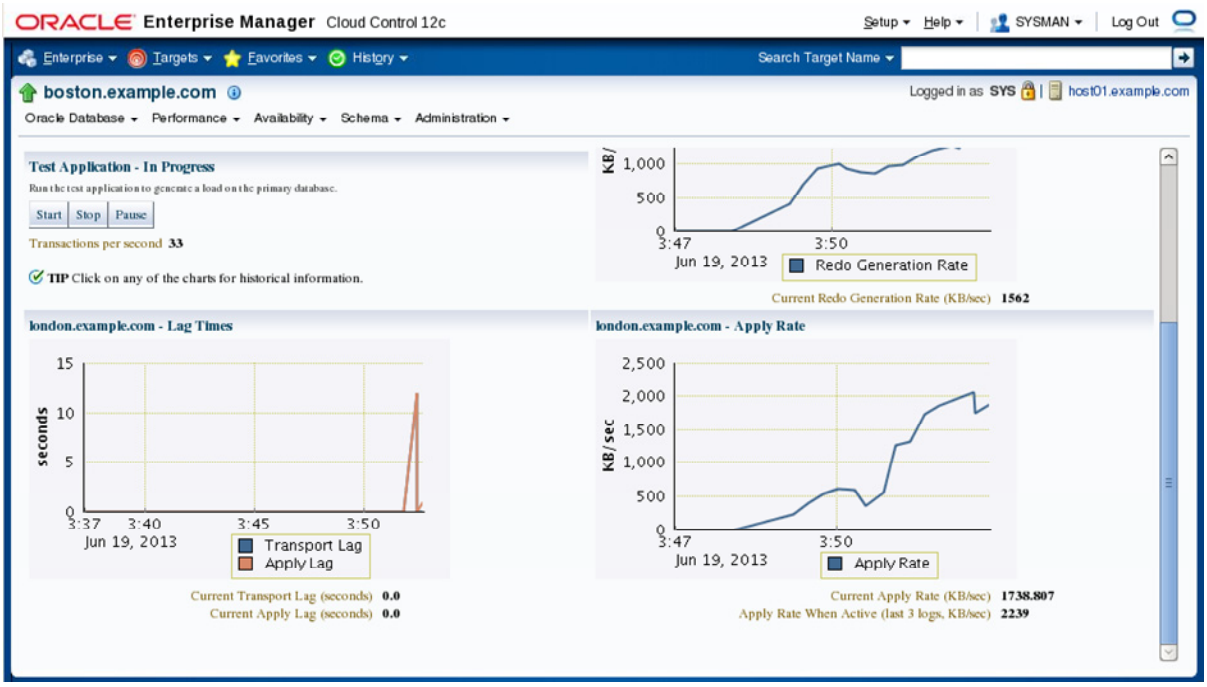
Additional Data Guard parameters that can be edited for the primary database can be found on the Standby Role properties tab and the common properties tab. The screenshot in the slide shows the Standby Role Properties tab found on The Edit Primary Database Properties page. On this tab you can set the redo transport mode to `SYNC`, `FASTSYNC`, or `ASYNC`, the Redo Compression setting to `ENABLE` or `DISABLE`, and you can specify the number of seconds for Net Timeouts and the number of minutes for the Apply Delay. You can also specify the Standby Archive location and turn on or turn off log shipping to the standby database. The setting for standby archive location will only take effect after a role reversal, which allows the current primary database to become a standby database.

Editing Primary Database Properties: Common Properties



The screenshot in the slide shows the Common Properties tab of the Edit Primary Database Properties page. On this tab, you can define redo routes, the Data Guard connect identifier for the primary database, the number of log archive process, and the trace level for Data Guard processes.

Test Application



The test application feature of Oracle Enterprise Manager on the Data Guard administration webpage allows a thorough testing of Data Guard by generating a workload on the primary database. This workload will generation many transactions per second, cause log switching at the primary, and monitor the reception of the redo on the standby database and the application of the received redo. The screenshot in the slide shows the test application page in progress of a running test. You have the ability to start, stop, or pause the test application. Statistics show the current transactions per minute (33 in the screenshot). There are three graphs on this page. The first graph shows the redo generation rate on the primary database measured in kilobytes per second. The second graph shows both the current transport lag and current apply lag measured in seconds on the standby database. The third graph shows the apply rate measure in kilobytes per second on the standby database.

Quiz

Oracle Enterprise Manager Database Express 12*c* can be used to create and configure a Data Guard configuration.

a. True
b. False

Answer: b

Quiz

The Verify Data Guard Configuration feature of Oracle Enterprise Manager (EM) can also be performed using DGMGRL.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- Create a Data Guard broker configuration
- Create a physical standby database
- Verify a Data Guard configuration
- Edit database properties related to Data Guard
- Test a Data Guard configuration

Practice 2: Overview

This practice covers the following topics:

- Creating a physical standby database using Enterprise Manager
- Verifying and examining the Data Guard Environment
- Generating a test workload
- Preparing for command-line practices
- Creating a new primary database

3

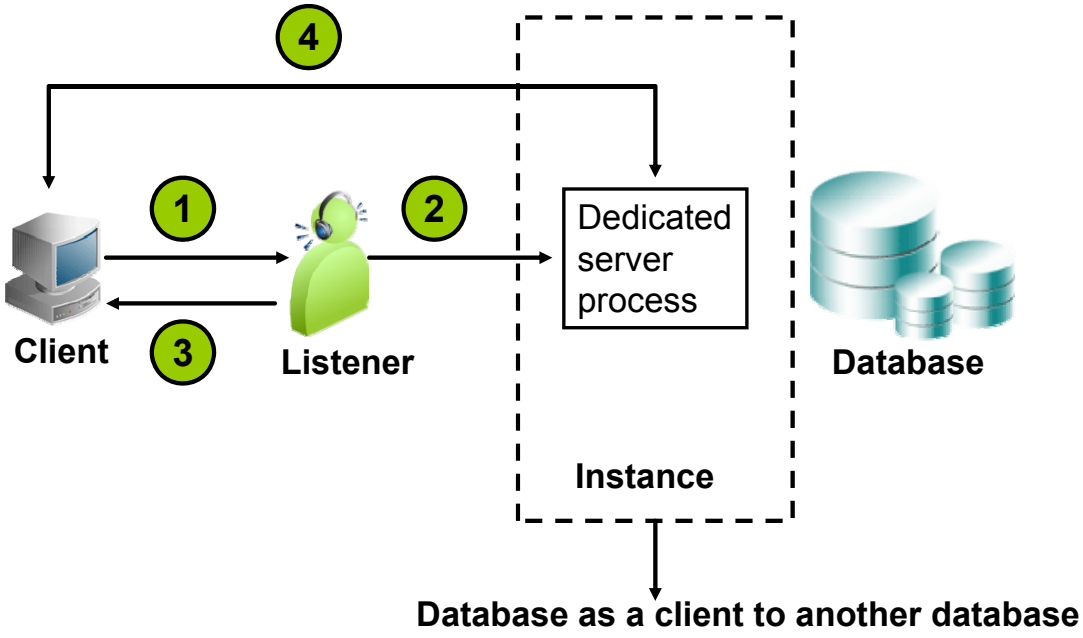
Oracle Net Services in a Data Guard Environment

Objectives

After completing this lesson, you should be able to:

- Understand the basics of Oracle Net Services
- Configure client connectivity in a Data Guard configuration
- Implement Data Guard best practice solutions in networking setup

Oracle Net Services Overview



Oracle Net Services provide enterprise-wide connectivity solutions in distributed, heterogeneous computing environments. It enables a network session from a client application to an Oracle Database server.

A dedicated server process is a type of service handler that the listener starts when it receives a client request. The slide depicts the process of establishing a connection from a client to an Oracle Database (non-shared server architecture) by using the following steps:

1. The listener receives a client connection request.
2. The listener starts a dedicated server process.
3. The listener provides the location of the dedicated server process in a redirect message.
4. The client connects directly to the dedicated server.

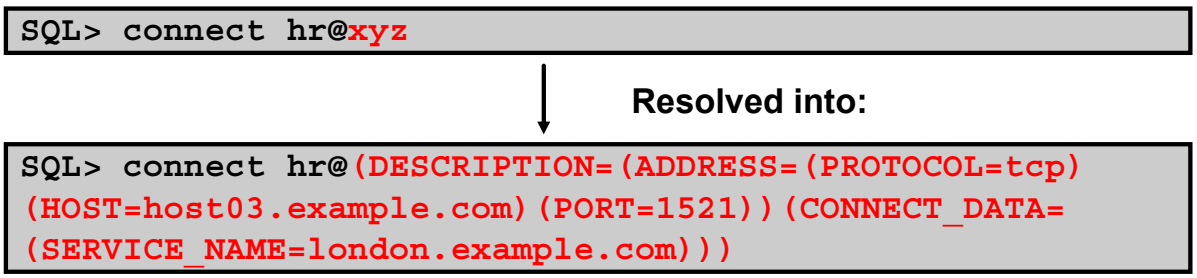
Note: Depending on the operating system and transport protocol, step three may be eliminated. In this case, the dedicated server process inherits the connection request from the listener. The result is the same—a network session established is between the client and the dedicated server process.

If the client and database exist on the same computer, then the application initiating the session can spawn a dedicated server process without going through the listener. This is known as the bequeath protocol and is often used when starting a database instance.

Understanding Name Resolution

A naming method is a resolution method used by a client application to resolve a connect identifier to a connect descriptor when attempting to connect to a database service. The following methods are available:

- Local naming (uses static text-based configuration files)
- Directory naming (uses a dynamic LDAP-compliant server)
- Easy Connect naming (uses simple hard-coded strings)
- External naming (uses a third-party naming service)



A client application needs a connect descriptor to create a network session. The connect descriptor contains two components:

- Location of the listener through a protocol address, also known as the listening endpoint
- A specific database service name or Oracle system identifier recognized by the listener

The connect descriptor can be explicitly specified as in the second example in the slide, which does not require name resolution. This technique is used by the Data Guard broker when the broker modifies the `LOG_ARCHIVE_DEST_n` parameters for redo transport. Also, a direct TCP/IP address can be specified in place of the hostname to avoid additional lookups such as using Domain Name Services (DNS).

In most cases though, a user initiates a connection request by providing a connection string that can include a username, password, and a simplified connect identifier. This requires that the simplified connect identifier be resolved into the appropriate details using one of the many name resolution methods that are available. There are advantages and disadvantages to each naming method. The `sqlnet.ora` file indicates which naming methods are available to the client or server. This course will utilize the local naming method, which stores network service names and their connect descriptors in a localized configuration file named `tnsnames.ora`.

Local Naming Configuration Files

The location of the local naming method configuration files is specified by either the TNS_ADMIN or ORACLE_HOME operating system environment variables. For ORACLE_HOME locations consider:

- Multiple products each with a different ORACLE_HOME
- Multiple versions of the same product, each with a different ORACLE_HOME
- Same product version installed multiple times for business isolation or upgrade path requirements, each with a different ORACLE_HOME

The local naming method uses text-based configuration files stored on both the client and the database server to resolve network service names (connect identifiers) into detailed connect descriptors. Operating-system environment variables determine the location of these files. The TNS_ADMIN variable is checked first. It allows the configuration files to be centrally located (for example, on a cluster file system and shared among many machines).

If the TNS_ADMIN variable is not defined, then the ORACLE_HOME variable is used to locate the configuration files. Several different ORACLE_HOME locations can be present on the same host machine for different reasons. Each may contain network configuration files if desired. For example, the tnsnames.ora and sqlnet.ora file could be found in the ORACLE_HOME location for each of the grid infrastructure software, database software, middleware software, and Enterprise Manager software products. The value of the ORACLE_HOME environment variable will point to a single ORACLE_HOME location at a time. The listener.ora configuration file is used only for database software ORACLE_HOME locations.

Note: In a Real Application Cluster environment, the Oracle Database listener usually runs from the ORACLE_HOME of the Grid Infrastructure software and not the ORACLE_HOME of the database software. The ORACLE_HOME of the Grid Infrastructure software will need networking configuration if ASM is being deployed. If separation of duties exist between the cluster software administrator and the database administrator, the database listener could run from the ORACLE_HOME of the database software.

Local Naming: tnsnames.ora

Database parameter file (spfileNAME.ora/initNAME.ora)

```
LOG_ARCHIVE_DEST_2='SERVICE=london …'
```

Oracle Net Configuration (on database host machine):
$ORACLE_HOME/network/admin/tnsnames.ora

```
LONDON =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS=(PROTOCOL=TCP)(HOST=host03.example.com)
      (PORT=1521)(SEND_BUF_SIZE=10485760)
      (RECV_BUF_SIZE=10485760))
    )
    (SDU=65535)
    (CONNECT_DATA=(SERVICE_NAME=london.example.com))
  )
```

ORACLE

The slide shows an example of using the local naming method for a Data Guard environment. Located inside of the database parameter file, the LOG_ARCHIVE_DEST_n parameter with the SERVICE attribute is used to define the remote Oracle database instance to which redo data will be sent. A network service name (shown in the slide with the value LONDON) will be resolved into the connect descriptor using the local naming method. The connect descriptor provides the protocol and address information needed to contact the listener on its listening endpoint. After the listener is contacted, a network session is requested for the specified SERVICE_NAME. There is no requirement that the network service name (LONDON) have the same value or similarly named value as the SERVICE_NAME(london.example.com).

Connect-Time Failover: Planning for Role Reversal

Connect-time failover is turned on by default for multiple address lists (ADDRESS_LIST used) and does not have to be specified.

```
PRMY =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (FAILOVER=on)
      (ADDRESS=(PROTOCOL=TCP)(HOST=host01.example.com)
      (PORT=1521)(SEND_BUF_SIZE=10485760)
      (RECV_BUF_SIZE=10485760))
      (ADDRESS=(PROTOCOL=TCP)(HOST=host03.example.com)
      (PORT=1521)(SEND_BUF_SIZE=10485760)
      (RECV_BUF_SIZE=10485760))
    )
    (SDU=65535)
    (CONNECT_DATA=(SERVICE_NAME=prmy.example.com))
  )
```


Role-reversal operations such as failover and switchover can modify which host the primary or standby database is currently running on. The client configuration should include a connect descriptor that includes all potential host that a particular service can run on. This can be configured by specifying an ADDRESS_LIST with multiple listening endpoints or addresses within it. Connect-time failover instructs Oracle Net to fail over to a different listener if the first listener fails. The use of the ADDRESS_LIST clause turns on connect-time failover by default. It is not necessary to use the FAILOVER=on statement, as indicated by the red text in the slide.

When the application connections are being made, if they should happen to attempt to connect to an old primary host that is unavailable, the connection attempt to that host should last no longer than 3 seconds. This allows for connection attempts to get through the ADDRESS_LIST quickly until a new primary host is found. This can be configured with the following entry placed into the sqlnet.ora file:

```
SQLNET.OUTBOUND_CONNECT_TIMEOUT=3
```

Note: The ADDRESS_LIST clause is optional. Multiple addresses can be defined without explicitly using the ADDRESS_LIST syntax. In this case, the failover mode would default to be off and it would explicitly require the FAILOVER=on syntax to enable connect-time failover.

Listener Configuration: `listener.ora`

Oracle Net Configuration (on database host machine):
`$ORACLE_HOME/network/admin/listener.ora`

The following entry defines two listening endpoints:

```
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS=(PROTOCOL=TCP)(HOST=host03.example.com)
      (PORT=1521)(SEND_SDU=10485760)
      (RECV_SDU = 10485760)))
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521)))
  )
ADR_BASE_LISTENER = /u01/app/oracle
LISTENER2 = …
ADR_BASE_LISTENER2 = /u01/app/oracle
```


A listener is configured with one or more listening protocol addresses, information about supported services, and parameters that control its runtime behavior. The listener configuration is stored in a configuration file named `listener.ora`. Because the configuration parameters have default values, it is possible to start and use a listener with no configuration. In an Oracle Data Guard environment, best practices usually indicate a need to adjust the session data unit (SDU) buffer to improve network performance. This is especially important on a wide area network (WAN) that has long delays. Modifying this value will require that the `listener.ora` file be created with non-default values.

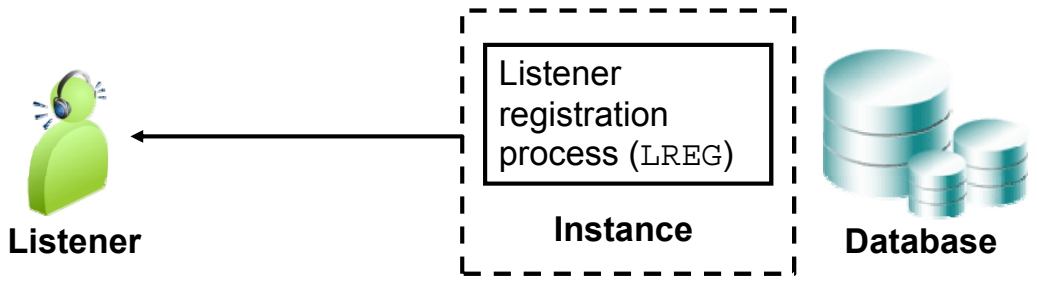
The red and blue text in the slide indicates the name of a listener. This name is used in commands that refer to the listener such as `lsnrctl start LISTENER2`. If the default name is used, it is not necessary to identify it in most commands. For example, the command `lsnrctl start` will start a listener with the default name of `LISTENER`.

One listener can listen for many databases, or there can exist a single listener for each database. Inside of a single `listener.ora` file, multiple listener configurations can be created by duplicating the contents of the file and changing the listener name to a different value for each occurrence. In the slide, the blue text shows a second listener named `LISTENER2`. Each separate listener requires a distinct listening endpoint. This technique can allow the listener to be started and stopped separately for each database on the same host machine.

Dynamic Service Registration

Dynamic service registration allows the database instance to identify its available services to the listener. Available service names are determined by the following parameters:

- DB_UNIQUE_NAME, DB_NAME, and DB_DOMAIN
- SERVICE_NAMES
- INSTANCE_NAME
- LOCAL_LISTENER and REMOTE_LISTENER



Dynamic service registration is configured in the database initialization file. It does not require any configuration in the listener.ora file. If not specified, the value for the SERVICE_NAMES parameter defaults to the global database name, a name comprised of the DB_UNIQUE_NAME and DB_DOMAIN parameters in the initialization parameter file. If not explicitly defined, the DB_UNIQUE_NAME parameter defaults to the value DB_NAME. The value for the INSTANCE_NAME parameter defaults to the Oracle system identifier (SID). All of these names can be explicitly defined to non-default values.

SERVICE_NAMES specifies one or more names by which clients can connect to the instance. The instance registers its service names with the listener. You can specify multiple service names to distinguish among different uses of the same database. For example:

```
SERVICE_NAMES=PROD,DG_PRMY,DG_RW,MAIN_REPORTING
```

By default, the LREG process registers service information with its local listener on the default local address of TCP/IP, port 1521. To have the LREG process register with a local listener that does not use TCP/IP, port 1521, configure the LOCAL_LISTENER parameter in the initialization parameter file to locate the local listener.

A remote listener is a listener residing on one computer that redirects connections to a database instance on another computer. You can configure registration to remote listeners using the REMOTE_LISTENER parameter.

Static Listener Entries: `listener.ora`

Static listener entries are needed for Recovery Manager and Data Guard broker operations.

```
SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (GLOBAL_DBNAME = london.example.com)
      (ORACLE_HOME =
   /u01/app/oracle/product/12.1.0/dbhome_1)
      (SID_NAME = london))
    (SID_DESC =
      (GLOBAL_DBNAME = london_DGMGRL.example.com)
      (ORACLE_HOME =
   /u01/app/oracle/product/12.1.0/dbhome_1)
      (SID_NAME = london))
  )
```


Dynamic service registration allows the listener registration process (LREG) of a database instance to identify its available services to the listener without entries in the `listener.ora` configuration file. The listener then acts as a port mapper for those services. However, when the database instance is stopped, the listener discards all information for the dynamic services related to that database. Any attempt to establish a network session to the unknown service will usually receive the error message “ORA-12514: Listener does not currently know of service requested in connect descriptor.” Static registration allows the listener to know of a service even if the database instance is not running. This is often important with tools and utilities that try to remotely start and stop a database instance. Configuration of static service information is necessary in the following cases:

- Use of external procedure calls
- Use of heterogeneous services
- Use of Oracle Data Guard
- Remote database startup from a tool other than Oracle Enterprise Manager Cloud Control

To enable DGMGRL to restart instances during the course of broker operations, a static service must be registered with the local listener and assume a static service name of `db_unique_name_DGMGRL.db_domain`.

Optimizing Oracle Net for Data Guard

- Calculate the TCP Socket buffer size by multiplying the bandwidth delay product (BDP) by 3.

```
TCP Socket Buffer size=BDP*3=network bandwidth*latency*3
```

- Set the send and receive buffer sizes to the larger of BDP*3 or 10 MB in both the `tnsnames.ora` and `listener.ora`.

```
(SEND_BUF_SIZE=10485760)
(RECV_BUF_SIZE=10485760)
```

- Increase the session data unit (SDU) to 64 KB in both the `tnsnames.ora` and `listener.ora`.

```
(SDU=65535)
```

- Disable the TCP Nagle algorithm in the `sqlnet.ora` file.

```
TCP.NODELAY=YES
```


To achieve high network throughput, especially for a high-latency, high-bandwidth network, the minimum recommended setting for the sizes of the TCP send and receive socket buffers is the bandwidth-delay product (BDP) of the network link between the primary and standby systems. BDP is the product of the network bandwidth and latency. Tests have shown incremental throughput increases with socket buffer settings up to three times the BDP. Socket buffer sizes are set using the Oracle Net parameters `RECV_BUF_SIZE` and `SEND_BUF_SIZE`. For example, if bandwidth is 622 Mbit/s per second and latency is 30 ms, you would calculate the minimum size for the `RECV_BUF_SIZE` and `SEND_BUF_SIZE` parameters as follows: 622,000,000 / 8 x 0.030 = 2,332,500 bytes. Then, multiply the BDP 2,332,500 x 3 for a total of 6,997,500. Set the send and receive buffer sizes at either the value you calculated or 10 MB (10485760), whichever is larger.

Oracle Net buffers data into what is called a session data unit (SDU), with a default size of 8192 bytes. These data units are then sent to the network layer when they are either full, flushed, or read by the client. Generally Data Guard sends redo in much larger chunks than 8192 bytes, so this default is insufficient, because you can end up having to send more pieces (chopping up the data) to Oracle Net Services. Increase this to 64 KB.

To preempt delays in buffer flushing in the TCP protocol stack, disable the TCP Nagle algorithm by setting `TCP.NODELAY` to YES in the `sqlnet.ora` file on both the primary and standby systems.

The actual value of the SEND_BUF_SIZE and RECV_BUF_SIZE parameters may be less than the value specified because of limitations in the host operating system or due to memory constraints. The default values for these parameters are operating-system specific. The following are the defaults for the Linux operating system:

SEND_BUF_SIZE: 131,072 bytes (128 KB)

RECV_BUF_SIZE: 174,700 bytes

These values are typically modified to be higher when the oracle-rdbms-server-12cR1-preinstall package is installed, but the modified values are still lower than the Data Guard best practice recommended values.

The default and maximum amount for the receive socket memory can be determined with:

```
# cat /proc/sys/net/core/rmem_default
# cat /proc/sys/net/core/rmem_max
```

The default and maximum amount for the send socket memory can be determined with:

```
# cat /proc/sys/net/core/wmem_default
# cat /proc/sys/net/core/wmem_max
```

Adjust values with the following commands:

```
# echo 'net.core.wmem_max=10485760' >> /etc/sysctl.conf
# echo 'net.core.rmem_max=10485760' >> /etc/sysctl.conf
```

You must also set minimum size, initial size, and maximum size in bytes:

```
# echo 'net.ipv4.tcp_rmem= 10240 131072 10485760' >>
/etc/sysctl.conf
# echo 'net.ipv4.tcp_wmem= 10240 131072 10485760' >>
/etc/sysctl.conf
```

Reload the changes made by using the following command:

```
# sysctl –p
```

Note: Starting with Oracle Database 12*c* Release 1 (12.1), Oracle Net supports large session data unit (SDU) sizes, with a new upper limit of 2 MB. The larger SDU size can be used to achieve better utilization of available network bandwidth in networks that have high bandwidth delay products and host resources, according to application characteristics. At the moment in time of writing this course, current best practices recommend a value of 64 KB. This may change in the future with additional testing.

Quiz

Running the `oracle-rdbms-server-12cR1-preinstall` package is sufficient to set up operating system kernel parameters for an Oracle Linux Data Guard environment using best practice guidelines.

a. True
b. False

Answer: b

Quiz

In order to use the connect-time failover feature, the failover clause must be included in the `tnsnames.ora` file.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- Understand the basics of Oracle Net Services
- Configure client connectivity in a Data Guard configuration
- Implement Data Guard best practice solutions in networking setup

Practice 3: Overview

This practice covers the following topics:

- Declaring Oracle Net naming methods
- Creating Oracle Net service names
- Implementing best practice values for the Oracle Net configuration

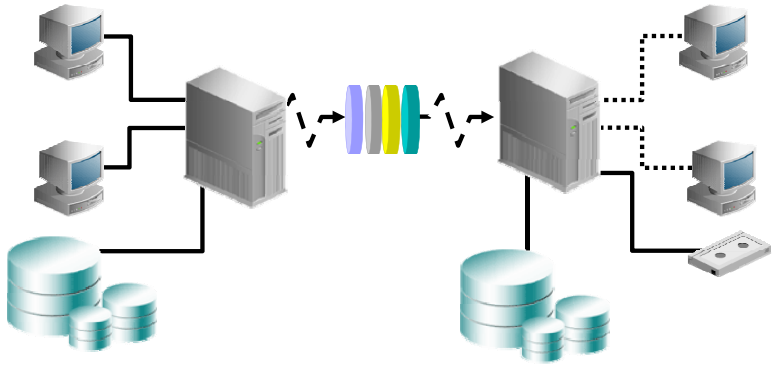
4

Creating a Physical Standby Database by Using SQL and RMAN Commands

Objectives

After completing this lesson, you should be able to:

- Configure the primary database and Oracle Net Services to support the creation of the physical standby database and role transition
- Create a physical standby database by using the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command



ORACLE

Steps to Create a Physical Standby Database

1. Prepare the primary database.
2. Set parameters on the physical standby database.
3. Configure Oracle Net Services.
4. Start the standby database instance.
5. Execute the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command.
6. Start the transport and application of redo.

You perform the steps listed in the slide when using SQL and RMAN commands to create a physical standby database. Detailed information about each step is provided in the remaining slides of the lesson.

Note: See *Oracle Data Guard Concepts and Administration* for detailed information about creating a physical standby database by using only SQL commands.

Preparing the Primary Database

- Enable FORCE LOGGING at the database level.
- Create a password file if required.
- Create standby redo logs.
- Set initialization parameters.
- Enable archiving.

```
SQL> SHUTDOWN IMMEDIATE;
SQL> STARTUP MOUNT;
SQL> ALTER DATABASE ARCHIVELOG;
SQL> ALTER DATABASE OPEN;
```

ORACLE

The FORCE LOGGING mode determines whether the Oracle database server logs all changes in the database (except for changes to temporary tablespaces and temporary segments).

Notes:

Additional information about enabling FORCE LOGGING follows in this lesson.

Unless you have configured Oracle Advanced Security and public key infrastructure (PKI) certificates, every database in a Data Guard configuration must use a password file, and the password for the SYS user must be identical on every system for redo data transmission to succeed. For details about creating a password file, see the *Oracle Database Administrator's Guide*.

A standby redo log is used to store redo received from another Oracle database.

Additional information about creating standby redo log files is provided in this lesson.

On the primary database, you define initialization parameters that control redo transport services while the database is in the primary role. There are other parameters that you need to add that control the receipt of the redo data and log apply services when the primary database is transitioned to the standby role. Additional information about setting initialization parameters is provided in this lesson.

The Data Guard broker requires the use of a server parameter file.

If archiving is not enabled, issue the ALTER DATABASE ARCHIVELOG command to put the primary database in ARCHIVELOG mode and enable automatic archiving. See the *Oracle Database Administrator's Guide* for additional information about archiving.

FORCE LOGGING Mode

- FORCE LOGGING mode is recommended to ensure data consistency.
- FORCE LOGGING forces redo to be generated even when NOLOGGING operations are executed.
- Temporary tablespaces and temporary segments are not logged.
- FORCE LOGGING is recommended for both physical and logical standby databases.
- Issue the following command on the primary database:

```
SQL> ALTER DATABASE FORCE LOGGING;
```


FORCE LOGGING mode determines whether the Oracle database server logs all changes in the database (except for changes to temporary tablespaces and temporary segments). The [NO] FORCE LOGGING clause of the ALTER DATABASE command contains the following settings:

- **FORCE LOGGING:** This setting takes precedence over (and is independent of) any NOLOGGING or FORCE LOGGING settings that you specify for individual tablespaces and any NOLOGGING setting that you specify for individual database objects. All ongoing, unlogged operations must finish before forced logging can begin.
- **NOFORCE LOGGING:** Places the database in NOFORCE LOGGING mode. This is the default.

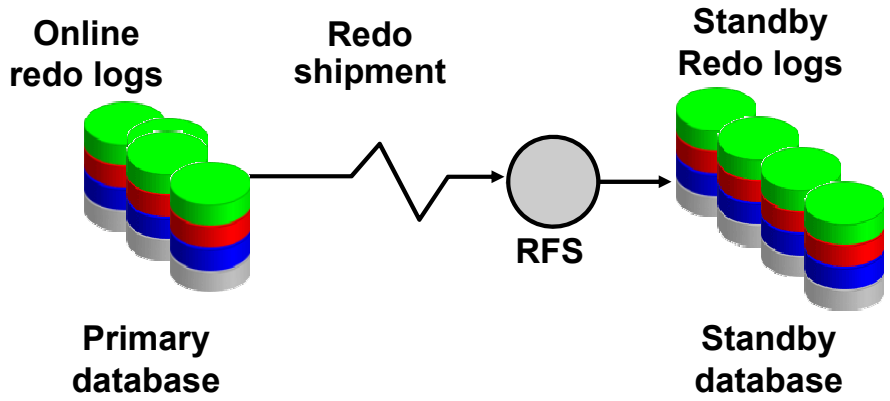
The FORCE_LOGGING column in V$DATABASE contains a value of YES if the database is in FORCE LOGGING mode.

Although the database can be placed in FORCE LOGGING mode when the database is OPEN, the mode does not change until the completion of all operations that are currently running in NOLOGGING mode. Therefore, it is recommended that you enable FORCE LOGGING mode when the database is in the MOUNT state.

Note: You should enable FORCE LOGGING before performing the backup operation to create the standby database, and then maintain FORCE LOGGING mode for as long as the Data Guard configuration exists.

Configuring Standby Redo Logs

- Create standby redo logs on the primary database initially (recommended).
- Create standby redo logs using the same file size as the primary database online redo logs.
- Create one additional group more than the number of online redo log groups.



A standby redo log is used only when the database is in the standby role to store redo data received from the primary database. Standby redo logs form a separate pool of log file groups. Configuring standby redo log files is highly recommended for all databases in a Data Guard configuration to aid in role reversal.

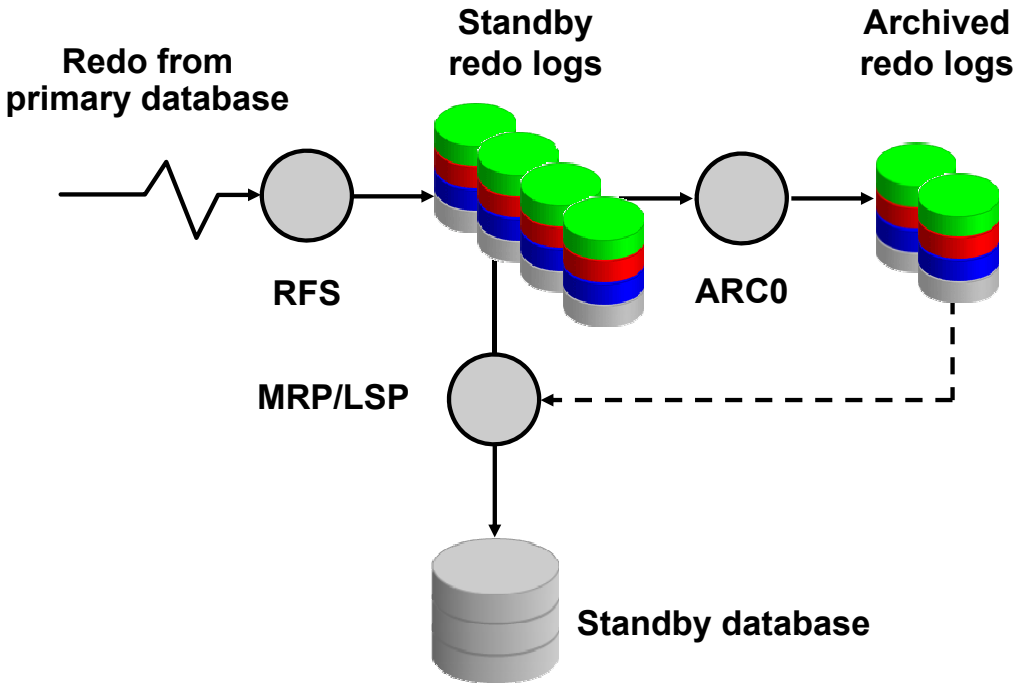
You should create at least one more standby redo log group than you have online redo log groups in the primary database. In addition, each standby redo log file must be at least as large as the largest redo log file in the redo log of the redo source database.

A standby redo log is required to implement:

- Synchronous transport mode
- Real-time apply
- Cascaded redo log destinations

Note: By configuring the standby redo logs on the primary database, the standby redo logs are created automatically on the standby database when you execute the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command. They will also be automatically created on the Far Sync server as well when it is created, provided that they already exist on the primary machine. Far Sync will be discussed later in the course.

Standby Redo Log Usage



If standby redo logs are available and configured correctly, the RFS process will write the redo received from the primary database to the standby redo logs. Because the files are generally the same size as the online redo logs of the primary database, they should fill up and switch at a similar frequency as the online redo logs. There may be some delay due to the latency of the network transmission of the redo. The additional standby redo log group should allow the log switch of the standby redo logs without causing wait states. At log switch of the standby redo logs, the `ARC0` process will create an archived redo log using the completed standby redo log.

The RFS process creates and writes directly to an archived redo log file instead of the standby redo log if any of the following conditions are met:

- There are no standby redo logs.
- It cannot find a standby redo log that is the same size as or larger than the incoming online redo log file.
- All standby redo logs of the correct size have not yet been archived.

Note: In previous releases, the parameter `STANDBY_ARCHIVE_DEST` was used to identify the location of the archived redo logs created on the standby database host that were created from the standby redo logs. This parameter is now deprecated, because an appropriate location is automatically chosen.

Using SQL to Create Standby Redo Logs

Create standby redo logs on the primary database to assist role changes:

```
SQL> alter database add standby logfile
('/u01/app/oracle/oradata/boston/stdbyredo01.log')
size 50M;
Database altered.

or

SQL> ALTER DATABASE ADD STANDBY LOGFILE
  2  '+DATA' SIZE 52428800;
Database altered.
```


You can create standby redo logs by using the `ADD STANDBY LOGFILE` clause of the `ALTER DATABASE` statement. Although standby redo logs are used only when the database is operating in the standby role, you should create standby redo logs on the primary database so that switching roles does not require additional DBA intervention.

You should create standby redo log files on the primary database prior to using the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command so that RMAN creates standby redo log files automatically on the standby database.

Create standby redo log file groups by using the following guidelines:

- Each standby redo log file must be at least as large as the largest redo log file in the redo source database. For administrative ease, Oracle recommends that all redo log files in the redo source database and the redo transport destination be of the same size.
- The standby redo log must have at least one more redo log group than the redo log on the redo source database.

Viewing Standby Redo Log Information

View information about the standby redo logs:

```
SQL> SELECT group#, type, member FROM v$logfile
  2  WHERE type = 'STANDBY';
GROUP# TYPE    MEMBER
------ ------- ----------------------------------
     4 STANDBY /u01/app/oracle/oradata/boston/stdbyredo01.log
     5 STANDBY /u01/app/oracle/oradata/boston/stdbyredo02.log
     6 STANDBY /u01/app/oracle/oradata/boston/stdbyredo03.log
     7 STANDBY /u01/app/oracle/oradata/boston/stdbyredo04.log

SQL> SELECT group#, dbid, thread#, sequence#, status
  2  FROM v$standby_log;
    GROUP# DBID          THREAD#  SEQUENCE# STATUS
---------- ---------- ---------- ---------- ----------
         4 2581955083          1         44 ACTIVE
         5 UNASSIGNED          1          0 UNASSIGNED
         6 UNASSIGNED          1          0 UNASSIGNED
         7 UNASSIGNED          0          0 UNASSIGNED
```

ORACLE

To verify that standby redo logs were created, query V$STANDBY_LOG or V$LOGFILE. An example output using Automatic Storage Management (ASM) is shown below:

```
SQL> SELECT group#, type, member FROM v$logfile WHERE type =
   'STANDBY';
GROUP# TYPE    MEMBER
---------- ------- ------------------------------------------------
         4 STANDBY +SBDAT/london/onlinelog/group_4.266.711624939
         5 STANDBY +SBDAT/london/onlinelog/group_5.267.711624945
         6 STANDBY +SBDAT/london/onlinelog/group_6.268.711624951
         7 STANDBY +SBDAT/london/onlinelog/group_7.269.711624957
         4 STANDBY +SBFRA/london/onlinelog/group_4.259.711624941
         5 STANDBY +SBFRA/london/onlinelog/group_5.260.711624947
         6 STANDBY +SBFRA/london/onlinelog/group_6.261.711624955
         7 STANDBY +SBFRA/london/onlinelog/group_7.262.711624963
8 rows selected.
```

Setting Initialization Parameters on the Primary Database to Control Redo Transport

Parameter Name	Description
LOG_ARCHIVE_CONFIG	Specifies the unique database name for each database in the configuration Enables or disables sending and receiving of redo
LOG_ARCHIVE_DEST_*n*	Controls redo transport services
LOG_ARCHIVE_DEST_STATE_*n*	Specifies the destination state
ARCHIVE_LAG_TARGET	Forces a log switch after the specified number of seconds
LOG_ARCHIVE_TRACE	Controls output generated by the archiver process

ORACLE

On the primary database, you define initialization parameters that control redo transport services while the database is in the primary role. These parameters are described in more detail in the following slides.

Setting LOG_ARCHIVE_CONFIG

Specify the DG_CONFIG attribute to list the DB_UNIQUE_NAME for the primary database and each standby database in the Data Guard configuration.

```
LOG_ARCHIVE_CONFIG='DG_CONFIG=(boston,london)'
```


Specify the DG_CONFIG attribute of the LOG_ARCHIVE_CONFIG parameter to list the DB_UNIQUE_NAME of the primary and standby databases in the Data Guard configuration. By default, the LOG_ARCHIVE_CONFIG parameter enables the database to send and receive redo. The LOG_ARCHIVE_CONFIG parameter can be used to disable the sending of redo logs to remote destinations or disable the receipt of remote redo logs. The complete syntax for the LOG_ARCHIVE_CONFIG parameter is as follows:

```
LOG_ARCHIVE_CONFIG = {
[ SEND | NOSEND ][ RECEIVE | NORECEIVE ]
[ DG_CONFIG=(remote_db_unique_name1
[, ... remote_db_unique_name9) | NODG_CONFIG ] }
```

Use the V$DATAGUARD_CONFIG view to see the unique database names defined with the DB_UNIQUE_NAME and LOG_ARCHIVE_CONFIG initialization parameters; you can thus view the Data Guard environment from any database in the configuration. The first row of the view lists the unique database name of the current database that was specified with the DB_UNIQUE_NAME initialization parameter. Additional rows reflect the unique database names of the other databases in the configuration that were specified with the DG_CONFIG keyword of the LOG_ARCHIVE_CONFIG initialization parameter.

The following example illustrates the use of V$DATAGUARD_CONFIG:

```
SQL> show parameter log_archive_config

NAME                 TYPE        VALUE
-------------------- ------- ----------------------------
log_archive_config   string   dg_config=(boston,london)

SQL> SELECT * FROM v$dataguard_config;

DB_UNIQUE_NAME
------------------------------
boston
london
```

Setting LOG_ARCHIVE_DEST_*n*

- Specify LOG_ARCHIVE_DEST_*n* parameters for:
  - Local archiving
  - Standby database location
- Include (at a minimum) one of the following:
  - LOCATION: Specifies a valid path name
  - SERVICE: Specifies a valid Oracle Net Services name referencing a standby database
- Include a LOG_ARCHIVE_DEST_STATE_*n* parameter for each defined destination.

```
LOG_ARCHIVE_DEST_2=
 'SERVICE=london
  VALID_FOR=(ONLINE_LOGFILES,PRIMARY_ROLE)
  DB_UNIQUE_NAME=london'
LOG_ARCHIVE_DEST_STATE_2=ENABLE
```

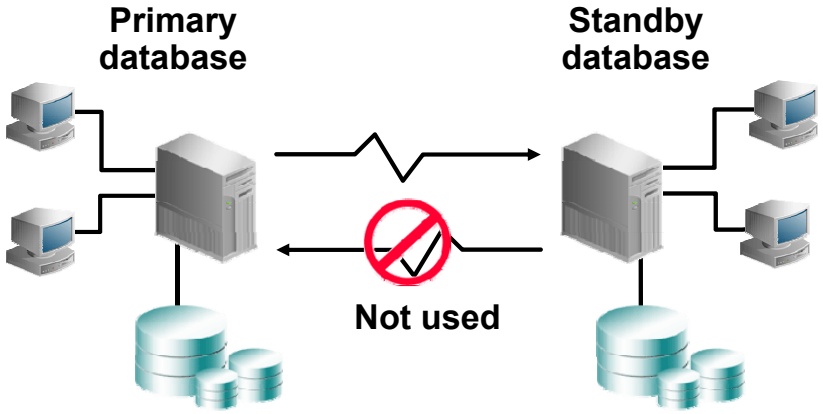

By using the various LOG_ARCHIVE_DEST_*n* attributes, you define most of the settings for the Data Guard configuration. The Redo Transport Service is directly controlled by these settings. A number of different attributes can be set for each LOG_ARCHIVE_DEST_*n* parameter. Most have defaults that are adequate for most configurations. See *Oracle Data Guard Concepts and Administration* for a complete list and a description of each.

You should specify a LOG_ARCHIVE_DEST_*n* parameter (where *n* is an integer from 1 to 31) for the local archiving destination and one for the standby location. In previous versions of Oracle Database, LOG_ARCHIVE_DEST_10 was set to USE_DB_RECOVERY_FILE_DEST when the fast recovery area was being used. Beginning with Oracle Database 11*g* Release 2, you must manually set a location to use the fast recovery area. Query the V$ARCHIVE_DEST view to see current settings of the LOG_ARCHIVE_DEST_*n* initialization parameter.

All defined LOG_ARCHIVE_DEST_*n* parameters must contain, at a minimum, either a LOCATION attribute or a SERVICE attribute.

In addition, you must have a LOG_ARCHIVE_DEST_STATE_*n* parameter for each defined destination. LOG_ARCHIVE_DEST_STATE_*n* defaults to ENABLE.

Specifying Role-Based Destinations




```
log_archive_dest_2 =
'service=london async
 valid_for=
 (online_logfile,
  primary_role)
db_unique_name=london'
```

```
log_archive_dest_2 =
'service=boston async
 valid_for=
 (online_logfile,
  primary_role)
db_unique_name=boston'
```


The VALID_FOR attribute of the LOG_ARCHIVE_DEST_*n* initialization parameter enables you to identify exactly when the archive destination is to be used, as well as which type of log file it is used for. The attribute uses a keyword pair to identify the redo log type as well as the database role. Using this attribute enables you to set up parameters in anticipation of switchover and failover operations.

In the example in the slide, there is a destination on the standby database and the primary database defined with the VALID_FOR setting shown. This destination is to be used on the standby database only after a switchover, when the standby becomes a primary. The destination on the old primary is ignored when it becomes a standby.

You supply two values for the VALID_FOR attribute: redo_log_type and database_role.

The redo_log_type keywords are:

- **ONLINE_LOGFILE:** This destination is used only when archiving online redo log files.
- **STANDBY_LOGFILE:** This destination is used only when archiving standby redo log files or receiving archive logs from another database.
- **ALL_LOGFILES:** This destination is used when archiving either online or standby redo log files.

The *database_role* keywords are the following:

- **PRIMARY_ROLE:** This destination is used only when the database is in the primary database role.
- **STANDBY_ROLE:** This destination is used only when the database is in the standby (logical or physical) role.
- **ALL_ROLES:** This destination is used when the database is in either the primary or the standby (logical or physical) role.

Note: Because the keywords are unique, the *archival_source* and *database_role* values can be specified in any order.

For example, `VALID_FOR=(PRIMARY_ROLE,ONLINE_LOGFILE)` is functionally equivalent to `VALID_FOR=(ONLINE_LOGFILE,PRIMARY_ROLE)`.

Combinations for VALID_FOR

Combination	Primary	Physical	Logical
ONLINE_LOGFILE, PRIMARY_ROLE	Valid	Ignored	Ignored
ONLINE_LOGFILE, STANDBY_ROLE	Ignored	Ignored	Valid
ONLINE_LOGFILE, ALL_ROLES	Valid	Ignored	Valid
STANDBY_LOGFILE,STANDBY_ROLE	Ignored	Valid	Valid
STANDBY_LOGFILE, ALL_ROLES	Ignored	Valid	Valid
ALL_LOGFILES, PRIMARY_ROLE	Valid	Ignored	Ignored
ALL_LOGFILES, STANDBY_ROLE	Ignored	Valid	Valid
ALL_LOGFILES, ALL_ROLES	Valid	Valid	Valid

In the table in the slide, *Valid* indicates that the archive log destination is used in a database that is in the role defined by the column heading. *Ignored* means that the archive log destination is not appropriate and that a destination of this type is ignored. An ignored destination does not generate an error.

There is only one invalid combination: STANDBY_LOGFILE, PRIMARY_ROLE. Specifying this combination causes an error for all database roles. If it is set, you receive the following error at startup:

```
ORA-16026: The parameter LOG_ARCHIVE_DEST_n contains an invalid
attribute value
```

Note: Both single and plural forms of the keywords are valid. For example, you can specify either PRIMARY_ROLE or PRIMARY_ROLES, as well as ONLINE_LOGFILE or ONLINE_LOGFILES.

Defining the Redo Transport Mode

Use the attributes of LOG_ARCHIVE_DEST_*n*:

- SYNC and ASYNC
  - Specify that network I/O operations are to be performed synchronously or asynchronously when using LGWR.
  - ASYNC is the default.
- AFFIRM and NOAFFIRM
  - Ensure that redo was successfully written to disk on the standby destination.
  - NOAFFIRM is the default when ASYNC is specified; AFFIRM is the default when SYNC is specified.

The following attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter define the redo transport mode that is used by the primary database to send redo to the standby database.

- **SYNC:** Specifies that redo data generated by a transaction must have been received at a destination that has this attribute before the transaction can commit; otherwise, the destination is deemed to have failed. In a configuration with multiple SYNC destinations, the redo must be processed as described here for every SYNC destination.
- **ASYNC (default):** Specifies that redo data generated by a transaction need not have been received at a destination that has this attribute before the transaction can commit
- **AFFIRM:** Specifies that a redo transport destination acknowledges received redo data *after* writing it to the standby redo log
- **NOAFFIRM:** Specifies that a redo transport destination acknowledges received redo data *before* writing it to the standby redo log

If neither the AFFIRM nor the NOAFFIRM attribute is specified, the default is AFFIRM when the SYNC attribute is specified and NOAFFIRM when the ASYNC attribute is specified.

Note: Specifying the AFFIRM attribute without the SYNC attribute is deprecated and will not be supported in future releases.

Setting Initialization Parameters on the Primary Database

- Specify parameters when standby databases have disk or directory structures that differ from the primary database.
- Use parameters when the primary database is transitioned to a standby database.

Parameter Name	Description
DB_FILE_NAME_CONVERT	Converts primary database file names
LOG_FILE_NAME_CONVERT	Converts primary database log file names
STANDBY_FILE_MANAGEMENT	Controls automatic standby file management
FAL_SERVER	Specifies the fetch archive log server for a standby database

The parameters listed in the slide are required if the disk configuration is not the same for the primary and standby databases. The parameters are also applicable when the primary database is transitioned to a standby database.

Specifying Values for DB_FILE_NAME_CONVERT

- DB_FILE_NAME_CONVERT must be defined on standby databases that have different disk or directory structures from the primary.
- Multiple pairs of file names can be listed in the DB_FILE_NAME_CONVERT parameter.
- DB_FILE_NAME_CONVERT applies only to a physical standby database.
- DB_FILE_NAME_CONVERT can be set in the DUPLICATE RMAN script.

```
DB_FILE_NAME_CONVERT =('/oracle1/dba/',
                       '/ora1/stby_dba/',
                       '/oracle2/dba/',
                       '/ora2/stby_dba/')
```


When files are added to the standby database, the DB_FILE_NAME_CONVERT parameter is used to convert the data file name on the primary database to a data file name on the standby database. The file must exist and be writable on the physical standby database; if it is not, the recovery process halts with an error.

You specify the path name and file name location of the primary database data files followed by the standby location by setting the value of this parameter to two strings. The first string is the pattern found in the data file names on the primary database. The second string is the pattern found in the data file names on the physical standby database. You can use as many pairs of primary and standby replacement strings as required. You can use single or double quotation marks. Parentheses are optional.

In the example in the slide, /oracle1/dba/ and /oracle2/dba/ are used to match file names coming from the primary database. /ora1/stby_dba/ and /ora2/stby_dba/ are the corresponding strings for the physical standby database. A file on the primary database named /oracle1/dba/system01.dbf is converted to /ora1/stby_dba/system01.dbf on the standby database.

Multiple pairs can be specified such as ('a','b','1','2').

Note: If the standby database uses Oracle Managed Files (OMF), do not set the DB_FILE_NAME_CONVERT parameter. There is a 255-character limit on this parameter.

Specifying Values for LOG_FILE_NAME_CONVERT

- LOG_FILE_NAME_CONVERT is similar to DB_FILE_NAME_CONVERT.
- LOG_FILE_NAME_CONVERT must be defined on standby databases that have different disk or directory structures from the primary.
- LOG_FILE_NAME_CONVERT applies only to a physical standby database.
- LOG_FILE_NAME_CONVERT can be set in the DUPLICATE RMAN script.

```
LOG_FILE_NAME_CONVERT = ('/oracle1/logs/',
                         '/ora1/stby_logs/')
```


The LOG_FILE_NAME_CONVERT parameter is used to convert the name of a redo log file on the primary database to the name of a redo log file on the standby database. Adding a redo log file to the primary database requires adding a corresponding file to the standby database. When the standby database is updated, this parameter is used to convert the log file name from the primary database to the log file name on the standby database. This parameter is required if the standby database is on the same system as the primary database or on a separate system that uses different path names.

Specify the location of the primary database online redo log files followed by the standby location. The use of parentheses is optional.

Both DB_FILE_NAME_CONVERT and LOG_FILE_NAME_CONVERT parameters perform simple string substitutions. For example, ('a','b') will transform the following:

/disk1/primary/mya/a.dbf into

/disk1/prim**b**ry/my**b**/**b**.dbf

Multiple pairs can be specified such as ('a','b','1','2').

Note: If the standby database uses OMF, do not set the LOG_FILE_NAME_CONVERT parameter. There is a 255-character limit on this parameter.

Specifying a Value for STANDBY_FILE_MANAGEMENT

- STANDBY_FILE_MANAGEMENT is used to maintain consistency when you add or delete a data file on the primary database.
  - MANUAL (default)
    - Data files must be manually added to the standby database.
  - AUTO
    - Data files are automatically added to the standby database.
    - Certain ALTER statements are no longer allowed on the standby database.
- STANDBY_FILE_MANAGEMENT applies to physical standby databases only, but can be set on a primary database for role changes.

```
STANDBY_FILE_MANAGEMENT = auto
```

ORACLE

When STANDBY_FILE_MANAGEMENT is set to AUTO, you cannot execute the following commands on the standby database:

- ALTER DATABASE RENAME
- ALTER DATABASE ADD/DROP LOGFILE [MEMBER]
- ALTER DATABASE ADD/DROP STANDBY LOGFILE MEMBER
- ALTER DATABASE CREATE DATAFILE AS . . .

When you add a log file to the primary database and want to add it to the physical standby database as well (or when you drop a log file from the primary and want to drop it from the physical), you must do the following:

1. Set STANDBY_FILE_MANAGEMENT to MANUAL on the physical standby database.
2. Add the redo log files to (or drop them from) the primary database.
3. Add them to (or drop them from) the standby database.
4. Reset to AUTO afterward on the standby database.

Specifying a Value for FAL_SERVER

- FAL_SERVER is:
  - Used to specify the name of the fetch archive log server, usually the primary database
  - Used only by databases in the standby role
  - Created on both primary and standby databases for role reversal
- On the boston primary database:

```
FAL_SERVER = london
```

- On the london standby database:

```
FAL_SERVER = boston
```


The FAL_SERVER parameter is used to specify the Oracle Net service name of the fetch archive log (FAL) server (typically this is the database running in the primary role). When the London database is running in the standby role, it uses the Boston database as the FAL server from which to fetch (request) missing archived redo log files if Boston is unable to automatically send the missing log files. It is assumed that the Oracle Net service name is configured properly on the standby database system to point to the desired FAL server.

Example: Setting Initialization Parameters on the Primary Database

Primary database: boston; standby database: london

```
DB_NAME=boston
DB_UNIQUE_NAME=boston
LOG_ARCHIVE_CONFIG='DG_CONFIG=(boston,london)'
CONTROL_FILES='/u01/app/oracle/oradata/boston/control01.ctl',
 '/u01/app/oracle/oradata/boston/control02.ctl'
LOG_ARCHIVE_DEST_2=
 'SERVICE=london
  VALID_FOR=(ONLINE_LOGFILES,PRIMARY_ROLE)
  DB_UNIQUE_NAME=london'
LOG_ARCHIVE_DEST_STATE_2=ENABLE
REMOTE_LOGIN_PASSWORDFILE=EXCLUSIVE
LOG_ARCHIVE_FORMAT=arch_%t_%s_%r.log
FAL_SERVER=london
STANDBY_FILE_MANAGEMENT=auto
DB_FILE_NAME_CONVERT='boston','london'
LOG_FILE_NAME_CONVERT='boston','london'
```

ORACLE

In the example in the slide, assume that the primary database is named `boston` and the standby is named `london`. For each, an Oracle Net Services name is defined.

Note: The convert parameters can also be used to change ASM disk groups. For example:
`DB_FILE_NAME_CONVERT=('+DATA','+SBDAT')`

Creating an Oracle Net Service Name for Your Physical Standby Database

Use Oracle Net Manager to update the `tnsnames.ora` file:

```
LONDON =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS=(PROTOCOL=TCP)(HOST=host03.example.com)
      (PORT=1521)(SEND_BUF_SIZE=10485760)
      (RECV_BUF_SIZE=10485760))
    )
    (SDU=65535)
    (CONNECT_DATA=(SERVICE_NAME=london.example.com))
  )
```


Use Oracle Net Manager to define a network service name for your physical standby database. The slide shows the entry in the `tnsnames.ora` file that was generated by Oracle Net Manager.

Note: This entry is used to connect to the standby database when invoking RMAN and executing the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` command. It is also used in the `LOG_ARCHIVE_DEST_2` parameter for the `SERVICE` value to define the redo transport to the standby database.

Creating a Listener Entry for Your Standby Database

Use Oracle Net Manager to configure an entry for your standby database in the `listener.ora` file:

```
SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (GLOBAL_DBNAME = london.example.com)
      (ORACLE_HOME =
/u01/app/oracle/product/12.1.0/dbhome_1)
      (SID_NAME = london)
    )
  )
```


Use Oracle Net Manager to configure a new listener (if necessary) or to update the `listener.ora` file with an entry for your physical standby database. The slide shows the entry in the `listener.ora` file that was generated by Oracle Net Manager.

Note: This entry is needed because the instance is shut down and restarted during the standby database creation using RMAN.

Copying Your Primary Database Password File to the Physical Standby Database Host

1. Copy the primary database password file to the `$ORACLE_HOME/dbs` directory on the standby database host.
2. Rename the file for your standby database: `orapw<SID>`.

You can create a password file for your physical standby database by copying the primary database password file to the physical standby database host and renaming it.

Note: It would be possible to use the `orapwd` utility to create a password file, but that technique should always be avoided with Data Guard. The Recovery Manager `DUPLICATE DATABASE` command is being used in this lesson and RMAN will automatically copy the password file from the primary and replace the one that was created. Manual creation will not work when creating other Data Guard servers such as Far Sync (to be discussed later in the course).

Creating an Initialization Parameter File for the Physical Standby Database

Create an initialization parameter file containing enough information to match the network listener services entries:

File: `$ORACLE_HOME/dbs/initlondon.ora`

```
DB_NAME=london
DB_DOMAIN=example.com
```


Create a text initialization parameter file containing only the `DB_NAME` and `DB_DOMAIN` initialization parameters.

This initialization parameter file is used to start the physical standby database in `NOMOUNT` mode prior to the execution of the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command. When you execute this command, RMAN creates a server parameter file for the standby database.

Creating Directories for the Physical Standby Database

Create the baseline directory structures needed on the physical standby database host.

```
[oracle@host03]$ mkdir -p
/u01/app/oracle/admin/london/adump
[oracle@host03]$ mkdir -p
/u01/app/oracle/oradata/london
[oracle@host03]$ mkdir -p
/u01/app/oracle/oradata/london/pdbseed
[oracle@host03]$ mkdir -p
/u01/app/oracle/oradata/london/dev1
[oracle@host03]$ mkdir -p
/u01/app/oracle/fast_recovery_area/london
```


Create an initial directory structure for the physical standby database in the `$ORACLE_BASE/admin` and `$ORACLE_BASE/oradata` locations. The directories to be created depend on whether the primary database is using file locations for data files or Automatic Storage Manager. If multi-tenant architecture is being used, additional directories are needed. The above example is for a file system–based, multi-tenant installation using a single pluggable database named `DEV1`. Additional pluggable databases on the primary server will require additional directories to be created if the file system is used for storage.

Starting the Physical Standby Database

Start the physical standby database instance in NOMOUNT mode:

```
SQL> startup nomount
pfile=$HOME/dbs/initlondon.ora
ORACLE instance started.

Total System Global Area  150667264 bytes
Fixed Size                  1298472 bytes
Variable Size              92278744 bytes
Database Buffers           50331648 bytes
Redo Buffers                6758400 bytes
```


Set the ORACLE_SID environment variable to your physical standby database. Start the physical standby database instance in NOMOUNT mode by using the text initialization parameter file. With ASM installed, there will be multiple software home locations on each machine. This will require that the ORACLE_HOME and PATH location change accordingly. Oracle recommends the oraenv utility to change environment variables provided entries exist in the /etc/oratab file. The oraenv utility will adjust ORACLE_SID, ORACLE_BASE, ORACLE_HOME, PATH, and LD_LIBRARY_PATH environment variables. The ORACLE_HOME variable should point to the Grid Infrastructure software directories when starting the listener by using the LSNRCTL utility. However, the ORACLE_HOME variable should point to the database software directories when starting the database.

Note: Because the initialization parameter file contains only entries for DB_NAME and DB_DOMAIN, memory sizes for the System Global Area will use default values. Later the DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE RMAN command will copy the initialization parameter values for memory sizing from the primary database configuration.

Creating an RMAN Script to Create the Physical Standby Database

Create an RMAN script to create the physical standby database:

```
run {
    allocate channel prmy1 type disk;
    allocate channel prmy2 type disk;
    allocate channel prmy3 type disk;
    allocate channel prmy4 type disk;
    allocate auxiliary channel stby type disk;

    duplicate target database for standby
       from active database
```

Note: The script continues in the next slide.

Create an RMAN script containing the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` command.

There are advantages to using RMAN to create the standby database. They include:

- RMAN can create a standby database by copying the files currently in use by the primary database. No backups are required.
- RMAN can create a standby database by restoring backups of the primary database to the standby site. Thus, the primary database is not affected during the creation of the standby database.
- RMAN automates renaming of files, including Oracle Managed Files (OMF) and directory structures.
- RMAN restores archived redo log files from backups and performs media recovery so that the standby and primary databases are synchronized.

Note: You can use the `CONFIGURE … PARALLELISM` *`integer`* command to configure automatic channels for the specified device type. For additional information, see the *Oracle Database Backup and Recovery Reference*.

Creating an RMAN Script to Create the Physical Standby Database

```
spfile
  parameter_value_convert 'boston','london'
  set db_unique_name='london'
  set db_file_name_convert='boston','london'
  set log_file_name_convert='boston','london'
  set log_archive_max_processes='10'
  set fal_server='boston'
  set log_archive_config='dg_config=(boston,london)'
  set log_archive_dest_2='service=boston ASYNC
      valid_for=(ONLINE_LOGFILE,PRIMARY_ROLE)
      db_unique_name=boston'
nofilenamecheck;
}
```


In the RMAN script, specify the settings for the physical standby initialization parameters. You only need to specify parameters that are different than those on the primary database. Parameters related to directory paths for data files, control files, and online redo logs that are different do not have to be specified if `DB_FILE_NAME_CONVERT` and `LOG_FILE_NAME_CONVERT` are used.

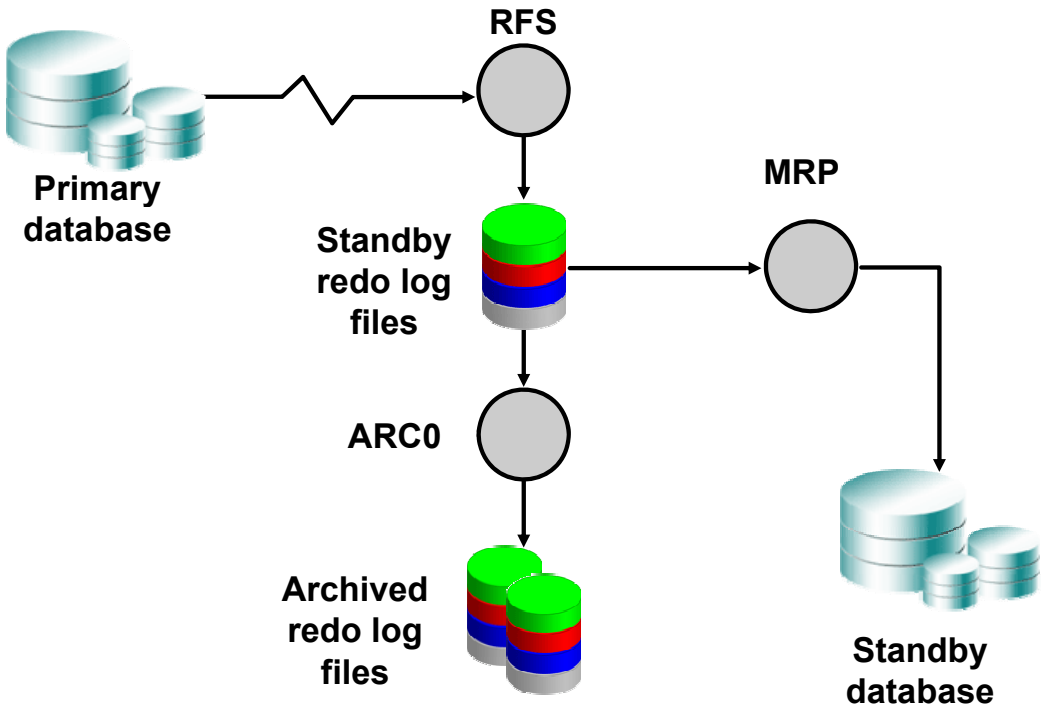
Creating the Physical Standby Database

1. Invoke RMAN and connect to the primary database and the physical standby database.
2. Execute the RMAN script (previous two slides) to create the physical standby database.

```
RMAN> connect target sys/oracle_4U@boston
RMAN> connect auxiliary sys/oracle_4U@london
RMAN> @cr_phys_standby
```


Connect to the primary database instance (target) and physical standby database instance (auxiliary). Execute the script that you created. The script can be run using the RMAN utility either on the primary database or standby database.

Real-Time Apply (Default)



With the real-time apply feature, log apply services apply the redo data from standby redo log files in real time (at the same time the log files are being written to) as opposed to recovering redo from archived redo log files when a log switch occurs. If for some reason the apply service is unable to keep up (for example, if you have a physical standby in read-only mode for a period of time), then the apply service automatically goes to the archived redo log files as needed. The apply service also tries to catch up and go back to reading the standby redo log files as soon as possible.

Real-time application of redo information provides a number of benefits, including faster switchover and failover operations, up-to-date results after you change a physical standby database to read-only, up-to-date reporting from a logical standby database, and the ability to leverage larger log files on the primary database resulting in larger standby redo logs on the standby database.

Having larger log files with real-time apply is desirable because the apply service stays with a log longer and the overhead of switching has less impact on the real-time apply processing.

The `RECOVERY_MODE` column of the `V$ARCHIVE_DEST_STATUS` view contains the value `MANAGED REAL TIME APPLY` when log apply services are running in real-time apply mode.

If you define a delay on a destination (with the `DELAY` attribute) and use real-time apply, the delay is ignored.

For physical standby databases, the managed recovery process (MRP) applies the redo from the standby redo log files after the remote file server (RFS) process finishes writing.

Note: Beginning with Oracle Database 12*c*, real-time apply is enabled by default during Redo Apply. You can disable real-time apply by stopping the Redo Apply process, and you can restart it with the `USING ARCHIVED LOGFILE` clause as follows:

```
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE CANCEL;
ALTER DATABASE RECOVER MANAGED STANDBY DATABASE USING ARCHIVED
LOGFILE;
```

Starting Redo Apply in Real-Time

- Execute the following command on the standby database to start Redo Apply in real time:

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY
DATABASE DISCONNECT;
```

- To disable real-time Redo Apply:

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY
DATABASE CANCEL;

SQL> ALTER DATABASE RECOVER MANAGED STANDBY
DATABASE USING ARCHIVED LOGFILE DISCONNECT;
```

ORACLE

On the standby database, issue the `ALTER DATABASE RECOVER MANAGED STANDBY DATABASE` SQL command to start Redo Apply. This statement automatically mounts the database. In addition, include the `DISCONNECT FROM SESSION` option so that Redo Apply runs in a background session. The `FROM SESSION` portion of syntax is no longer needed, but is acceptable.

The transmission of redo data to the remote standby location does not occur until after a log switch. Issue the following command on the primary database to force a log switch:

```
SQL> ALTER SYSTEM SWITCH LOGFILE;
```

Note: The syntax option `USING CURRENT LOGFILE` of the `ALTER DATABASE RECOVER MANAGED STANDBY DATABASE` statement has been deprecated for Oracle Database 12*c* Release 1.

Preventing Primary Database Data Corruption from Affecting the Standby Database

- Oracle Database processes can validate redo data before it is applied to the standby database.
- Corruption detection checks occur on the primary database during redo transport and on the standby database during Redo Apply.
- Implement lost-write detection by setting `DB_LOST_WRITE_PROTECT` to `TYPICAL` on the primary and standby databases.

Data Guard uses Oracle processes to validate redo data before it is applied to the standby database.

Corruption-detection checks occur at the following key interfaces:

- On the primary database during redo transport by the LGWR, LNS, and ARC*n* processes
- On the standby database during Redo Apply by the RFS, ARC*n*, MRP, and DBW*n* processes

If redo corruption is detected by Redo Apply at the standby database, Data Guard will re-fetch valid logs as part of archive log gap handling.

A lost write occurs when an I/O subsystem acknowledges the completion of a write but the write did not occur in persistent storage. On a subsequent block read, the I/O subsystem returns the stale version of the data block, which is used to update other blocks of the database, thereby corrupting the database.

Set the `DB_LOST_WRITE_PROTECT` initialization parameter on the primary and standby databases to enable the database server to record buffer cache block reads in the redo log so that lost writes can be detected.

You can set DB_LOST_WRITE_PROTECT as follows:

- **TYPICAL on the primary database:** The instance logs buffer cache reads for read/write tablespaces in the redo log.
- **FULL on the primary database:** The instance logs reads for read-only tablespaces as well as read/write tablespaces.
- **TYPICAL or FULL on the standby database or on the primary database during media recovery:** The instance performs lost-write detection.
- **NONE on either the primary database or the standby database (the default):** No lost-write detection functionality is enabled.

When a standby database applies redo during managed recovery, it reads the corresponding blocks and compares the system change numbers (SCNs) with the SCNs in the redo log before doing the following:

- If the block SCN on the primary database is lower than on the standby database, it detects a lost write on the primary database and returns an external error (ORA-752).
- If the SCN is higher, it detects a lost write on the standby database and returns an internal error (ORA-600 3020).

In both cases, the standby database writes the reason for the failure in the alert log and trace file.

The recommended procedure to repair a lost write on a primary database is to fail over to the physical standby and re-create the primary. To repair a lost write on a standby database, you must re-create the standby database or affected files.

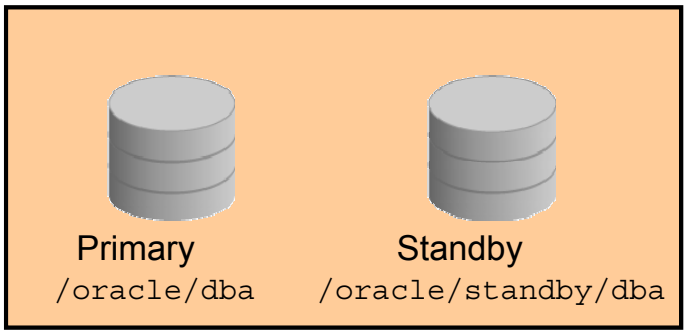
Special Note: Data Guard Support for Oracle Multitenant

- A multitenant container database (CDB) can have a physical standby database and/or a logical standby database.
- Database role is defined at the CDB level only.
- Individual pluggable databases (PDBs) do not have their own roles.
- Role transitions are executed at CDB level.
- DDL related to role changes is executed in the root container (CDB$ROOT) of the CDB.
- PDBs created from XML files or clones from a different PDB will need their data files manually copied to the standby host.

ORACLE

Data Guard includes support for the multitenant architecture in Oracle Database 12*c* Release 1 (12.1). Data Guard is managed at the CDB level. Individual PDBs cannot have a different database role than that of the CDB. Role transitions such as switchover and failover are performed at the CDB level. A primary database that is a CDB can have both physical and logical standby databases. You are not required to have the same set of PDBs at the primary database and standby. However, only tables that exist in the same container at both the primary and standby are replicated.

Special Note: Standby Database on the Same System



- Standby database data files must be at a different location.
- Each database instance must archive to different locations.
- Service names must be unique.
- This standby database does not protect against disaster.

If you have a standby database on the same system as the primary database, you must use the following guidelines:

- The data files must be renamed. The actual file names can be the same, but at least the directory path must be different. This means that you must use the `DB_FILE_NAME_CONVERT` and `LOG_FILE_NAME_CONVERT` parameters.

  Note: If the standby database uses Oracle Managed Files (OMF), do not set the `DB_FILE_NAME_CONVERT` or `LOG_FILE_NAME_CONVERT` parameters.
- If a standby database is located on the same system as the primary database, the archival directories for the standby database must use a different directory structure than the primary database. Otherwise, the standby database may overwrite the primary database files.
- If you do not explicitly specify unique service names and if the primary and standby databases are located on the same system, the same default global name (consisting of the database name and domain name from the `DB_NAME` and `DB_DOMAIN` parameters) will be in effect for both the databases.
- If the standby database is on the same system as the primary database, it does not protect against disaster. A disaster is defined as total loss of the primary database system. If the standby database is on the same system, it will be lost as well. *This configuration should be used only for testing and training purposes.*

Creating a Physical Standby Without Using RMAN

- Create a backup copy of the primary database data files.
- Create a control file for the standby database.

```
SQL> ALTER DATABASE CREATE STANDBY CONTROLFILE AS
'/tmp/london01.ctl';
```

- Create a parameter file from the server parameter file used by the primary database, and then adjust the parameters.

```
SQL> CREATE PFILE='/tmp/initlondon.ora' FROM
SPFILE;
```

- Transfer the complete backup, standby control file, and parameter file to the standby system.
- Create a server parameter file and a startup instance.
- Restore the backup and start Redo Apply.

ORACLE

If the database is using the file system for storage instead of Automatic Storage Manager (ASM), then a physical standby database can be created without using RMAN. With ASM, RMAN will have to be used to create the backup.

You can use any backup copy of the primary database to create the physical standby database, as long as you have the necessary archived redo log files to completely recover the database. A standby control file will have to be created on the primary database, and transferred to the standby database. A text-based parameter file will need to be created from the primary database binary server parameter file. The file needs adjusting in the same way as shown earlier in the lesson. It will then need to be transferred to the standby system and converted to a binary server parameter file.

Quiz

The `DB_FILE_NAME_CONVERT` parameter is required when Oracle Managed Files (OMF) are being used exclusively.

a. True
b. False

Answer: b

Quiz

A standby database cannot be created on the same system as the primary database.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- Enable `FORCE LOGGING`
- Create standby redo logs
- Set initialization parameters on the primary database to support the creation of the physical standby database and role transition
- Configure Oracle Net Services
- Create a physical standby database by using the `DUPLICATE TARGET DATABASE FOR STANDBY FROM ACTIVE DATABASE` RMAN command
- Start the transport and application of redo

Practice 4: Overview

This practice covers the following topics:

- Preparing the primary database prior to creating a physical standby database
- Creating the physical standby database
- Verifying that the physical standby database is performing correctly

5

Using Oracle Active Data Guard

Objectives

After completing this lesson, you should be able to:

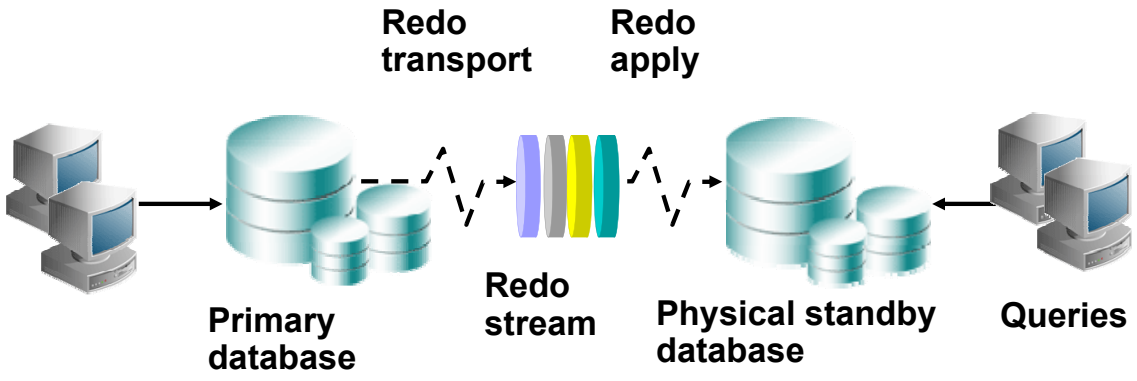
- Use real-time query to access data on a physical standby database
- Enable RMAN block change tracking for a physical standby database
- Use Far Sync to extend zero data loss protection for intercontinental configurations

Oracle Active Data Guard

- Is an option for Oracle Database 12*c* Enterprise Edition
- Enhances quality of service by offloading resource-intensive activities from a production database to a standby database
- Includes the following features:
  - Physical standby with real-time query
  - Fast incremental backup on physical standby
  - Automatic block repair
  - Active Data Guard Far Sync
  - Global Data Services
  - Real-time cascade
  - Application continuity
  - Rolling upgrade using Active Data Guard

Oracle Active Data Guard increases performance, availability, data protection, and return on investment wherever Data Guard is used for real-time data protection and availability. An Oracle Active Data Guard standby database can be used to offload a primary database of reporting, ad hoc queries, data extracts, and backups, making it a very effective way to insulate interactive users and critical business tasks on the production system from the overhead of long-running operations. Oracle Active Data Guard provides read-only access to a physical standby database while it is synchronized with a primary database, enabling minimal latency between reporting and production data. Unlike other replication methods, Active Data Guard is very simple to use, transparently supports all data types, and offers very high performance with complete read consistency at the reporting database. Oracle Active Data Guard automatically repairs physical corruption on either the primary or standby database, increasing availability and maintaining data protection at all times. Oracle Active Data Guard 12*c* reduces downtime for Oracle Database upgrades and other database maintenance while avoiding error-prone manual procedures. Oracle Active Data Guard 12*c* Far Sync enables zero data loss disaster protection (DR) across any distance without impacting database performance. Oracle Active Data Guard 12*c* also includes Global Data Services to extend the concepts of automated service failover and workload management to globally distributed systems, and application continuity to provide transparent failover of in-flight transactions in Data Guard configurations that do not use Oracle Real Application Cluster (Oracle RAC).

Using Real-Time Query



With Oracle Active Data Guard, you can use a physical standby database for queries while redo is applied to the physical standby database. This feature enables you to use a physical standby database for disaster recovery and to offload work from the primary database during normal operation. The physical standby is in a read-only mode, so no additional indexes or materialized views may be created to support reporting activities.

Note: If you need to create additional structures (such as indexes and materialized views), you can create a logical standby database as described in the lesson titled “Creating a Logical Standby Database.”

In addition, this feature provides a loosely coupled read/write clustering mechanism for OLTP workloads when configured as follows:

- **Primary database:** Recipient of all update traffic
- **Several readable standby databases:** Used to distribute the query workload

The physical standby database can be opened in read-only mode only if all files were recovered up to the same system change number (SCN). Otherwise, the open fails.

Enabling Real-Time Query

1. Stop Redo Apply:

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
CANCEL;
```

2. Open the database for read-only access:

```
SQL> ALTER DATABASE OPEN READ ONLY;
```

3. Restart Redo Apply with the real-time default option:

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
DISCONNECT;
```


A physical standby database instance cannot be opened if Redo Apply is active on a mounted instance of that database. In order to enable real-time query:

- Stop the Redo Apply process
- Open the database for read-only access
- Restart the Redo Apply with the real-time option that is now the default mode for Oracle Database 12*c* Release 1.

The `COMPATIBLE` database initialization parameter must be set to 11.0 or higher to use the real-time query feature of the Oracle Active Data Guard option.

Note: When using the Oracle Data Guard broker, it is not necessary to stop Redo Apply and restart Redo Apply when enabling real-time query. Real-time query is not the same as real-time apply, which was covered in the previous lesson. Real-time apply allows the recovery mechanisms to read from the standby redo logs at the same time that redo is being written to the standby redo logs. In the normal physical standby mode of operation, the database is only at the mount mode and would not allow any queries against user tables, even though real-time apply is enabled. Real-time query with Oracle Active Data Guard extends real-time apply by allowing the database to be opened and queries performed against it.

Disabling Real-Time Query

1. Shut down the standby database instance.

```
SQL> SHUTDOWN IMMEDIATE;
```

2. Restart the standby database instance in MOUNT mode.

```
SQL> STARTUP MOUNT;
```

3. Restart Redo Apply services (real-time apply).

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
DISCONNECT;
```


To disable real-time query, you must shut down the standby database instance and restart it in MOUNT mode.

Checking the Standby's Open Mode

- A physical standby database opened in read-only mode:

```
SQL> SELECT open_mode FROM V$DATABASE;
OPEN_MODE
--------------------
READ ONLY
```

- A physical standby database opened in real-time query mode:

```
SQL> SELECT open_mode FROM V$DATABASE;
OPEN_MODE
--------------------
READ ONLY WITH APPLY
```

ORACLE

You can use the `OPEN_MODE` column of `V$DATABASE` to check the open mode of a physical standby database. If the physical standby database stops Redo Apply in order to open the database in read-only mode, then the `OPEN_MODE` column will indicate "`READ ONLY`."

After the database has been opened read-only, Redo Apply can be restarted to enable Active Data Guard real-time query mode with the following command:

```
SQL> alter database recover managed standby database disconnect;
```

After Redo Apply has been started on an open read-only physical standby database, the `OPEN_MODE` column will indicate "`READ ONLY WITH APPLY`."

Understanding Lag in an Active Data Guard Configuration

- A standby database configured with real-time apply can lag behind the primary database as a result of:
  - Insufficient CPU capacity
  - High network latency
  - Limited bandwidth
- Queries on the standby database need to return current results and/or be within an established service level.
- Ways to “manage” the standby database lag and take necessary action:
  - Configure Data Guard with a maximum data lag that will trigger an error when it is exceeded.
  - Monitor the Redo Apply lag and take action when the lag is unacceptable.

Active Data Guard can improve performance by off-loading a read-only workload to a physical standby database. However, due to hardware and network issues, the data on a standby database may lag behind the data on the primary database. The standby database may not always be current with the primary database if it does not have the capacity to apply redo as quickly as it is received. Limited bandwidth may prevent the primary database from shipping redo as quickly as it is generated, particularly during periods of peak workload.

Oracle Database 12*c* Release 1 (12.1) includes features to enable you to determine the lag time and take appropriate action.

You can establish a tolerance level for data staleness by configuring a maximum value for *apply lag*. Query results are returned to the application if the lag is within the acceptable tolerance level; otherwise, an error results.

If you determine that you want your application to receive the results of a query, regardless of the “staleness” of the data, you can monitor the apply lag via the `V$DATAGUARD_STATS` view and then take appropriate action if the lag is unacceptable.

Monitoring Apply Lag: V$DATAGUARD_STATS

- *Apply lag*: This is the difference, in elapsed time, between when the last applied change became visible on the standby and when that same change was first visible on the primary.
- The `apply lag` row of the `V$DATAGUARD_STATS` view reflects statistics that are computed periodically and to the nearest second.

```
SQL> SELECT name, value, datum_time, time_computed
  2> FROM v$dataguard_stats
  3> WHERE name like 'apply lag';

NAME        VALUE           DATUM_TIME            TIME_COMPUTED
---------   -------------   -------------------   -------------------
apply lag   +00 00:00:00    27-MAY-2009 08:54:16  27-MAY-2009 08:54:17
```


Apply lag is a measure of the degree to which a standby database lags behind the primary database, due to delays in propagating and applying redo to the standby database. The current apply lag is the difference, in elapsed time, between when the last applied change became visible on the standby and when that same change was first visible on the primary. This metric is computed to the nearest second.

The `apply lag` metric is computed using data that is periodically received from the primary database. The `DATUM_TIME` column contains a timestamp of when this data was last received by the standby database. The `TIME_COMPUTED` column contains a timestamp taken when the `apply lag` metric was calculated. The difference between the values in these columns should be less than 30 seconds. If the difference is larger than this, the `apply lag` metric may not be accurate.

Monitoring Apply Lag: V$STANDBY_EVENT_HISTOGRAM

- View the histogram of apply lag on a physical standby database.
- Assess the value for STANDBY_MAX_DATA_DELAY.
- Focus on periods of time when the apply lag exceeds desired levels so that issue can be resolved.

```
SQL> SELECT * FROM V$STANDBY_EVENT_HISTOGRAM
2> WHERE NAME = 'apply lag' AND COUNT > 0;

NAME             TIME       UNIT         COUNT        LAST_TIME_UPDATED
---------     ---------   --------    -----------    ------------------------
apply lag          0        seconds        79681          06/18/2009 10:05:00
apply lag          1        seconds         1006          06/18/2009 10:03:56
apply lag          2        seconds           96          06/18/2009 09:51:06
apply lag          3        seconds            4          06/18/2009 04:12:32
apply lag          4        seconds            1          06/17/2009 11:43:51
apply lag          5        seconds            1          06/17/2009 11:43:52

6 rows selected
```


V$STANDBY_EVENT_HISTOGRAM is a new view that is available on the standby database. This view displays the histogram of the apply lag on the physical standby database.

Use the histogram to focus on periods of time when the apply lag exceeds desired levels. Determine the cause of the lag during those time periods and take steps to resolve the excessive lag. To evaluate the apply lag over a time period, take a snapshot of V$STANDBY_EVENT_HISTOGRAM at the beginning of the time period and compare that snapshot with one taken at the end of the time period.

Each distinct value of apply lag has its own bucket. The count in the bucket is incremented when the physical standby database samples its data delay.

Allowed Staleness of Standby Query Data

- The STANDBY_MAX_DATA_DELAY session parameter specifies a session-specific limit for the amount of time (in seconds) allowed to elapse between when changes are committed on the primary and when those same changes can be queried on the standby database.

```
ALTER SESSION
SET STANDBY_MAX_DATA_DELAY = {INTEGER|NONE}
```

- If the limit is exceeded, an error message is returned:

  ORA-3172 STANDBY_MAX_DATA_DELAY has been exceeded

- This setting is ignored for the SYS user.

You can configure a limit through the use of the STANDBY_MAX_DATA_DELAY session parameter. Use this session parameter to specify a limit for the amount of time (in seconds) allowed to elapse between when changes are committed on the primary database and when those same changes can be queried on the active standby database.

If the specified limit cannot be met, an error is returned to the query as follows:

ORA-3172 STANDBY_MAX_DATA_DELAY has been exceeded

This guarantees that a query will not receive a “stale result” if the apply lag exceeds the service level agreement. In addition, a warning message is written to the standby database alert log.

The default value is NONE, which indicates that queries issued to the physical standby database will be executed regardless of the apply lag on that database.

Configuring Zero Lag Between the Primary and Standby Databases

- Certain applications have zero tolerance for any lag.
- A query on the standby database must return the same result as if it were executed on the primary database.
- Enforce by setting `STANDBY_MAX_DATA_DELAY` to `0`.
- The standby database must have advanced to a value equal to that of the current SCN on the primary database at the time the query was issued.
- Results are guaranteed to be the same as the primary database; otherwise, an `ORA-3172` error is returned.
- The primary database must operate in maximum availability or maximum protection mode.
- `SYNC` must be specified for redo transport.
- Real-time query must be enabled.

ORACLE

You can ensure that your application querying data on the standby database sees all data that has been committed on the primary database by setting `STANDBY_MAX_DATA_DELAY` to `0`.

A query does not execute until the *query SCN* on the standby database has advanced to a value equal to that of the current SCN on the primary database at the time the query was issued.

To support zero lag, the primary database must operate in maximum availability or maximum protection mode. Protection modes will be discussed later in the course.

Specify `SYNC` for redo transport mode.

Real-time query must be enabled as a prerequisite for configuring zero lag.

Setting STANDBY_MAX_DATA_DELAY by Using an AFTER LOGON Trigger

Create an AFTER LOGON trigger that:

- Is *database role–aware*
  - It uses DATABASE_ROLE, a new attribute in the USERENV context.
  - SQL and PL/SQL clients can retrieve the database role programmatically using the SYS_CONTEXT function.
  - It enables you to write role-specific triggers.
- Sets STANDBY_MAX_DATA_DELAY when the application logs on to a real-time query–enabled standby database
- Allows for configuration of a maximum data delay without changing the application source code

You can create an AFTER LOGON trigger that sets the STANDBY_MAX_DATA_DELAY session parameter when the database is a physical standby database that is operating in real-time query mode.

In Oracle Database 12*c* Release 1 (12.1), the DATABASE_ROLE attribute of the USERENV context enables you to determine the role of the database. SQL and PL/SQL clients can retrieve this information by using the SYS_CONTEXT function. This enables you to write triggers that perform certain actions based on the database role.

Example: Setting STANDBY_MAX_DATA_DELAY by Using an AFTER LOGON Trigger

```
CREATE OR REPLACE TRIGGER sla_logon_trigger
   AFTER LOGON
   ON APP.SCHEMA
BEGIN
   IF (SYS_CONTEXT('USERENV', 'DATABASE_ROLE')
       IN ('PHYSICAL STANDBY'))
   THEN execute immediate
     'alter session set standby_max_data_delay=5';
   ENDIF;
END;
```


The slide presents an example of an AFTER LOGON trigger that is used to set the value of STANDBY_MAX_DATA_DELAY based on the database role.

Forcing Redo Apply Synchronization

- The `ALTER SESSION SYNC WITH PRIMARY` command:
  - Performs a blocking wait on the standby database upon execution
  - Blocks the application until the standby database is in sync with the primary database as of the time this command is executed
- When the `ALTER SESSION SYNC WITH PRIMARY` command returns control, the session can continue to process queries without having to wait for standby Redo Apply.
- An `ORA-3173 Standby may not be synced with primary` error is returned if Redo Apply is not active or is canceled before the standby database is in sync with the primary database as of the time this command is executed.

You can execute the `ALTER SESSION SYNC WITH PRIMARY` command to ensure that the standby database is completely synchronized with the primary database at the time of execution. The use of this command is particularly applicable in a reporting environment.

`ALTER SESSION SYNC WITH PRIMARY` performs a blocking wait on the standby database upon execution. This command causes the application to be blocked until the standby database is in sync with the primary database as of the time this command is executed.

When the `ALTER SESSION` command returns control to the session, the session can continue to process queries without having to wait for Redo Apply on the standby database.

If Redo Apply is not active or is canceled before the standby database is in sync with the primary database, an `ORA-3173 Standby may not be synced with primary` error is returned.

Creating an AFTER LOGON Trigger for Synchronization

- Use an AFTER LOGON trigger to force a wait for synchronization between primary and standby databases.
- Use for dedicated connection only.
- This ensures that the reporting application starts with the current data without requiring a change to the application source code.

```
CREATE TRIGGER adg_logon_sync_trigger
  AFTER LOGON ON user.schema
    BEGIN
      IF (SYS_CONTEXT('USERENV','DATABASE_ROLE') IN
         ('PHYSICAL STANDBY'))
      THEN
        execute immediate 'alter session sync with primary';
      END IF;
    END;
```

ORACLE

This type of trigger is useful when you are using the standby database for reporting and want to be sure that the reports have the most current data. The standby-only AFTER LOGON trigger executes the ALTER SESSION SYNC WITH PRIMARY command to force a wait for synchronization between the primary database and the standby database. A standby-only trigger is created and enabled on the primary database, and then becomes part of the redo that is propagated to the standby database. However, the trigger logic is designed only to take certain actions if the database role is set to “physical standby.”

Active Data Guard: DML on Temporary Tables

Supported by a new initialization parameter in Oracle Database 12*c* for all databases:

```
TEMP_UNDO_ENABLED = false | true
```

- Separates undo for temporary tables from undo for the persistent table
  - Temporary undo is not logged in redo.
  - Temporary undo uses the temporary tablespace.
- Enables DML on temporary tables when using Active Data Guard
  - It is set by default on an Active Data Guard standby database.
  - Data definition language (DDL) to create temporary tables must be issued on the primary database.

Redo generation on a read-only database is not allowed. When a data manipulation language (DML) operation makes a change to a global temporary table, the change itself does not generate redo because it is only a temporary table. However, the undo generated for the change does, in turn, generate redo. Prior to Oracle Database 12*c* Release 1 (12.1), global temporary tables could not be used on Active Data Guard standbys, which are read-only.

However, in Oracle Database 12*c* Release 1 (12.1), the temporary undo feature allows the undo for changes to a global temporary table to be stored in the temporary tablespace instead of in the undo tablespace. Undo stored in the temporary tablespace does not generate redo, thus enabling redo-less changes to global temporary tables. This allows DML operations on global temporary tables on Oracle Active Data Guard standbys.

To enable temporary undo on the primary database, use the `TEMP_UNDO_ENABLED` initialization parameter. On an Active Data Guard standby, temporary undo is always enabled by default, so the `TEMP_UNDO_ENABLED` parameter has no effect.

The temporary undo feature requires that the database initialization parameter `COMPATIBLE` be set to 12.0.0 or higher. The temporary undo feature on Active Data Guard instances does not support temporary binary and character large objects.

Active Data Guard: Support for Global Sequences

- Sequences created using the default `CACHE` and `NOORDER` options can be accessed from an Active Data Guard standby database.
- When first accessed by the standby, the primary allocates a unique range of sequence numbers.
- When all sequences within a range have been used, the standby requests another range of numbers.
- Because each range assigned to a standby is unique, there is a unique stream of sequences across the entire Data Guard configuration.
- Sequences created with the `ORDER` and `NOCACHE` options cannot be accessed on an Active Data Guard standby database.

To support read-mostly applications by using Active Data Guard, you might have to use sequences with global temporary tables. In an Active Data Guard environment, sequences created by the primary database with the default `CACHE` and `NOORDER` options can be accessed from standby databases as well. When a standby database accesses such a sequence for the first time, it requests that the primary database allocate a range of sequence numbers. The range is based on the cache size and other sequence properties specified when the sequence was created. Then the primary database allocates those sequence numbers to the requesting standby database by adjusting the corresponding sequence entry in the data dictionary. When the standby has used all numbers in the range, it requests another range of numbers.

Because the standby's requests for a range of sequences involve a round-trip to the primary, be sure to specify a large enough value for the `CACHE` keyword when you create a sequence for an Active Data Guard standby. Otherwise, performance could suffer.

In addition, the terminal standby should have a defined `LOG_ARCHIVE_DEST_n` parameter that points back to the primary.

Active Data Guard: Support for Session Sequences

- Session sequences are specifically designed for use with global temporary tables that have session visibility.
  - They return a unique range of sequence numbers within a session.
  - Session sequences are not persistent. The state of the session sequences accessed during a session is lost when the session terminates.
- Session sequences are created by the primary database and are accessed on any read-write or read-only database.
- To create a session sequence:

```
SQL> CREATE SEQUENCE … SESSION;
```

ORACLE

A session sequence is a special type of sequence that is specifically designed to be used with global temporary tables that have session visibility. Unlike existing regular sequences (referred to as “global” sequences for the sake of comparison), a session sequence returns a unique range of sequence numbers only within a session, not across sessions. Session sequences are not persistent. If a session goes away, so does the state of the session sequences that were accessed during the session.

Session sequences support most of the sequence properties that are specified when the sequence is defined. However, the `CACHE/NOCACHE` and `ORDER/NOORDER` options are not relevant to them and are ignored.

Session sequences must be created by a read-write database, but they can be accessed on any read-write or read-only database (either a regular database temporarily open as read-only or a standby database).

Benefits: Temporary Undo and Sequences

- Reporting and other applications that are generally read-only but require nonpersistent write access to the database can be run on an Active Data Guard standby by using temporary tables.
- Temporary undo reduces the redo volume if it is also enabled on the primary database. Temporary undo:
  - Is not logged in redo
  - Improves primary database performance
  - Reduces network bandwidth consumption
  - Reduces standby I/O
- Applications that are read-only except for the requirement to generate unique sequences can be offloaded to an Active Data Guard standby database.

ORACLE

Prior to Oracle Database 12*c* Release 1 (12.1), the inability to use global temporary tables and sequences for Active Data Guard limited some read-mostly applications from being offloaded from the primary database to a standby database.

The new temporary undo and sequences support for Active Data Guard provides many benefits. Additional reporting workload can now be migrated to a standby system to reduce the overhead of system resources in general. Because temporary undo reduces the amount of redo generated, the amount of redo needed to be shipped to standby database systems is reduced and results in network performance improvements. The reduction in redo also implies less redo being written to standby redo logs and local archiving of standby redo logs. Therefore, performance improvements are realized on the primary and standby database systems and on the network between the two systems.

Supporting Read-Mostly Applications

- *Read-mostly applications* are predominantly read-only applications, but require limited read-write database access.
- Active Data Guard supports the read-only portion of read-mostly applications if writes are redirected to the primary database or a local database.
- Redirection of read-write workload does not require application code changes.
- Writes can be transparently redirected to the primary database if the application adheres to the following:
  - Modified objects must not be qualified by a schema name.
  - SQL commands must be issued directly from the client, not in stored procedures.

Reporting applications that are predominantly read-only, but require limited read-write database access are referred to as “read-mostly” applications. Active Data Guard enables a standby database to support the read-only portion of read-mostly applications if writes are redirected to the primary database or a local database.

Example: Transparently Redirecting Writes to the Primary Database

- Application characteristics:
  - Executes as user U
  - Reads table U.R (R) and writes to table U.W (W)
  - User S has S.R synonym for U.R and S.W synonym for U.W@primary.
- Create an AFTER LOGON trigger on the standby database:

```
CREATE TRIGGER adg_logon_switch_schema_trigger
  AFTER LOGON ON u.schema
  BEGIN
    IF (SYS_CONTEXT('USERENV','DATABASE_ROLE')
        IN ('PHYSICAL STANDBY'))
    THEN
      execute immediate
        'alter session set current_schema = S';
    END IF;
  END;
```

ORACLE

Consider an application as described in the slide. The application executes as user U, reading the U.R table and writing to the U.W table. The application connects as user U when executing on the primary database.

User S accesses U.R with the S.R synonym and U.W with the S.W synonym.

The AFTER LOGON trigger is created on the standby database for another user, R.

When the application executes on the standby database, it connects as the U user. The AFTER LOGON trigger fires and transparently switches to the S schema. All reads on R execute as reads to the U.R table on the standby database. All reads and writes to W execute as reads and writes to U.W@primary.

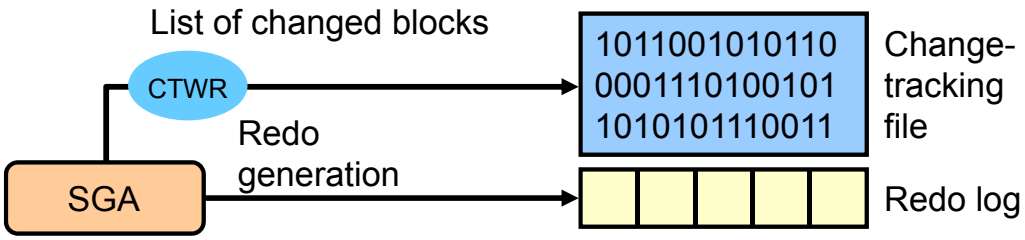
Enabling Block Change Tracking on a Physical Standby Database

- Enable block change tracking on a physical standby database for fast incremental backups.
- Data file blocks that are affected by each database update are tracked in a block change tracking file.
- The block change tracking file is a binary file used by RMAN to record changed blocks to improve incremental backup performance.

With the Oracle Active Data Guard option, you can enable block change tracking on a physical standby database. When you back up the physical standby database, RMAN uses the block change tracking file to quickly identify the blocks that have changed since the last incremental backup. RMAN reads only the changed blocks rather than the entire data file.

Creating Fast Incremental Backups

- Block change tracking optimizes incremental backups:
  - Tracks the blocks that have changed since the last backup
- Oracle Database has integrated change tracking:
  - A change tracking file is used.
  - Changed blocks are tracked as redo is generated.
  - Database backup automatically uses the changed-block list.



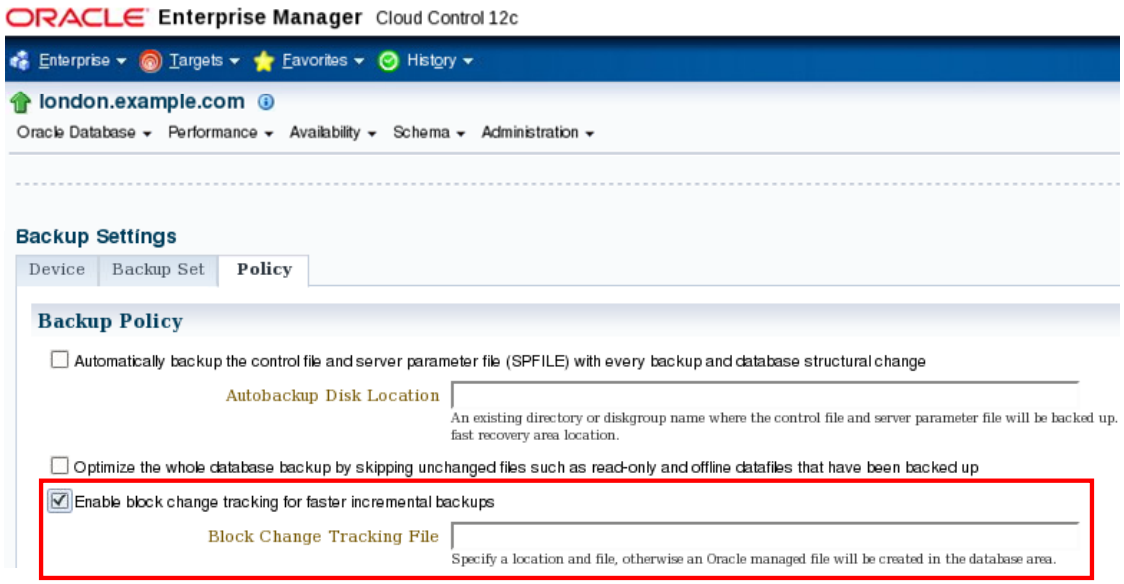
The goal of an incremental backup is to back up only those data blocks that have changed since a previous backup. You can use RMAN to create incremental backups of data files, tablespaces, or the entire database. During media recovery, RMAN examines the restored files to determine whether it can recover them from an incremental backup. RMAN always chooses incremental backups over archived redo logs because applying changes at a block level is faster than reapplying individual changes.

If you enable the block change tracking feature, Oracle Database tracks the physical location of all database changes in the change tracking file. RMAN uses this change-tracking data to determine which blocks to read during an incremental backup, creating much faster incremental backups by eliminating the need to read the entire data file. The maintenance of this file is fully automatic and does not require your intervention. The size of the change tracking file is proportional to the following:

- Database size in bytes
- Number of enabled threads in a RAC environment
- Number of old backups maintained by the change tracking file

The minimum size for the change tracking file is 10 MB; new space is allocated in 10 MB increments. By default, the Oracle Database server does not record block-change information.

Enabling Block Change Tracking




```
ALTER DATABASE
{ENABLE|DISABLE} BLOCK CHANGE TRACKING
[USING FILE '...']
```


You enable block change tracking from the database home page. Click the Policy tab on the Backup Settings page. You do not need to set the change tracking file destination if the `DB_CREATE_FILE_DEST` initialization parameter is set, because the file is created as an Oracle Managed File (OMF) file in the `DB_CREATE_FILE_DEST` location. You can, however, specify the name of the change tracking file and place it in any location you choose.

You can also enable or disable block change tracking by using an `ALTER DATABASE` SQL command. If the change tracking file is stored in the database area with your database files, it is deleted when you disable change tracking.

You can rename the change tracking file by using the `ALTER DATABASE RENAME` SQL command. Your database must be in the `MOUNT` state to rename the tracking file. The `ALTER DATABASE RENAME FILE` command updates the control file to refer to the new location. Use the following syntax to rename the change tracking file:

```
ALTER DATABASE RENAME FILE '...' TO '...';
```

Monitoring Block Change Tracking

```
SQL> SELECT filename, status, bytes
  2  FROM   v$block_change_tracking;
```

```
SQL> SELECT file#, avg(datafile_blocks),
  2         avg(blocks_read),
  3         avg(blocks_read/datafile_blocks)
  4             * 100 AS PCT_READ_FOR_BACKUP,
  5         avg(blocks)
  5  FROM   v$backup_datafile
  6  WHERE  used_change_tracking = 'YES'
  7  AND    incremental_level > 0
  8  GROUP  BY file#;
```


The output of the V$BLOCK_CHANGE_TRACKING view shows where the change tracking file is located, the status of block change tracking (ENABLED/DISABLED), and the size (in bytes) of the file.

Querying the V$BACKUP_DATAFILE view shows the effectiveness of block change tracking in minimizing the incremental backup I/O (the PCT_READ_FOR_BACKUP column). A high value indicates that RMAN reads most blocks in the data file during an incremental backup. You can reduce this ratio by decreasing the time between incremental backups.

Sample Formatted Output from the V$BACKUP_DATAFILE Query

FILE#	BLOCKS_IN_FILE	BLOCKS_READ	PCT_READ_FOR_BACKUP	BLOCKS_BACKED_UP
1	56320	4480	7	462
2	3840	2688	70	2408
3	49920	16768	33	4457
4	640	64	10	1
5	19200	256	1	91

Data Guard 12*c*: Far Sync

What is a Far Sync?

- A lightweight Oracle database instance
  - Only a standby control file, password file, standby redo logs, and archive logs
  - No Oracle data files, no database to open for access, not running Redo Apply
- Deployed within a distance such that the primary can tolerate the impact of network latency on synchronous transport
  - Looks like any other Data Guard destination to the primary database
  - With Data Guard 12*c* Fast Sync, further extends the practical distance between the primary and a Far Sync

ORACLE

A Data Guard Far Sync is a remote Data Guard destination that accepts redo from the primary database and redistributes that redo throughout the Data Guard configuration. It is similar to a physical standby database in that it manages a control file, receives redo into Standby Redo Logs (SRLs), and archives those SRLs to local Archived Redo Logs (ARLs). However, unlike a standby database, a Far Sync does not manage data files, cannot be opened for access, and cannot run Redo Apply. These limitations yield the benefit of using fewer disk and processing resources. More importantly, a Far Sync provides the ability to fail over to a terminal database with no data loss.

Many variables, such as the redo write size, available network bandwidth, round-trip network latency, and standby I/O performance while writing to the standby redo logs, impose practical limitations on the distance that a standby can reside from a primary database. This includes a Far Sync. A Data Guard 12*c* new feature called Fast Sync removes the standby I/O performance from the limitations list and helps to extend the distance that a Far Sync can reside from the primary database. Data Guard 12*c* Fast Sync will be discussed later in the course.

Far Sync: Redo Transport

The diagram in the slide displays a flat projection of a world map showing North America, South America, Europe, and Africa. Somewhere in North America, an icon in the slide represents the primary database system. Somewhere in Europe, an icon in the slide represents a standby database system. A short distance from the primary database system is a new type of instance called a Far Sync that is introduced as a new feature with Data Guard in Oracle Database 12*c*.

The redo transport uses synchronous (`SYNC`) transmission between the primary database system and a Far Sync. This imposes a practical limit on the distance between the primary database system and the Far Sync instance because it impacts the performance on the primary database. The log writer (`LGWR`) process of the primary database system has to wait for confirmation from the Network Server SYNC (`NSS`) process that the redo has been transmitted over the network before it can proceed with the next transaction. Redo transport from the Far Sync to the standby database system uses asynchronous communication. This eliminates the requirement of waiting for acknowledgment from the Network Server ASYNC (NSA) process on the Far Sync instance, creating near zero performance impact because of network transmission even if intercontinental distances are involved.

Far Sync: Redo Transport

How does redo transport work with a Far Sync configuration?

- The Far Sync instance receives redo synchronously from the primary database.
- The Far Sync instance forwards redo asynchronously in real time to its final destination.
- Redo transport supports one local and up to 30 additional local or remote destinations.
- Oracle Recovery Manager (Oracle RMAN) deletion polices are used to automate archive log management.
- A Far Sync instance can also compress redo transport.
- An alternate Far Sync can be used for HA.

In maximum availability mode, the Far Sync instance is relatively close to the primary database to minimize network latency, and the primary database services the Far Sync by using `SYNC` transport.

When a primary database services a Far Sync by using `SYNC` transport, all committed redo resides on disk at the Far Sync. That way, the Far Sync can use one of the terminal standby destinations for a no-data-loss failover if the primary database is lost.

The Far Sync uses `ASYNC` transport to redistribute the incoming redo to terminal standbys that can be much farther away. This extends no-data-loss protection to destinations that are too far away for a primary database to feasibly service directly with `SYNC` transport because of the resulting degradation in transaction throughput. This is a case where a Far Sync is beneficial even if there is only one standby destination in the configuration.

The redo transport architecture for a Far Sync is configured like any other type of standby database: Use the `LOG_ARCHIVE_DEST_n` parameter, which supports up to 30 destinations. Other parameters, such as `DB_UNIQUE_NAME`, `LOG_ARCHIVE_CONFIG`, `FAL_SERVER`, `FAL_CLIENT`, `LOG_FILE_NAME_CONVERT`, and `DB_FILE_NAME_CONVERT`, are also configured in the same way as any other type of standby database. With the Oracle Database 12*c* Advanced Security option, redo transport compression is also available.

Far Sync: Alternate Redo Transport Routes


```
LOG_ARCHIVE_DEST_STATE_2='ENABLE'

LOG_ARCHIVE_DEST_2='SERVICE=bostonFS SYNC AFFIRM MAX_FAILURE=1
ALTERNATE=LOG_ARCHIVE_DEST_3 VALID_FOR=(ONLINE_LOGFILES,PRIMARY_ROLE)
DB_UNIQUE_NAME=bostonFS'

LOG_ARCHIVE_DEST_STATE_3='ALTERNATE'

LOG_ARCHIVE_DEST_3='SERVICE=london ASYNC ALTERNATE=LOG_ARCHIVE_DEST_2
VALID_FOR=(ONLINE_LOGFILES,PRIMARY_ROLE) DB_UNIQUE_NAME=london'
```


In the event that communication with the Far Sync instance is lost, you can optionally configure the terminal standby to automatically become the alternate destination. This will reduce the amount of data loss by allowing Oracle Data Guard to ship redo asynchronously directly from the primary to the terminal standby, temporarily bypassing the Far Sync instance.

This enables Oracle Data Guard to continue sending redo, asynchronously, to the terminal standby `london` when it can no longer send the redo directly to the Far Sync instance `bostonFS`. When the Far Sync instance becomes available again, Oracle Data Guard automatically resynchronizes the Far Sync instance `bostonFS` and returns to the original configuration in which the primary sends redo to the Far Sync instance and the Far Sync instance forwards that redo to the terminal standby. When the synchronization is complete, the alternate destination (`LOG_ARCHIVE_DEST_3` in the example) will again become dormant as the alternate.

Oracle Data Guard 12*c*: Far Sync Creation

- Create a control file using the mounted primary database:

```
SQL> ALTER DATABASE CREATE FAR SYNC INSTANCE CONTROLFILE
     AS '/tmp/boston1.ctl';
```

- Copy the parameter file (PFILE) and password file used by the primary database. (Several initialization parameters must also be modified for the Far Sync instance.)
- Copy the control file to the Far Sync system.
- Create standby redo log files as you would for any standby.
- Start the Far Sync instance and mount the control file by using standard syntax.
- The `DATABASE_ROLE` column in `V$DATABASE` will contain the value `FAR SYNC`.

You start creating a Far Sync by issuing a command on the primary database to create a new Far Sync instance control file. The primary database must be mounted or open to issue the command. The syntax for creating a Far Sync instance control file is:

`SQL> ALTER DATABASE CREATE FAR SYNC INSTANCE CONTROLFILE AS '/PATH/FILENAME';`

Next, copy the resulting control file, along with a copy of the primary database parameter file and the password file, to the machine that will host the Far Sync instance. It is assumed that the database software has already been installed on the machine that will host the Far Sync instance and a database listener has been configured. If the primary database uses a server parameter file (SPFILE), create a parameter file (PFILE) before copying the file. The following list of initialization parameters may need changing for the Far Sync instance:

`DB_UNIQUE_NAME`; `CONTROL_FILES`, `FAL_SERVER`, `LOG_ARCHIVE_CONFIG`, and `LOG_ARCHIVE_DEST_n`.

On both the primary and Far Sync systems, use Oracle Net Manager to create a network service name for the primary and standby databases that will be used by redo transport services. This name will be supplied in the `LOG_ARCHIVE_DEST_n` parameter. Use standard syntax to start and mount the Far Sync. After you mount the Far Sync, the `DATABASE_ROLE` column in the view `V$DATABASE` shows '`FAR SYNC`'.

Benefits: Far Sync

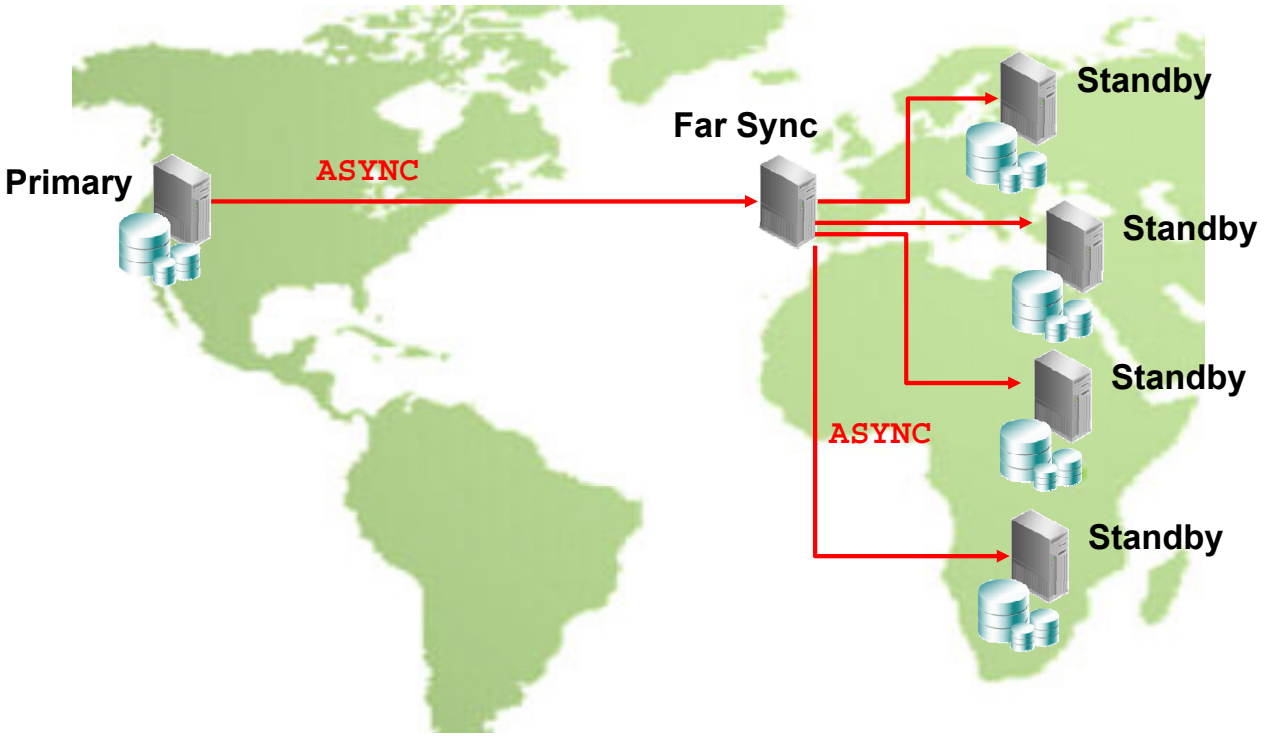
Provides no-compromise data protection and primary offload

- Ideal combination of attributes
  - Best data protection, least performance impact
  - Lower cost and complexity compared to previous multisite architecture
  - No change in applications required
  - Ideal for a combined near HA plus far DR model adopted by a growing number of users
- Relevant to existing Data Guard `ASYNC` configurations
  - Enables the implementation of SYNC zero data loss where only ASYNC was previously possible
  - Offloads overhead for redo transport compression and for servicing multiple destinations

There are many benefits of a Far Sync. A Far Sync transparently integrates in a maximum availability mode configuration for no-compromise data protection. It requires no changes in applications that currently use a primary database and standby database architecture. A Far Sync has the least performance impact, because it does not manage data files, cannot be opened for access, and cannot run Redo Apply. Features such as redo transport compression that would have a performance impact on a primary database can be offloaded to the Far Sync host.

It is ideal for an HA model serving as a nearby archived log repository, yet still allows the benefits of a far-away disaster recovery model by extending no-data-loss protection to destinations that are too far away for a primary database to feasibly service directly by using `SYNC` transport.

Far Sync: Alternate Design



Up to this point in this lesson, the discussion of a redo transport mechanism from a primary database to a Far Sync has used synchronous transmission. It is possible to also use asynchronous redo transport in a maximum performance Data Guard configuration.

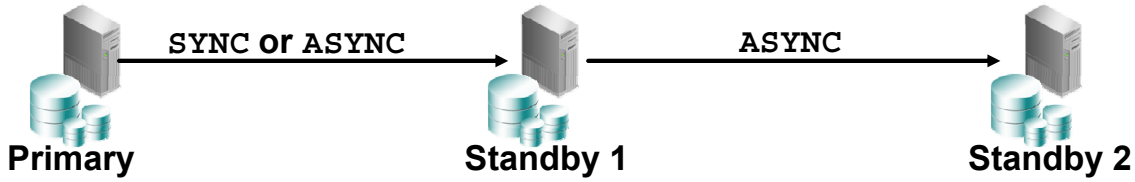
In maximum performance mode, the primary database services the Far Sync destination by using `ASYNC` redo transport, regardless of the physical distance between the primary and the Far Sync. High network latencies do not affect transaction throughput when a destination is serviced with ASYNC transport.

In maximum performance mode, a Far Sync can benefit Data Guard configurations that manage more than one remote destination. Each destination that a primary directly services takes computing resources away from applications running on the primary. When a Far Sync is used, the primary only has to service the Far Sync, which then services the rest of the configuration. The performance benefit increases as the number of destinations increases.

The slide shows a world map with a primary database in North America using `ASYNC` redo transport to a Far Sync located in Europe. The Far Sync is then used to cascade the primary redo to multiple standby sites located throughout Europe and Africa. Upto 30 terminal destinations are supported.

Real-Time Cascade

- Traditional cascaded standby database
  - Primary redo is shipped to Standby 1 and then forwarded to Standby 2.
  - The redo is forwarded at the log switch, making Standby 2 always in the past of Standby 1.
- Real-time cascade, new for Data Guard 12*c*
  - Standby 1 forwards the redo to Standby 2 in real time, as it is received.
  - There is no propagation delay of Standby 2 waiting for a primary log switch.



With traditional cascaded standby database configurations prior to Oracle Database 12*c*, primary database redo is transported to a standby database and that standby database cascades or forwards the primary redo to additional standby databases. However, traditionally the redo is not immediately cascaded. Primary database redo is written to the standby redo log as it is received at a cascading standby database. This redo was cascaded only after the standby redo log file that it was being written to had been archived locally.

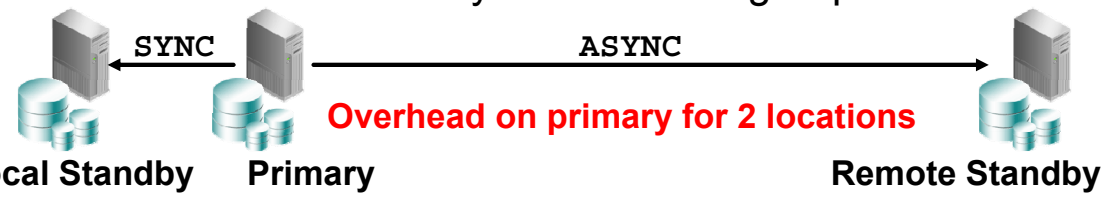
Because standby redo logs are usually configured with the same size as the primary database redo log files, the rate that a standby redo log switches and is archived locally should be about the same time frequency as the primary database log switch rate. Depending on the size of the redo logs and the rate of redo generation, this log switch could take hours. It is also common that the time interval between switches varies. Even when the redo logs were not full, forced log switches on the primary database were typically used to standardize the time interval of log switches. However, the time meant that the final cascaded standby database was guaranteed to lag behind the primary database.

As of Oracle Database 12*c* Release 1 (12.1), primary database redo is cascaded immediately, as it is being written to the standby redo log file. The terminal destination, therefore, has minimal redo transport lag with respect to the primary database. Only physical standbys and Far Syncs can cascade redo.

Traditional Multi-Standby Database Architecture

Multiple standby database configurations, the gold standard of WAN zero-data-loss architectures, consist of:

- A local SYNC standby database for zero-data-loss protection
  - Fast, local HA failover with zero data loss
  - Offload RMAN backups, read-only workload, data extracts
  - Real Application Testing using Data Guard Snapshot Standby
  - Database rolling maintenance and standby-first patching
- A remote ASYNC standby database for geo-protection



Multiple standby database configurations are very common in zero-data-loss architectures, where the primary database services one local standby database and one remote standby database. The local standby database may be located in the same data center with high-speed LAN connections using synchronous redo transport. This provides very fast HA failover in the event of a problem with the primary database system. It is also often used for offloading backups, read-only reporting, data extracts, rolling maintenance, and patching.

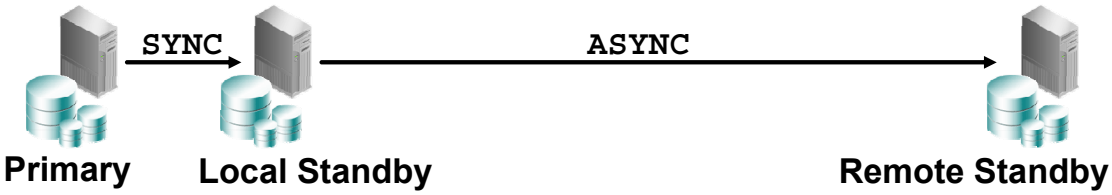
When the standby database is local to the primary database system, one disadvantage is site-wide disasters (for example, hurricanes or tornadoes could potentially impact entire data centers). To achieve zero data loss for all disasters, a second remote standby database is often used. The remote distance often necessitates asynchronous redo transport.

This multiple standby database configuration places additional performance overhead and burden on the primary database system because of the need to ship redo to two different locations. Traditional cascaded standby database configuration could not provide zero data loss for the remote standby database system because of redo transport lag.

Benefits: Real-Time Cascade

Multiple standby database configurations can now use real-time cascade.

- Less performance overhead
  - The primary database ships to a single destination.
- Less network volume at the primary database



With real-time cascade introduced in Oracle Database 12*c*, the terminal destination has minimal redo transport lag for the primary database. Therefore, the local standby can be used to cascade the redo to the remote standby. This reduces the volume of redo sent from the primary by half, which results in less performance impact on the primary database.

Quiz

The `STANDBY_MAX_DATA_DELAY` parameter can be set at the system level.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- Use real-time query to access data on a physical standby database
- Enable RMAN block change tracking for a physical standby database
- Use Far Sync to extend zero data loss protection for intercontinental configurations

Practice 5: Overview

This practice covers the following topics:

- Enabling Active Data Guard real-time query
- Enabling block change tracking
- Adding a Far Sync to the environment

6

Creating and Managing a Snapshot Standby Database

Objectives

After completing this lesson, you should be able to:

- Create a snapshot standby database to meet the requirement for a temporary, updatable snapshot of a physical standby database
- Convert a snapshot standby database back to a physical standby database

Snapshot Standby Databases: Overview

- A snapshot standby database is a fully updatable standby database created by converting a physical standby database.
- Snapshot standby databases receive and archive—but do not apply—redo data from a primary database.
- When the physical standby database is converted, an implicit guaranteed restore point is created and Flashback Database is enabled.

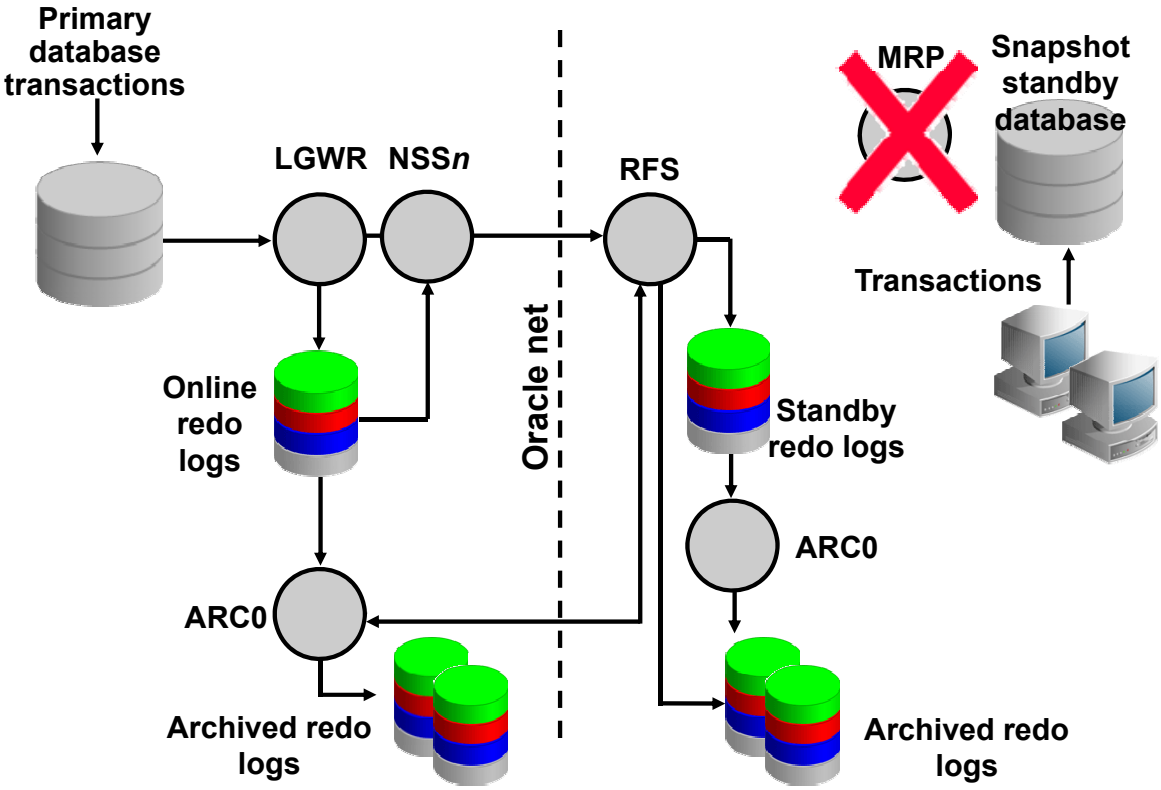
A snapshot standby database is a fully updatable standby database that is created by converting a physical standby database to a snapshot standby database. A snapshot standby database receives and archives—but does not apply—redo data from a primary database. Redo data received from the primary database is applied when a snapshot standby database is converted back to a physical standby database, after discarding all local updates to the snapshot standby database.

You can create a snapshot standby database by using DGMGRL commands or SQL commands.

When the standby database is converted to a snapshot standby database, an implicit guaranteed restore point is created and Flashback Database is enabled. After performing operations on the snapshot standby database, you can convert it back to a physical standby database. Flashback Database will be discussed later in the course.

Data Guard implicitly flashes the database back to the guaranteed restore point and automatically applies the primary database redo that was archived by the snapshot standby database since it was created. The guaranteed restore point is dropped after this process is completed.

Snapshot Standby Database: Architecture



After a physical standby database is converted to a snapshot standby database, Redo Apply no longer applies the redo data. The redo data continues to be received using the defined transport method (`SYNC` or `ASYNC`), but it is not applied until the snapshot standby database is converted back to a physical standby database.

Converting a Physical Standby Database to a Snapshot Standby Database

To convert a physical standby to a snapshot standby:

1. Stop Redo Apply if it is active.
2. Ensure that the database is mounted, but not open.
3. Ensure that the fast recovery area has been configured.
4. Issue the following SQL statement to perform the conversion:

```
SQL> ALTER DATABASE CONVERT TO SNAPSHOT STANDBY;
```

5. Open the snapshot standby database in read-write mode.

```
SQL> ALTER DATABASE OPEN;
```


The slide lists the steps necessary to convert a physical standby database to a snapshot standby database. Before converting the standby database, Redo Apply must be stopped if it is running. If Redo Apply is running, the following error message will be returned:

```
SQL> alter database convert to snapshot standby;
alter database convert to snapshot standby
*
ERROR at line 1:
ORA-38784: Cannot create restore point
   'SNAPSHOT_STANDBY_REQUIRED_08/16/2013
16:25:49'.
ORA-01153: an incompatible media recovery is active
```

Activating a Snapshot Standby Database: Issues and Cautions

When activating a snapshot standby database, be aware of:

- Potential data loss with a corrupted log file
- Lengthy conversion of the snapshot standby database to a primary database in the event of a failure of the primary database

Keep the following in mind when activating a snapshot standby database:

- **Potential data loss when a log file is corrupted:** The snapshot standby database accepts redo log files but does not apply them. If there is a corrupted redo log file at the snapshot standby database, it is not discovered until the database is converted back to a physical standby database and the managed recovery process (MRP) is started. If the primary database is unavailable at that time, there is no way to retrieve that log. Also, the loss or corruption of a flashback log file might prevent conversion back to a physical standby database.
- **Lengthy conversion of the snapshot standby database to a primary database:** In the event of a failure of the primary database, the snapshot standby database can be converted back to a physical standby database. The redo that has been received can then be applied, and the database can be converted to a primary database. If the snapshot standby database lags far behind the primary database, it may take a long time to apply the redo that has been received and convert it to the primary database.

Snapshot Standby Database: Target Restrictions

A snapshot standby database cannot be:

- The only standby database in a maximum protection configuration
- The target of a switchover
- A fast-start failover target

When you convert the physical standby database to a snapshot standby database, it cannot be the only standby database in the configuration if your configuration is in maximum protection mode. In addition, you cannot make changes to the configuration after converting to a snapshot standby database that would create this situation. Protection modes will be discussed later in the course.

You cannot perform a switchover to a snapshot standby database.

A snapshot standby database cannot be configured as a fast-start failover target.

Viewing Snapshot Standby Database Information

View the database role by querying V$DATABASE:

```
SQL> SELECT database_role FROM v$database;

DATABASE_ROLE

----------------

SNAPSHOT STANDBY
```


The DATABASE_ROLE column of the V$DATABASE view indicates that the database is a snapshot standby database.

Snapshot Standby Space Requirements

Monitor the space requirements for enabling snapshot standby:

```
SQL> select file_type, number_of_files,
   percent_space_used from v$recovery_area_usage;

FILE_TYPE           NUMBER_OF_FILES PERCENT_SPACE_USED
------------------- --------------- ------------------
CONTROL FILE                      0                  0
REDO LOG                          0                  0
ARCHIVED LOG                    106              41.81
BACKUP PIECE                      1                .17
IMAGE COPY                        0                  0
FLASHBACK LOG                     2                .98
FOREIGN ARCHIVED LOG              0                  0
AUXILIARY DATAFILE COPY           0                  0
```


Flashback Database does not have to be enabled to issue the command to convert a physical standby database to a snapshot database; however, the flash recovery area must be configured. The command will automatically enable Flashback Database and create a guaranteed restore point. This will result in a flashback logs being created in the flash recovery area. If there is not enough space in the flash recovery area to create the flashback logs, an error will occur and an error message written to the alert log. The longer the period of time that the standby database is in the snapshot mode, the larger the flashback logs will become. It will be necessary to monitor this space usage with the SQL statement presented in the above slide. The above query monitors overall space usage in the flash recovery area. To determine the specific space requirements for the guaranteed restore point that was created, the following query can be used:

```
SQL> SELECT NAME, STORAGE_SIZE FROM V$RESTORE_POINT;
NAME                                              STORAGE_SIZE
------------------------------------------------  ------------
SNAPSHOT_STANDBY_REQUIRED_08/16/2013 16:41:11       52428800
```

Converting a Snapshot Standby Database to a Physical Standby Database

To convert a physical standby to a snapshot standby:

1. On an Oracle Real Applications Cluster (Oracle RAC) database, shut down all but one instance.
2. Ensure that the database is mounted, but not open.
3. Issue the following SQL statement to perform the conversion:

```
SQL> ALTER DATABASE CONVERT TO PHYSICAL STANDBY;
```

ORACLE

Use the `CONVERT DATABASE` command to convert the database back to a physical standby database. The snapshot database must have been opened at least once in read-write mode before it can be converted back to a physical standby; otherwise the following error is returned:

```
SQL> alter database convert to physical standby;
alter database convert to physical standby
*
ERROR at line 1:ORA-16433: The database or pluggable database must
be opened in read/write mode.
```

Note: The above error was generated by attempting to immediately convert back to physical standby before ever opening the snapshot standby database.

If the snapshot standby is currently open, it must be shut down and started back up in the MOUNT mode to covert back to a physical standby. Otherwise the following error occurs:

```
ORA-01126: database must be mounted in this instance and not open
in any instance
```

Quiz

A snapshot standby database can be created from a logical standby database.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- Create a snapshot standby database
- Convert a snapshot standby database back to a physical standby database

Practice 6: Overview

This practice covers the following topics:

- Converting a physical standby database to a snapshot standby database
- Updating the primary database and the snapshot standby database
- Converting the snapshot standby database back to a physical standby database

7

Creating a Logical Standby Database

Objectives

After completing this lesson, you should be able to:

- Determine when to create a logical standby database
- Create a logical standby database
- Manage SQL Apply filtering

Benefits of Implementing a Logical Standby Database

- Provides an efficient use of system resources:
  - Open, independent, and active production database
  - Additional indexes and materialized views can be created for improved query performance
- Reduces workload on the primary database by offloading the following workloads to a logical standby database:
  - Reporting
  - Summations
  - Queries

A logical standby database provides benefits in disaster recovery, high availability, and data protection that are similar to those of a physical standby database. It also provides the following specialized benefits:

- **Efficient utilization of system resources:** A logical standby database is an open, independent, and active production database. It can include data that is not part of the primary database, and users can perform data manipulation operations on tables in schemas that are not updated from the primary database. It remains open while the tables are updated from the primary database, and those tables are simultaneously available for read access. Because the data can be presented with a different physical layout, additional indexes and materialized views can be created to improve your reporting and query requirements and to suit your specific business requirements.

  Note: Oracle Database 12*c* offers the Active Data Guard option. Active Data Guard includes the real-time query feature, which enables you to open a physical standby database in read-only mode while Redo Apply is active. However, you cannot add additional structures to the physical standby database as you can with a logical standby database.
- **Reduction in primary database workload:** The logical standby tables that are updated from the primary database can be used for other tasks (such as reporting, summations, and queries), thereby reducing the primary database workload.

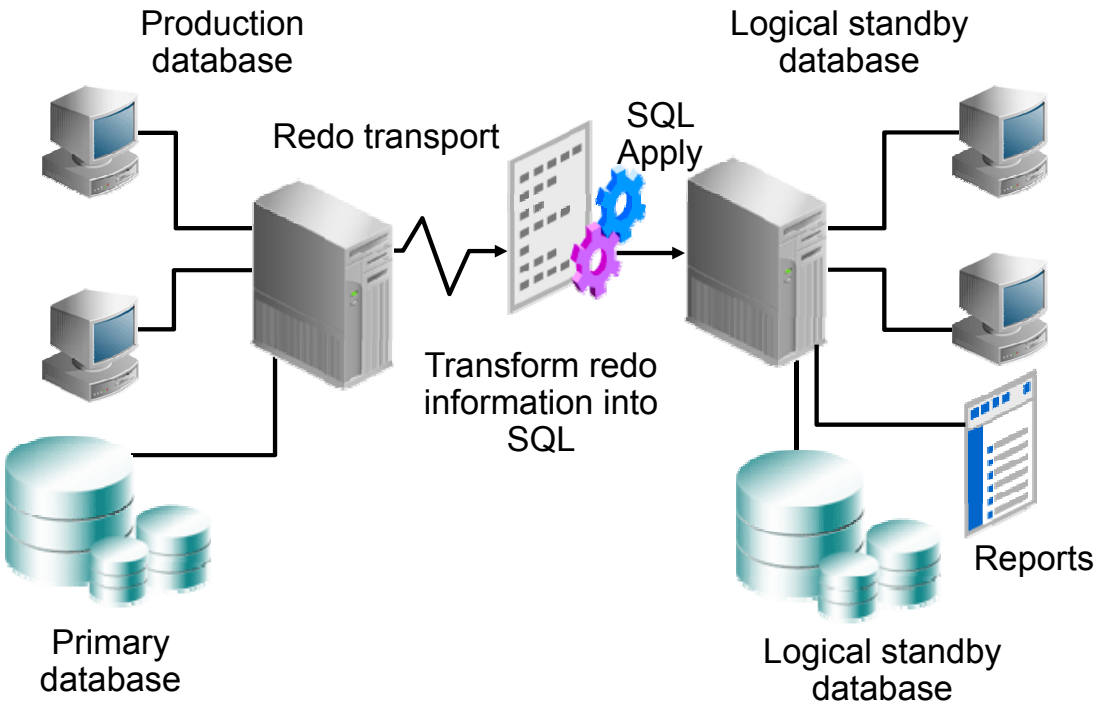
Benefits of Implementing a Logical Standby Database

- Provides data protection:
  - Primary database corruptions not propagated
- Provides disaster recovery capabilities:
  - Switchover and failover
  - Minimizes down time for planned and unplanned outages
- Can be used to upgrade Oracle Database software and apply patch sets

- **Data protection:** A logical standby database provides a safeguard against data corruption and user errors. Primary-side physical corruptions do not propagate through the redo data that are transported to the logical standby database. Similarly, user errors that may cause the primary database to be permanently damaged can be resolved before application on the logical standby through delay features.
- **Disaster recovery:** A logical standby database provides a robust and efficient disaster-recovery solution. Easy-to-manage switchover and failover capabilities enable easy role reversals between primary and logical standby databases, thereby minimizing the down time of the primary database for planned and unplanned outages.
- **Database upgrades:** A logical standby database can be used to upgrade Oracle Database software and apply patch sets with almost no down time. The logical standby database can be upgraded to the new release and switched to the primary database role. The original primary database is then converted to a logical standby database and upgraded.

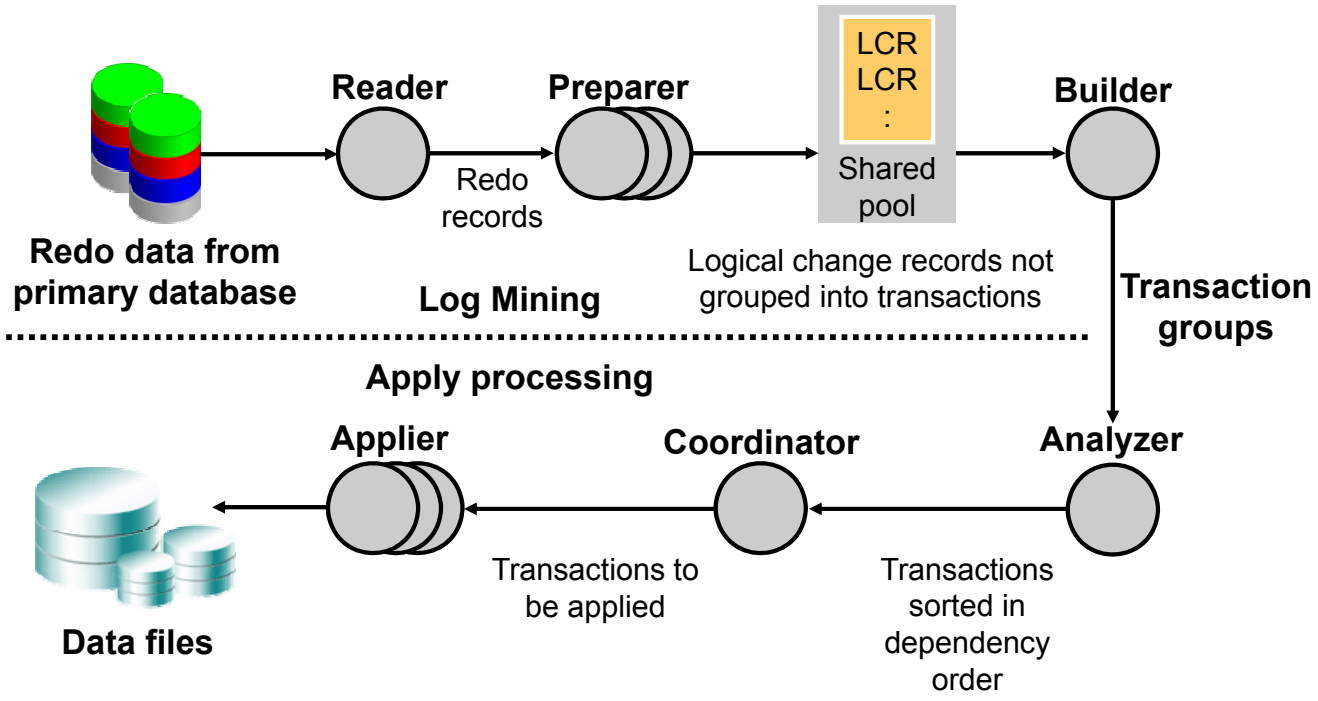
  Note: See the lesson titled “Patching and Upgrading Databases in a Data Guard Configuration” for additional information about using a logical standby database to perform upgrades.

Logical Standby Database: SQL Apply Architecture



In a logical standby database configuration, Data Guard SQL Apply uses redo information shipped from the primary system. However, instead of using media recovery to apply changes (as in the physical standby database configuration), archived redo log information is transformed into equivalent SQL statements by using LogMiner technology. These SQL statements are then applied to the logical standby database. The logical standby database is open in read/write mode and is available for reporting capabilities.

SQL Apply Process: Architecture



SQL Apply uses a collection of parallel execution servers and background processes that apply changes from the primary database to the logical standby database as follows:

- The reader process reads redo records from the archived redo log files.
- The preparer processes convert the block changes into table changes or logical change records (LCRs). At this point, the LCRs do not represent any specific transactions.
- The builder process assembles completed transactions from the individual LCRs.
- The analyzer process examines the records, possibly eliminating transactions and identifying dependencies between the different transactions.
- The coordinator process (LSP):
  - Assigns transactions
  - Monitors dependencies between transactions and coordinates scheduling
  - Authorizes the commitment of changes to the logical standby database
- The applier process:
  - Applies the LCRs to the database
  - Asks the coordinator process to approve transactions with unresolved dependencies, scheduled appropriately so that the dependencies are resolved.
  - Commits the transactions

Preparing to Create a Logical Standby Database

Perform the following steps on the primary database before creating a logical standby database:

- Check for unsupported objects.
- Be aware of unsupported DDL commands.
- Ensure row uniqueness.
- Verify that the primary database is configured for `ARCHIVELOG` mode.

When creating a logical standby database, you must take several actions before you begin. The next slides cover the preparation steps in detail.

Unsupported Objects

- User tables placed in internal schemas will not be maintained.
- Log apply services *automatically exclude* unsupported objects when applying redo data to the logical standby database.
- Unsupported objects:
  - Tables and sequences in the `SYS` schema
  - Tables used to support materialized views
  - Global temporary tables
  - Tables with unsupported data types (listed in the next slide)

If the primary database contains unsupported tables, log apply services automatically exclude these tables when applying redo data to the logical standby database.

Unsupported Data Types

- Log apply services *automatically exclude* entire tables with unsupported data types when applying redo data to the logical standby database.
- Unsupported data types:
  - `BFILE`
  - `ROWID` and `UROWID`
  - Collections (including `VARRAYS` and nested tables)
  - Objects with nested tables and `REFS`
  - The following Spatial types
    - `MDSYS.SDO_GEORASTER`
    - `MDSYS.SDO_TOPO_GEOMETRY`

Ensure that your logical standby database can support the data types of the database objects that are defined in your primary database.

Note: See *Oracle Data Guard Concepts and Administration* for additional information about unsupported data types and objects. SecureFile LOBs are supported if the database compatibility level is set to 11.2 or higher.

Identifying Internal Schemas

Query DBA_LOGSTDBY_SKIP on the primary database for internal schemas that will be skipped:

```
SQL> SELECT OWNER FROM DBA_LOGSTDBY_SKIP WHERE STATEMENT_OPT =
'INTERNAL SCHEMA' ORDER BY OWNER;
Owner
----------------------------------------
ANONYMOUS
APPQOSSYS
AUDSYS
BI
CTXSYS
…
SYSKM
SYSTEM
WMSYS
XDB
XS$NULL
30 rows selected.
```


Some schemas that ship with the Oracle database (for example, SYSTEM) contain objects that will be implicitly maintained by SQL Apply. However, if you put a user-defined table in SYSTEM, it will not be maintained even if it has columns of supported data types. Query the view DBA_LOGSTANDBY_SKIP to indentify internal schemas as shown in the slide. Tables in these schemas that are created by Oracle will be maintained on a logical standby if the feature implemented in the schema is supported in the context of logical standby. User tables placed in these schemas will not be replicated on a logical standby database and will not show up in the DBA_LOGSTDBY_UNSUPPORTED view.

Note: The query shown in the slide can return a different listing if it is run at the container level or the PDB level in a multitenant architecture. Because Data Guard standby databases are copies of the entire container database, you should run the query against each container database and all pluggable databases within the container.

For the demo database used in the practices that contains 1 PDB with the sample schemas installed, there are 29 internal schemas in the CDB, and 30 internal schemas in the PDB. The following query returns over 1875 individual tables:

```
SELECT owner,table_name FROM dba_tables WHERE owner IN (SELECT
owner FROM dba_logstdby_skip WHERE statement_opt = 'INTERNAL
SCHEMA');
```

Checking for Unsupported Tables

Query DBA_LOGSTDBY_UNSUPPORTED on the primary database for unsupported owners and tables:

```
SQL> SELECT DISTINCT OWNER,TABLE_NAME FROM
DBA_LOGSTDBY_UNSUPPORTED ORDER BY OWNER,TABLE_NAME;

OWNER                          TABLE_NAME
------------------------------ ------------------------------
IX                             AQ$_STREAMS_QUEUE_TABLE_G
IX                             AQ$_STREAMS_QUEUE_TABLE_H
...
OE                             CATEGORIES_TAB
OE                             CUSTOMERS
OE                             PURCHASEORDER
PM                             PRINT_MEDIA
SH                             DIMENSION_EXCEPTIONS

20 rows selected.
```


You can query DBA_LOGSTDBY_UNSUPPORTED_TABLE to determine which tables in the primary database are not supported by log apply services.

Note: User tables owned by the schemas identified in the previous slide will not show up in this query. This query will return different results if run against the container or pluggable database in a multitenant architecture.

Checking for Tables with Unsupported Data Types

Query DBA_LOGSTDBY_UNSUPPORTED on the primary database for tables with specific unsupported data types:

```
SQL> SELECT table_name, column_name, data_type FROM
dba_logstdby_unsupported WHERE owner = 'OE';

TABLE_NAME       COLUMN_NAME           DATA_TYPE
---------------- --------------------- ----------
CUSTOMERS        PHONE_NUMBERS         VARRAY
PURCHASEORDER    SYS_NC_ROWINFO$       OPAQUE
CATEGORIES_TAB   CATEGORY_NAME         VARCHAR2
CATEGORIES_TAB   CATEGORY_DESCRIPTION  VARCHAR2
CATEGORIES_TAB   CATEGORY_ID           NUMBER
CATEGORIES_TAB   PARENT_CATEGORY_ID    NUMBER

6 rows selected.
```

Note: Remove the WHERE clause for a full listing.

You can query the DBA_LOGSTDBY_UNSUPPORTED data dictionary view to see all tables that contain data types that are not supported by logical standby databases. These tables are not maintained (that is, they do not have DML applied) in the logical standby database. Changes made to unsupported data types, tables, sequences, or views on the primary database are not propagated to the logical standby database, and no error message is returned.

It is a good idea to query this view on the primary database to ensure that those tables that are necessary for critical applications are not in this list before you create the logical standby database. If the primary database includes unsupported tables that are critical, consider using a physical standby database instead.

Note: This view does not show any tables from the SYS schema because changes to the SYS schema object are not applied to the logical standby database.

SQL Commands That Do Not Execute on the Standby Database

- ALTER DATABASE
- ALTER MATERIALIZED VIEW
- ALTER MATERIALIZED VIEW LOG
- ALTER SESSION
- ALTER SYSTEM
- CREATE CONTROL FILE
- CREATE DATABASE
- CREATE DATABASE LINK
- CREATE PFILE FROM SPFILE
- CREATE MATERIALIZED VIEW
- CREATE MATERIALIZED VIEW LOG
- CREATE SCHEMA AUTHORIZATION
- CREATE SPFILE FROM PFILE
- DROP DATABASE LINK
- DROP MATERIALIZED VIEW
- DROP MATERIALIZED VIEW LOG
- EXPLAIN
- LOCK TABLE
- SET CONSTRAINTS
- SET ROLE
- SET TRANSACTION

ORACLE

Not all data SQL commands that are executed on the primary database are applied to the logical standby database. If you execute any of the commands (shown in the slide) on the primary database, they are not executed on any logical standby database in your configuration. You must execute them on the logical standby database if needed to maintain consistency between the primary database and the logical standby database.

Unsupported PL/SQL-Supplied Packages

Unsupported PL/SQL-supplied packages include:

- DBMS_JAVA
- DBMS_REGISTRY
- DBMS_ALERT
- DBMS_SPACE_ADMIN
- DBMS_REFRESH
- DBMS_REDEFINITION
- DBMS_AQ

Packages supported only in the context of a rolling upgrade:

```
SQL> SELECT OWNER, PKG_NAME FROM DBA_LOGSTDBY_PLSQL_SUPPORT
WHERE SUPPORT_LEVEL = 'DBMS_ROLLING';
```

ORACLE

Oracle PL/SQL-supplied packages that do not modify system metadata or user data leave no footprint in the archived redo log files, and therefore are safe to use on the primary database. Examples of such packages are DBMS_OUTPUT, DBMS_RANDOM, DBMS_PIPE, DBMS_DESCRIBE, DBMS_TRACE, DBMS_METADATA, and DBMS_CRYPTO.

Oracle PL/SQL-supplied packages that do not modify system metadata but may modify user data are supported by SQL Apply, as long as the modified data belongs to the supported data types. Examples of such packages are DBMS_LOB, DBMS_SQL, and DBMS_TRANSACTION.

Oracle Data Guard logical standby supports replication of actions performed through the following packages: DBMS_DDL, DBMS_FGA, SDO_META, DBMS_REDACT, DBMS_REDEFINITION, DBMS_RLS, DBMS_SQL_TRANSLATOR, DBMS_XDS, DBMS_XMLINDEX, and DBMS_XMLSCHEMA.

Oracle PL/SQL-supplied packages that modify system metadata typically are not supported by SQL Apply, and therefore their effects are not visible on the logical standby database. Examples of such packages are DBMS_JAVA, DBMS_REGISTRY, DBMS_ALERT, DBMS_SPACE_ADMIN, DBMS_REFRESH, and DBMS_AQ.

Note: See *Oracle Data Guard Concepts and Administration* for additional information about support for PL/SQL packages.

Ensuring Unique Row Identifiers

- Query DBA_LOGSTDBY_NOT_UNIQUE on the primary database to find tables without a unique identifier:

```
SQL> SELECT * FROM dba_logstdby_not_unique;
OWNER          TABLE_NAME                     BAD_COLUMN
------         ---------------------------    ----------
APEX_040200    EMP_HIST                       Y
SCOTT          BONUS                          N
SCOTT          SALGRADE                       N
SH             SUPPLEMENTARY_DEMOGRAPHICS     N
SH             COSTS                          N
SH             SALES                          N
```

- BAD_COLUMN indicates a table column is defined using an unbounded data type such as LONG or BLOB.
- Add a primary key or unique index to ensure that SQL Apply can efficiently apply data updates.

Because the row IDs on a logical standby database might not be the same as the row IDs on the primary database, a different mechanism must be used to match the updated row on the primary database to its corresponding row on the logical standby database. Primary keys and non-null unique indexes can be used to match the corresponding rows. It is recommended that you add a primary key or a unique index to tables on the primary database (whenever appropriate and possible) to ensure that SQL Apply can efficiently apply data updates to the logical standby database.

You can query the DBA_LOGSTDBY_NOT_UNIQUE view to identify tables in the primary database that do not have a primary key or unique index with NOT NULL columns. Issue the following query to display a list of tables that SQL Apply might not be able to uniquely identify:

```
SQL> SELECT OWNER, TABLE_NAME,BAD_COLUMN
  2  FROM DBA_LOGSTDBY_NOT_UNIQUE
  3  WHERE TABLE_NAME NOT IN
  4  (SELECT TABLE_NAME FROM DBA_LOGSTDBY_UNSUPPORTED);
```

The key column in this view is BAD_COLUMN. If the view returns a row for a given table, you may want to consider adding a primary or unique key constraint on the table.

A value of Y indicates that the table does not have a primary or unique constraint and that the column is defined using an unbounded data type (such as CLOB). If two rows in the table match except for values in their LOB columns, the table cannot be maintained properly and SQL Apply stops.

A value of N indicates that the table does not have a primary or unique constraint, but that it contains enough column information to maintain the table in the logical standby database. However, the redo transport services and log apply services run more efficiently if you add a primary key. You should consider adding a disabled RELY constraint to these tables (as described in the next slide).

Adding a Disabled Primary Key RELY Constraint

You can add a disabled RELY constraint to uniquely identify rows:

```
SQL> ALTER TABLE hr.employees
  2  ADD PRIMARY KEY (employee_id, last_name)
  3  RELY DISABLE;
```


If your application ensures that the rows in a table are unique, you can create a disabled primary key RELY constraint on the table without incurring the overhead of maintaining a primary key on the primary database.

The RELY constraint tells the system to log the named columns (in this example, EMPLOYEE_ID and LAST_NAME) to identify rows in this table. Be careful to select columns for the disabled RELY constraint that uniquely identify the row. If the columns selected for the RELY constraint do not uniquely identify the row, SQL Apply does not apply redo information to the logical standby database.

To improve the performance of SQL Apply, add a unique constraint/index to the columns to identify the row on the logical standby database. Failure to do so results in full table scans during UPDATE or DELETE statements carried out on the table by SQL Apply.

Note: For this example, assume that the HR.EMPLOYEES table does not have a primary key.

Creating a Logical Standby Database by Using SQL Commands

To create a logical standby database by using SQL commands:

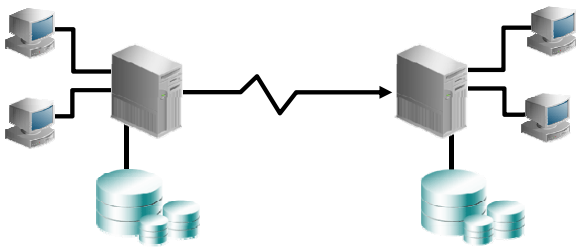
1. Create a physical standby database.
2. Stop Redo Apply on the physical standby database.
3. Prepare the primary database to support a logical standby database.
4. Build a LogMiner dictionary in the redo data.
5. Transition to a logical standby database.
6. Open the logical standby database.
7. Verify that the logical standby database is performing properly.

The next slides cover each of these steps in detail.

Note: Some steps are performed on the primary database, and some steps are performed on the standby database.

Step 1: Create a Physical Standby Database

a. Create a physical standby database.
b. Ensure that the physical standby database is current with the primary database.



You create a logical standby database by first creating a physical standby database. Then you convert the physical standby database into a logical standby database.

To create the physical standby database:

a. Create a physical standby database as described in the lesson titled “Creating a Physical Standby Database by Using SQL and RMAN Commands.”
b. Ensure that the physical standby database is current with the primary database by allowing recovery to continue until the physical standby database is consistent with the primary database, including all database structural changes (such as adding or dropping data files).

Step 2: Stop Redo Apply on the Physical Standby Database

- Before converting the physical standby database to a logical standby database, stop Redo Apply.
- Stopping Redo Apply is required to avoid applying changes past the redo that contains the LogMiner dictionary.

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY
DATABASE CANCEL;
```


To stop Redo Apply, issue one of the following commands:

- **If the configuration is not managed by the broker**
  `SQL > ALTER DATABASE RECOVER MANAGED STANDBY DATABASE CANCEL;`
- **If the configuration is managed by the broker**
  `DGMGRL > EDIT DATABASE '<database name>' set state='apply-off';`

The Data Guard broker will be discussed later in the course.

Step 3: Prepare the Primary Database to Support Role Transitions

Set the LOG_ARCHIVE_DEST_*n* initialization parameter for transitioning to a logical standby role.

```
LOG_ARCHIVE_DEST_1=
 'LOCATION=/arch1/boston/
  VALID_FOR=(ONLINE_LOGFILES,ALL_ROLES)
  DB_UNIQUE_NAME=boston'
LOG_ARCHIVE_DEST_3=
 'LOCATION=/arch2/boston/
  VALID_FOR=(STANDBY_LOGFILES,STANDBY_ROLE)
  DB_UNIQUE_NAME=boston'
LOG_ARCHIVE_DEST_STATE_3=ENABLE
```

Note: This step is necessary only if you plan to perform switchovers.

If you plan to transition the primary database to the logical standby role, you must also modify the parameters so that no parameters need to change after a role transition:

- Change the VALID_FOR attribute in the original LOG_ARCHIVE_DEST_1 destination to archive redo data only from the online redo log and not from the standby redo log.
- Include the LOG_ARCHIVE_DEST_3 destination on the primary database. This parameter only takes effect when the primary database is transitioned to the logical standby role.

When the boston database is running in the primary role, LOG_ARCHIVE_DEST_1 directs archiving of redo data generated by the primary database from the local online redo log files to the local archived redo log files in /arch1/boston and LOG_ARCHIVE_DEST_3 is not used.

When the boston database is running in the logical standby database role, LOG_ARCHIVE_DEST_1 directs archiving of redo data generated by the logical standby database from the local online redo log files to the local archived redo log files in /arch1/boston/.

LOG_ARCHIVE_DEST_3 directs archiving of redo data from the standby redo log files to the local archived redo log files in /arch2/boston/.

Step 4: Build a LogMiner Dictionary in the Redo Data

- Build a LogMiner dictionary in the redo data so that SQL Apply can properly interpret changes in the redo.
- Supplemental logging is automatically enabled.
- Execute the procedure on the primary database:

```
SQL> EXECUTE DBMS_LOGSTDBY.BUILD;
```


SQL Apply requires a LogMiner dictionary in the redo data so that it can properly interpret changes in the redo. When you execute the `DBMS_LOGSTDBY.BUILD` procedure, the LogMiner dictionary is built and supplemental logging is automatically enabled for logging primary key and unique key columns. Supplemental logging ensures that each update contains enough information to logically identify the affected row.

Note: The `DBMS_LOGSTDBY.BUILD` procedure waits for all existing transactions to complete so that long-running transactions executing on the primary database will affect its operation.

Step 5: Transition to a Logical Standby Database

a. Execute the following command on the standby database to convert it to a logical standby database:

```
SQL> ALTER DATABASE RECOVER TO LOGICAL STANDBY
db_name;
```

b. Shut down the logical standby database instance and restart it in `MOUNT` mode.

c. Adjust initialization parameters: `LOG_ARCHIVE_DEST_n`

ORACLE

To prepare the physical standby database to transition to a logical standby database:

a. Issue the `ALTER DATABASE RECOVER TO LOGICAL STANDBY db_name` command to continue applying redo data to the physical standby database until it is ready to convert to a logical standby database. Specify a database name (`db_name`) to identify the new logical standby database.
   The redo log files contain the information needed to convert your physical standby database to a logical standby database. The statement applies redo data until the LogMiner dictionary is found in the redo log files.

b. Shut down the logical standby database instance and restart it in `MOUNT` mode.

c. Modify the `LOG_ARCHIVE_DEST_n` parameters to specify separate local destinations for:
   - Archived redo log files that store redo data generated by the logical standby database
   - Archived redo log files that store redo data received from the primary database

Step 6: Open the Logical Standby Database

a. Open the new logical standby database with the `RESETLOGS` option:

```
SQL> ALTER DATABASE OPEN RESETLOGS;
```

b. Start the application of redo data to the logical standby database:

```
SQL> ALTER DATABASE START LOGICAL STANDBY APPLY
IMMEDIATE;
```

ORACLE

To open the logical standby database and start SQL Apply:

a. On the logical standby database, issue the `ALTER DATABASE OPEN RESETLOGS` command to open the database.

b. On the logical standby database, issue the following command to start SQL Apply:

```
ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE
```

Step 7: Verify That the Logical Standby Database Is Performing Properly

a. Verify that the archived redo log files were registered:

```
SQL> SELECT sequence#, first_time, next_time,
dict_begin, dict_end FROM dba_logstdby_log ORDER BY
sequence#;
```

b. Begin sending redo data to the standby database:

```
SQL> ALTER SYSTEM ARCHIVE LOG CURRENT;
```

c. Query the DBA_LOGSTDBY_LOG view to verify that the archived redo log files were registered.

After creating your logical standby database and setting up redo transport services and log apply services, you should verify that redo data is being transmitted from the primary database and applied to the standby database.

To verify that the logical standby database is functioning properly:

a. Connect to the logical standby database and query the DBA_LOGSTDBY_LOG view to verify that the archived redo log files were registered on the logical standby system:

```
SQL> SELECT SEQUENCE#, FIRST_TIME, NEXT_TIME, DICT_BEGIN,DICT_END
FROM DBA_LOGSTDBY_LOG ORDER BY SEQUENCE#;
```

b. Connect to the primary database and issue the following command to begin sending redo data to the standby database:

```
SQL> ALTER SYSTEM ARCHIVE LOG CURRENT;
```

c. Connect to the logical standby database and requery the DBA_LOGSTDBY_LOG view as shown in step a. This enables you to verify that the new archived redo log files were registered.

Step 7: Verify That the Logical Standby Database Is Performing Properly

d. Verify that redo data is being applied correctly:

```
SQL> SELECT name, value FROM v$logstdby_stats
WHERE name = 'coordinator state';
```

e. Query the V$LOGSTDBY_PROCESS view to see current SQL Apply activity:

```
SQL> SELECT sid, serial#, spid, type, high_scn
FROM v$logstdby_process;
```

f. Check the overall progress of SQL Apply:

```
SQL> SELECT applied_scn, latest_scn FROM
v$logstdby_progress;
```

ORACLE

d. On the logical standby database, query the V$LOGSTDBY_STATS view to verify that redo data is being applied correctly:

```
SQL> SELECT NAME, VALUE FROM V$LOGSTDBY_STATS WHERE NAME =
'coordinator state';
```

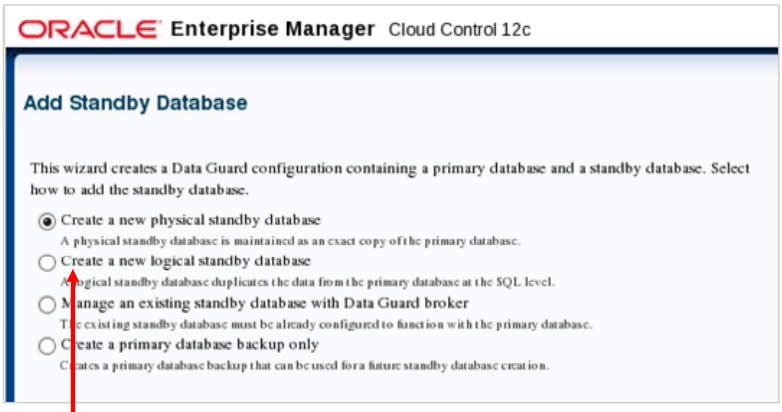
A value of INITIALIZING in the VALUE column indicates that log apply services are preparing to begin SQL Apply, but that data from the archived redo log files is not being applied to the logical standby database.

e. Query the V$LOGSTDBY_PROCESS view to see information about the current state of the SQL Apply processes.

f. Query the V$LOGSTDBY_PROGRESS view on the logical standby database to check the overall progress of SQL Apply:

```
SQL> SELECT APPLIED_SCN, LATEST_SCN FROM V$LOGSTDBY_PROGRESS;
```

Creating a Logical Standby Database by Using Enterprise Manager



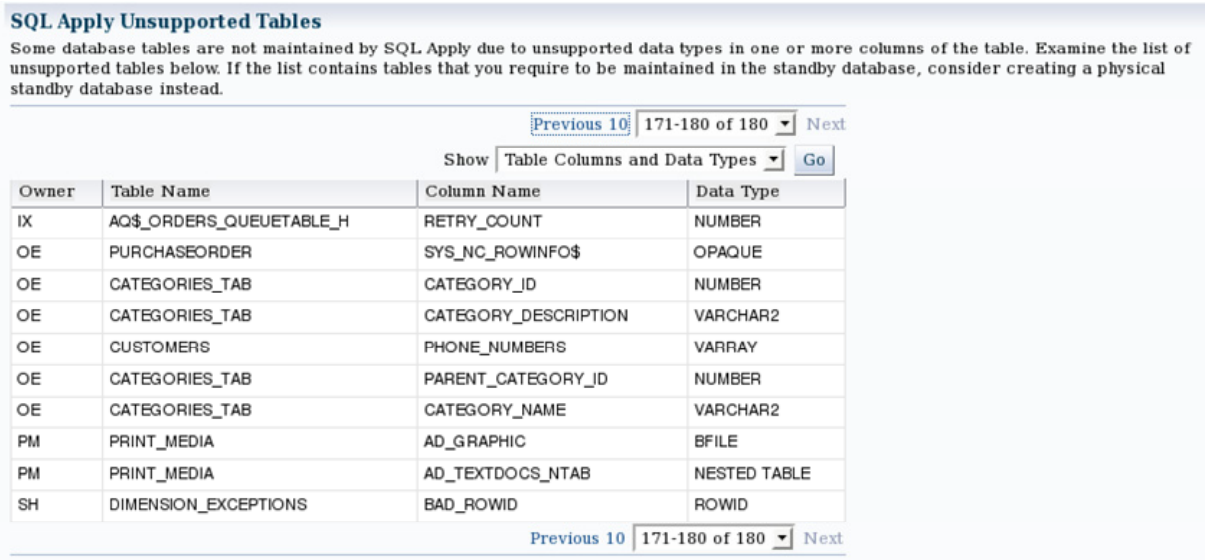
Select “Create a new logical standby database.”

To create a logical standby database by using Enterprise Manager:

1. Click Add Standby Database on the Data Guard overview page to invoke the Add Standby Database Wizard.

   Note: If the logical standby database is the first database to be created in your configuration, access the Add Standby Database Wizard by clicking “Setup and Manage” in the Data Guard section on the Availability page. Next, click the Add Standby Database link to invoke the Add Standby Database Wizard.
2. Select “Create a new logical standby database” on the Add Standby Database page, and click Continue.
3. The wizard guides you through a procedure that is similar to adding a physical standby database.

Using the Add Standby Database Wizard



SQL Apply Unsupported Tables

Some database tables are not maintained by SQL Apply due to unsupported data types in one or more columns of the table. Examine the list of unsupported tables below. If the list contains tables that you require to be maintained in the standby database, consider creating a physical standby database instead.

Previous 10 | 171-180 of 180 | Next

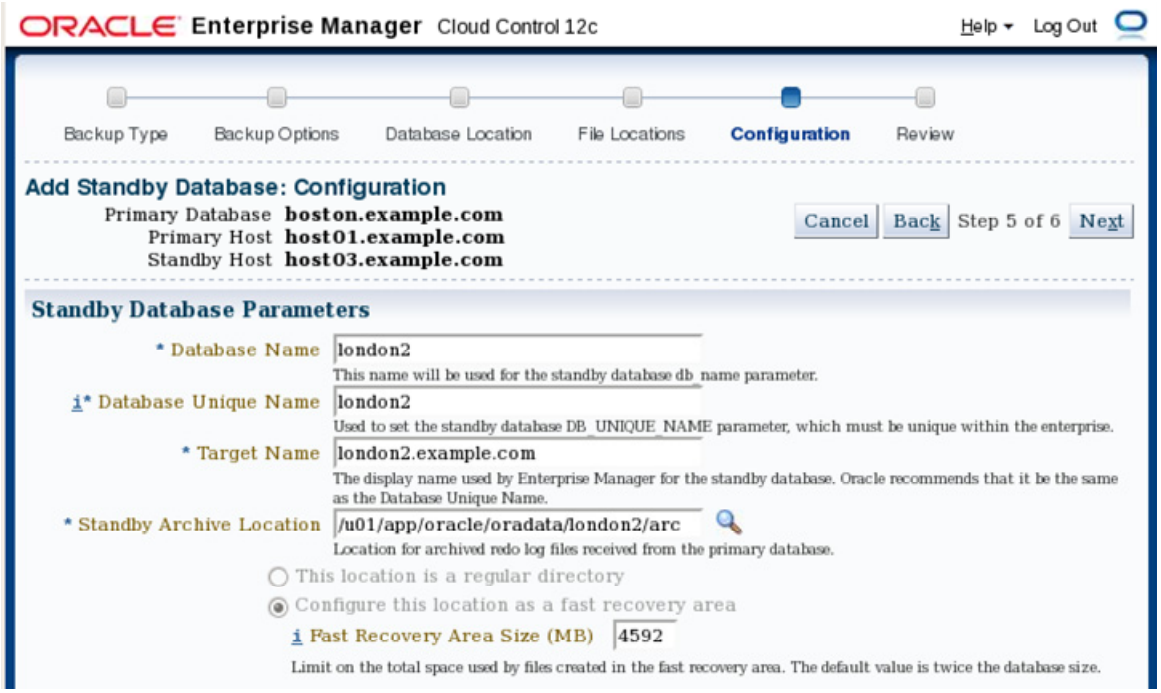
Show Table Columns and Data Types | Go

Owner	Table Name	Column Name	Data Type
IX	AQ$_ORDERS_QUEUETABLE_H	RETRY_COUNT	NUMBER
OE	PURCHASEORDER	SYS_NC_ROWINFO$	OPAQUE
OE	CATEGORIES_TAB	CATEGORY_ID	NUMBER
OE	CATEGORIES_TAB	CATEGORY_DESCRIPTION	VARCHAR2
OE	CUSTOMERS	PHONE_NUMBERS	VARRAY
OE	CATEGORIES_TAB	PARENT_CATEGORY_ID	NUMBER
OE	CATEGORIES_TAB	CATEGORY_NAME	VARCHAR2
PM	PRINT_MEDIA	AD_GRAPHIC	BFILE
PM	PRINT_MEDIA	AD_TEXTDOCS_NTAB	NESTED TABLE
SH	DIMENSION_EXCEPTIONS	BAD_ROWID	ROWID

Previous 10 | 171-180 of 180 | Next

On the Add Standby Database: Backup Type page, you determine whether you have tables or columns that are not supported by SQL Apply. In the SQL Apply Unsupported Tables section, select “Table Columns and Data Types” in the drop-down list and click Go to see the unsupported tables.

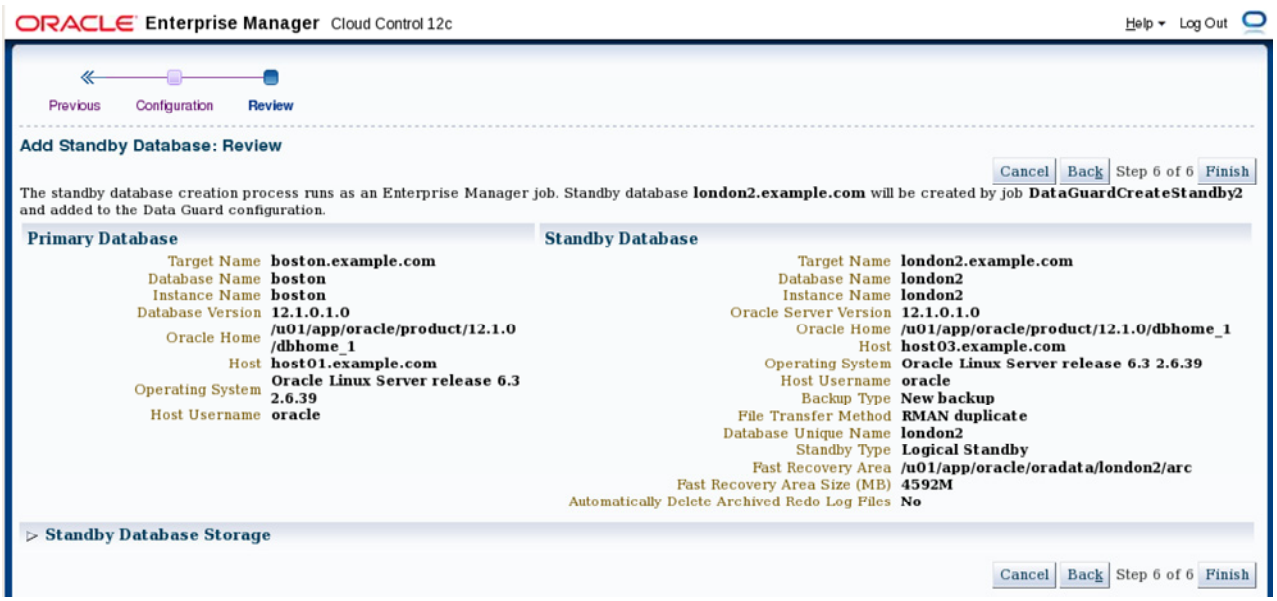
Using the Add Standby Database Wizard



On the Add Standby Database: Configuration page, specify the values for the `DB_NAME` and `DB_UNIQUE_NAME` parameters for your logical standby database. In addition, specify the target name to be used by Enterprise Manager Cloud Control.

In the Standby Database Monitoring Credentials section, you can choose between a non-SYSDBA user such as `DBSNMP` to monitor the database or choose to use the SYSDBA monitoring credentials.

Using the Add Standby Database Wizard



After completing all pages in the wizard, you see the Add Standby Database Review page. Review the information on this page and click **Finish** to submit a job to create your logical standby database.

Securing Your Logical Standby Database

- Configure the database guard to control user access to tables.
- `ALTER DATABASE GUARD` command keywords:
  - `ALL`: Prevents users from making changes to any data in the database
  - `STANDBY`: Prevents users from making changes to any data maintained by Data Guard SQL Apply
  - `NONE`: Normal security
- Query the `GUARD_STATUS` column in `V$DATABASE`.
- The database guard level is automatically set to `ALL` by the broker on the logical standby database.
- The database guard level applies to all users except `SYS`.

The *database guard* controls user access to tables in a logical standby database. Use the `ALTER DATABASE GUARD` command to configure the database guard.

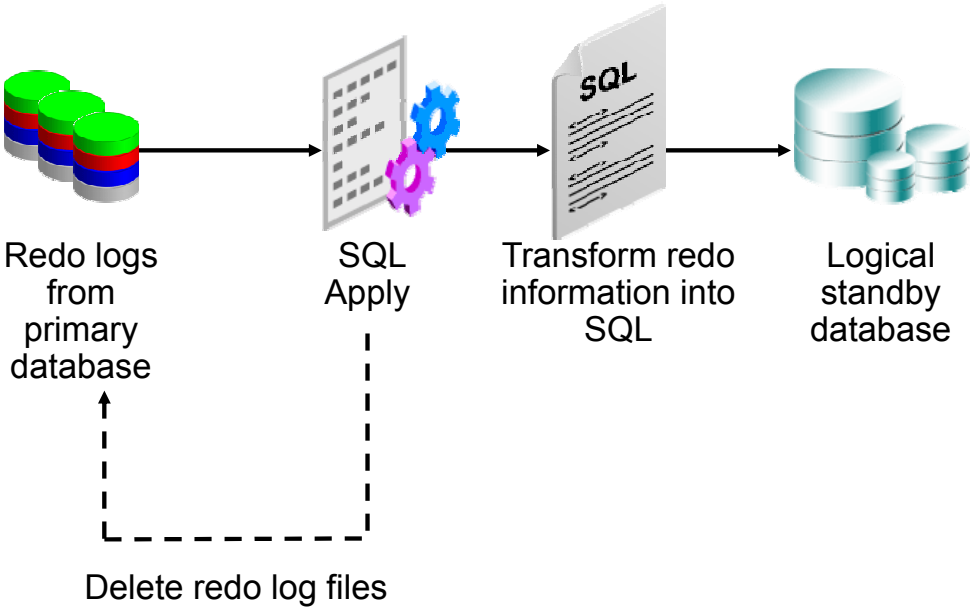
By default, it is not possible for a nonprivileged user to modify data on a Data Guard SQL Apply database, because the database guard is automatically set to `ALL`. With this level of security, only the `SYS` user can modify data.

When you set the security level to `STANDBY`, users are able to modify data that is not maintained by the logical apply engine. A security level of `NONE` permits any users to access the standby database if they have the appropriate privileges.

When creating a logical standby database manually with SQL commands, you must issue the `ALTER DATABASE GUARD ALL` command before opening the database. Failure to do so allows jobs that are submitted through `DBMS_JOB.SUBMIT` to be scheduled and to potentially modify tables in the logical standby database.

Note: Be careful not to let the primary and logical standby databases diverge while the database guard is disabled.

Automatic Deletion of Redo Log Files by SQL Apply



Archived redo logs on the logical standby database are automatically deleted by SQL Apply after they are applied. This feature reduces the amount of space consumed on the logical standby database and eliminates the manual step of deleting the archived redo log files.

You can enable and disable the auto-delete feature for archived redo logs by using the `DBMS_LOGSTDBY.APPLY_SET` procedure. By default, the auto-delete feature is enabled.

Managing Remote Archived Log File Retention

The LOG_AUTO_DEL_RETENTION_TARGET parameter:

- Is used to specify the number of minutes that SQL Apply keeps a remote archived log after completely applying its contents
- Is applicable only if LOG_AUTO_DELETE is set to TRUE and the flash recovery area is not being used to store remote archived logs
- Has a default value of 1,440 minutes

```
SQL> execute DBMS_LOGSTDBY.APPLY_SET
   ('LOG_AUTO_DEL_RETENTION_TARGET','2880');
```


By default, SQL Apply deletes an archived redo log file after applying the contents. This behavior is controlled by the LOG_AUTO_DELETE parameter. During a flashback operation or point-in-time recovery of the logical standby database, SQL Apply must stop and re-fetch all remote archived redo log files.

In Oracle Database 12*c*, you use the LOG_AUTO_DEL_RETENTION_TARGET parameter to specify a retention period for remote archived redo log files. You can modify LOG_AUTO_DEL_RETENTION_TARGET by using the DBMS_LOGSTDBY.APPLY_SET and DBMS_LOGSTDBY.APPLY_UNSET procedures.

Note: This parameter is applicable only when you are not using the fast recovery area.

Creating SQL Apply Filtering Rules

Procedures exist to create SQL Apply filtering rules.

- To skip SQL statements:

```
SQL> EXECUTE DBMS_LOGSTDBY.SKIP(STMT => 'DML',
schema_name => 'HR', object_name => '%');
```

- To skip errors and continue applying changes:

```
SQL> EXECUTE DBMS_LOGSTDBY.SKIP_ERROR('GRANT');
```

- To skip applying transactions:

```
SQL> EXECUTE DBMS_LOGSTDBY.SKIP_TRANSACTION
(XIDUSN => 1, XIDSLT => 13, XIDSQN => 1726);
```

- To skip all SQL statements for a container:

```
SQL> EXECUTE DBMS_LOGSTDBY.SKIP(stmt =>
'CONTAINER', object_name => 'DEV1');
```

ORACLE

You can use SQL Apply filters to control what to apply and what not to apply. To define a SQL Apply filter, use the `DBMS_LOGSTDBY` package with the `SKIP`, `SKIP_ERROR`, and `SKIP_TRANSACTION` procedures. The first example in the slide uses the SKIP procedure to ignore all DML changes made to the HR schema. The second example in the slide uses the `SKIP_ERROR` procedure to ignore any error raised from any `GRANT DDL` command and continue applying changes instead of stopping SQL Apply. The third example in the slide uses the `SKIP_TRANSACTION` procedure and skips a DDL transaction with `XIDUSN=1`, `XIDSLT=13`, and `XIDSQN=1726`. The following describes the fields used:

- **XIDUSN NUMBER:** Transaction ID undo segment number of the transaction being skipped
- **XIDSLT NUMBER:** Transaction ID slot number of the transaction being skipped
- **XIDSQN NUMBER:** Transaction ID sequence number of the transaction being skipped

See the *Oracle Database PL/SQL Packages and Types Reference* for detailed information about the `DBMS_LOGSTDBY` package and its procedures.

Deleting SQL Apply Filtering Rules

Procedures exist to delete SQL Apply filtering rules.

- To delete SQL statement rules:

```
SQL> EXECUTE DBMS_LOGSTDBY.UNSKIP(STMT => 'DML',
schema_name => 'HR', object_name => '%');
```

- To delete error apply filter rules:

```
SQL> EXECUTE DBMS_LOGSTDBY.UNSKIP_ERROR('GRANT');
```

- To delete transaction filter rules:

```
SQL> EXECUTE DBMS_LOGSTDBY.UNSKIP_TRANSACTION
(XIDUSN => 1, XIDSLT => 13, XIDSQN => 1726);
```

ORACLE

To delete a SQL Apply filter, use the `DBMS_LOGSTANDBY` package with the `UNSKIP`, `UNSKIP_ERROR`, and `UNSKIP_TRANSCATION` procedures. The syntax to delete a SQL Apply filter is almost identical to the syntax to create a SQL Apply filter.

See the *Oracle Database PL/SQL Packages and Types Reference* for detailed information about the `DBMS_LOGSTDBY` package and its procedures.

Viewing SQL Apply Filtering Settings

Query DBA_LOGSTDBY_SKIP to view SQL Apply filtering settings:

```
SQL> SELECT error, statement_opt, name
  2  FROM dba_logstdby_skip
  3  WHERE owner='HR';
ERROR      STATEMENT_OPT        NAME
---------- -------------------- ----------
N          DML                  JOBS
```


You can query the DBA_LOGSTDBY_SKIP view on the logical standby database to determine the SQL Apply filtering settings. The view contains the following columns:

- **ERROR:** Indicates whether the statement should be skipped or should return an error for the statement
- **STATEMENT_OPT:** Specifies the type of statement that should be skipped (It must be one of the SYSTEM_AUDIT statement options.)
- **OWNER:** Name of the schema under which the skip option is used
- **NAME:** Name of the table that is being skipped
- **USE_LIKE:** Indicates whether the statement uses a SQL wildcard search when matching names
- **ESC:** Escape character to be used when performing wildcard matches
- **PROC:** Name of a stored procedure that is executed when processing the skip option

You can query DBA_LOGSTDBY_SKIP_TRANSACTION to view settings for transactions to be skipped.

Using DBMS_SCHEDULER to Create Jobs on a Logical Standby Database

- Scheduler jobs can be created on a standby database.
- When a Scheduler job is created, it defaults to the local role.
- Activate existing jobs by using the DATABASE_ROLE attribute of DBMS_SCHEDULER.SET_ATTRIBUTE, which has the following settings:
  - PRIMARY: The job runs only when the database is in the role of the primary database.
  - LOGICAL STANDBY: The job runs only when the database is in the role of a logical standby.

Use the DBMS_SCHEDULER package to create Scheduler jobs so that they are executed when intended. Scheduler jobs can be created on a standby database.

When a Scheduler job is created, it defaults to the local role. If the job is created on the standby database, it defaults to the role of a logical standby. The Scheduler executes jobs that are specific to the current database role. Following a role transition (switchover or failover), the Scheduler automatically switches to executing jobs that are specific to a new role.

Scheduler jobs are replicated to the standby, but they do not run by default. However, existing jobs can be activated under the new role by using the DBMS_SCHEDULER.SET_ATTRIBUTE procedure. If you want a job to run for all database roles on a particular host, you must create two copies of the job on that host: one with a database_role of PRIMARY and the other with a database_role of LOGICAL STANDBY. Query the DBA_SCHEDULER_JOB_ROLES view to determine which jobs are specific to which role.

See the *Oracle Database PL/SQL Packages and Types Reference* for detailed information about using the DBMS_SCHEDULER package.

Quiz

By default, users are able to create additional indexes and materialized views in a logical standby database.

a. True
b. False

Answer: b

Quiz

Scheduler jobs created on the primary database are replicated to the logical standby database, but do not execute by default.

a. True
b. False

Answer: a

Summary

In this lesson, you should have learned how to:

- Determine when to create a logical standby database
- Create a logical standby database
- Manage SQL Apply filtering

Practice 7: Overview

This practice covers using Enterprise Manager to create a logical standby database.

8

Oracle Data Guard Broker: Overview

Objectives

After completing this lesson, you should be able to describe:

- The Data Guard broker architecture
- Data Guard broker components
- Benefits of the Data Guard broker
- Data Guard broker configurations
- How to use Enterprise Manager to manage your Data Guard configuration
- How to invoke DGMGRL to manage your Data Guard configuration

ORACLE

Oracle Data Guard Broker: Features

- The Oracle Data Guard broker is a distributed management framework.
- The broker automates and centralizes the creation, maintenance, and monitoring of Data Guard configurations.
- With the broker, you can perform all management operations locally or remotely with easy-to-use interfaces:
  - Oracle Enterprise Manager Cloud Control
  - DGMGRL (a command-line interface)

ORACLE

The following are some of the operations that the broker automates and simplifies:

- Automated creation of Data Guard configurations incorporating a primary database, a new or existing standby database, redo transport services, and log apply services

  Note: Any of the databases in the configuration can be a Real Application Clusters (RAC) database.
- Adding new or existing standby databases to each existing Data Guard configuration, for a total of one primary database and from one to 30 standby databases in the same configuration
- Managing an entire Data Guard configuration (including all databases, redo transport services, and log apply services) through a client connection to any database in the configuration
- Invoking switchover or failover with a single command to initiate and control complex role changes across all databases in the configuration
- Monitoring the status of the entire configuration, capturing diagnostic information, reporting statistics (such as the log apply rate and the redo generation rate), and detecting problems quickly with centralized monitoring, testing, and performance tools

Data Guard Broker: Components

- Client-side:
  - Oracle Enterprise Manager Cloud Control
  - DGMGRL (command-line interface)
- Server-side: Data Guard monitor
  - DMON process
  - Configuration files

The Oracle Data Guard broker consists of both client-side and server-side components.

On the client, you can use the following Data Guard components to define and manage a configuration:

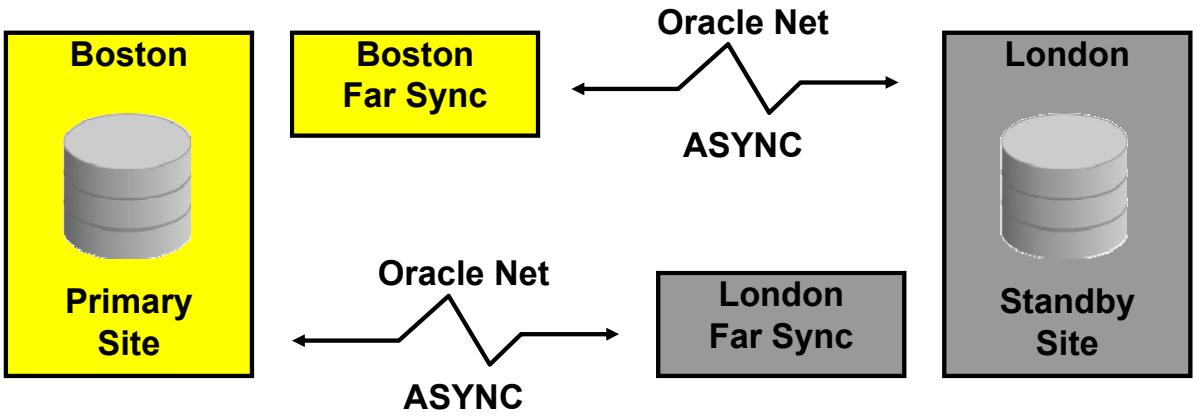
- Oracle Enterprise Manager Cloud Control
- DGMGRL, which is the Data Guard command-line interface (CLI)

On the server, the Data Guard monitor is a broker component that is integrated with the Oracle database. The Data Guard monitor comprises the Data Guard monitor (DMON) process and broker configuration files, with which you can control the databases of that configuration, modify their behavior at run time, monitor the overall health of the configuration, and provide notification of other operational characteristics.

The configuration file contains profiles that describe the current state and properties of each database in the configuration. Associated with each database are various properties that the DMON process uses to control the database's behavior. The properties are recorded in the configuration file as a part of the database's object profile that is stored there. Many database properties are used to control database initialization parameters related to the Data Guard environment.

Data Guard Broker: Configurations

The most common configuration is a primary database at one location and a standby database at another location.

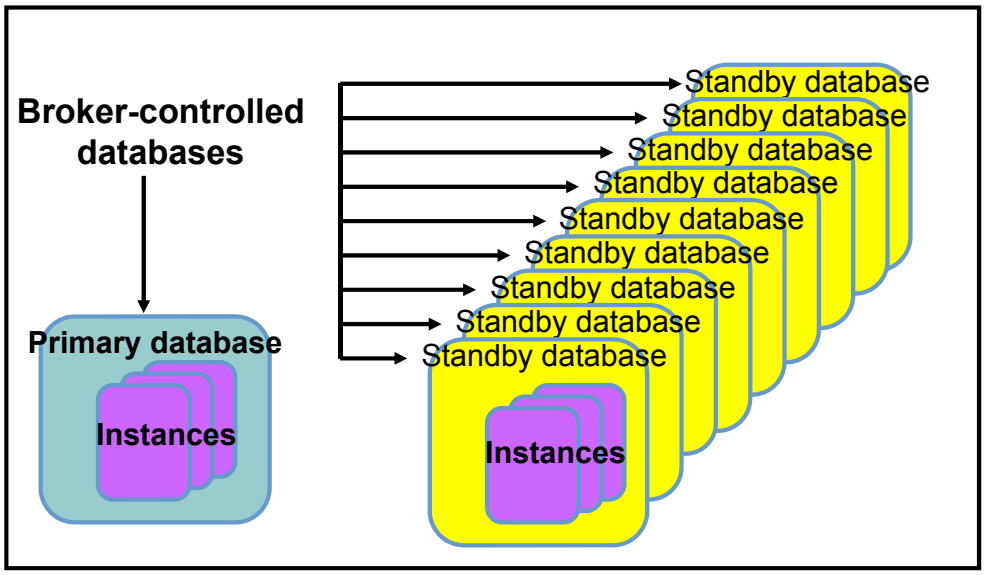


A Data Guard configuration consists of one primary database and up to 30 standby databases or Far Sync locations. The databases in a Data Guard configuration are typically dispersed geographically and are connected by Oracle Net.

A Data Guard broker configuration is a logical grouping of the primary, Far Sync, and standby databases in a Data Guard configuration. The broker's `DMON` process configures and maintains the broker configuration components as a unified group of resource objects that you can manage and monitor as a single unit.

Data Guard Broker: Management Model

Data Guard Broker Configuration



ORACLE

The Data Guard broker performs operations on the following logical objects:

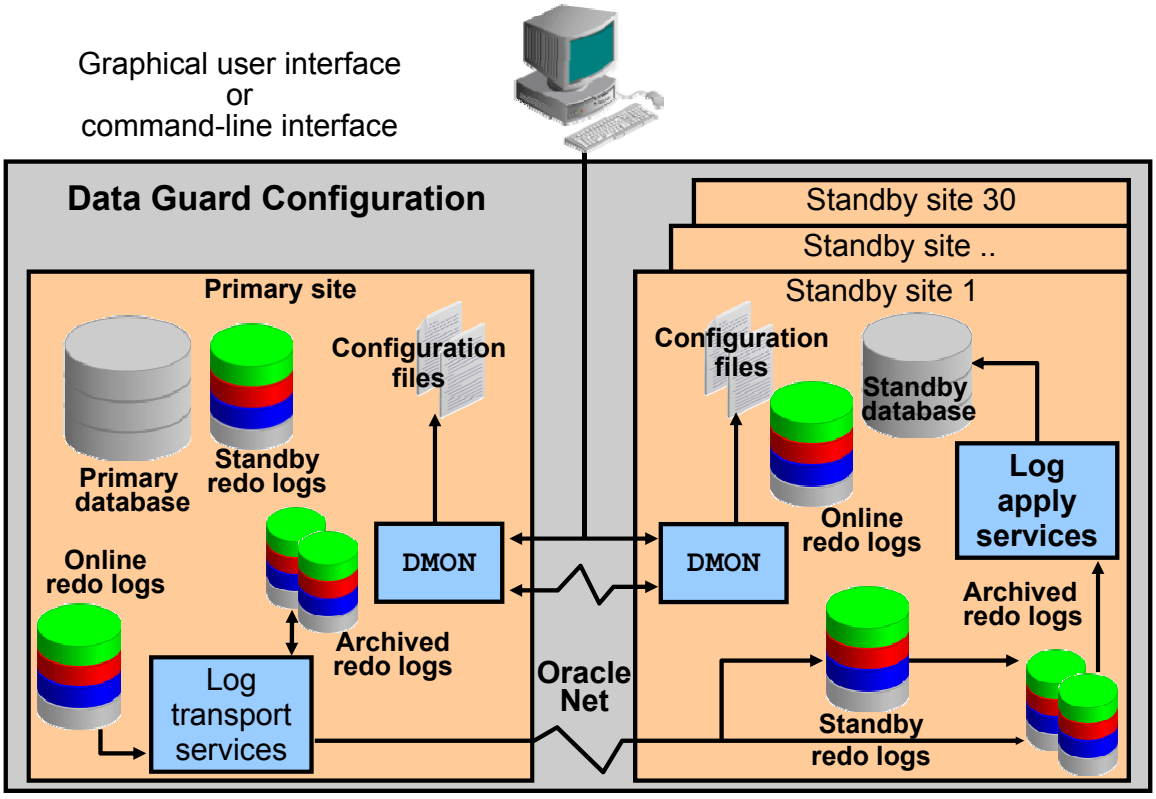
- Configuration of databases
- Single database

A broker configuration consists of:

- **Configuration object:** A named collection of database profiles. A database profile is a description of a database object, including its current state, current status, and properties.
- **Database objects:** Objects corresponding to primary or standby databases
- **Instance objects:** A database object may comprise one or more instance objects if it is a RAC database.

The broker supports one or more Data Guard configurations, each of which includes a profile for one primary database as well as profiles for up to 30 physical, logical, RAC or non-RAC standby databases. Far Sync destinations are also counted towards the limit of 30 locations.

Data Guard Broker: Architecture



The Data Guard broker helps you create, control, and monitor a Data Guard configuration. This configuration consists of a primary database that is protected by one or more standby databases. After the broker has created the Data Guard configuration, the broker monitors the activity, health, and availability of all systems in that configuration.

The Data Guard monitor process (DMON) is an Oracle background process that runs on every instance that is managed by the broker, including Far Sync instances. When you start the Data Guard broker, a DMON process is created.

When you use Enterprise Manager or the Data Guard command-line interface (CLI), the DMON process is the server-side component that interacts with the local instance and the DMON processes that are running on other sites to perform the requested function. The DMON process is also responsible for monitoring the health of the broker configuration and for ensuring that every instance has a consistent copy of the configuration files in which the DMON process stores its configuration data. There are two multiplexed versions of the configuration file on each database.

Data Guard Monitor: DMON Process

- A server-side background process
- A part of each database instance in the configuration
- Created when you start the broker
- Performs requested functions and monitors the resource
- Communicates with other DMON processes in the configuration
- Updates the configuration file
- Creates the drc<SID> trace file in the location set by the DIAGNOSTIC_DEST initialization parameter
- Modifies initialization parameters during role transitions as necessary

The Data Guard monitor comprises two components: the DMON process and the configuration file.

The DMON process is an Oracle background process that is part of each database instance managed by the broker. When you start the Data Guard broker, a portion of the SGA is allocated and a DMON process is created. The amount of memory allocated is typically less than 50 KB per site; the actual amount on your system varies.

When you use Enterprise Manager or the CLI, the DMON process is the server-side component that interacts with the local instance and the DMON processes running on other sites to perform the requested function.

The DMON process is also responsible for monitoring the health of the broker configuration and for ensuring that every database has a consistent copy of the broker configuration files in which the DMON process stores its configuration data.

The DIAGNOSTIC_DEST parameter defines the location for diagnostic files such as trace files and core files. The structure of the directory specified by DIAGNOSTIC_DEST is as follows:

<diagnostic_dest>/diag/rdbms/*<dbname>*/*<instance_name>*/

This location is known as the Automatic Diagnostic Repository (ADR) Home. The DMON trace files are located in the <adr-home>/trace subdirectory.

Benefits of Using the Data Guard Broker

- Enhances the high-availability, data protection, and disaster protection capabilities inherent in Oracle Data Guard by automating both configuration and monitoring tasks
- Streamlines the process for any one of the standby databases to replace the primary database and take over production processing
- Enables easy configuration of additional standby databases
- Provides simplified, centralized, and extended management
- Automatically communicates between the databases in a Data Guard configuration by using Oracle Net Services
- Provides built-in validation that monitors the health of all databases in the configuration

ORACLE

By automating the tasks required to configure and monitor a Data Guard configuration, the broker enhances the high-availability, data protection, and disaster protection capabilities that are inherent in Oracle Data Guard. If the primary database fails, the broker streamlines the process for any one of the standby databases to replace the primary database and take over production processing.

The broker enables easy configuration of additional standby databases. After creating a Data Guard configuration consisting of two databases—a primary and standby—you can add up to 29 additional standby databases to the existing two for each Data Guard configuration.

Comparing Configuration Management With and Without the Data Guard Broker

	With the Broker	Without the Broker
General	Manage databases as one	Manage databases separately
Creation of the standby database	Use Enterprise Manager (EM) Cloud Control wizards	Manually create files
Configuration and management	Configure and manage from single interface	Set up services manually for each database
Monitoring	• Monitor continuously • Unified status and reports • Integrate with EM events	Monitor each database individually through views
Control	Invoke role transitions with a single command	Coordinate sequences of multiple commands across database sites for role transitions

ORACLE

The table in the slide provides an overview of configuration management with and without the Data Guard broker (source: Table 1-1, “Configuration Management With and Without the Broker,” in *Oracle Data Guard Broker*).

Data Guard Broker Interfaces

- Command-line interface (CLI):
  - Is started by entering `DGMGRL` at the command prompt where the Oracle server or an Oracle client is installed
  - Enables you to control and monitor a Data Guard configuration from the prompt or in scripts
- Oracle Enterprise Manager Cloud Control:
  - Provides wizards to simplify creating and managing standby databases

ORACLE

The DGMGRL command-line interface includes:

- Configuration and setup tasks
- Management and control of the configuration
- Commands to check the status and health of the configuration
- Commands to execute role changes

Oracle Enterprise Manager (EM) Cloud Control includes the following Data Guard features:

- Wizard-driven creation of standby databases
- Wizard-driven creation of a broker configuration based on an existing databases
- Complete monitoring and proactive event reporting through email or pagers
- Simplified control of the databases through their potential states. For example, with Enterprise Manager you can start or stop the redo transport services, start or stop the log apply services, and place a standby database in read-only mode.
- “Pushbutton” switchover and failover. EM enables you to execute a switchover or failover between a primary and a standby database by simply clicking a button.

Broker Controlled Database Initialization Parameters

The broker controls the following parameters:

- LOG_ARCHIVE_DEST_n
- LOG_ARCHIVE_DEST_STATE_n
- ARCHIVE_LAG_TARGET
- DB_FILE_NAME_CONVERT
- LOG_ARCHIVE_FORMAT
- LOG_ARCHIVE_MAX_PROCESSES
- LOG_ARCHIVE_MIN_SUCCEED_DEST
- LOG_ARCHIVE_TRACE
- LOG_FILE_NAME_CONVERT
- STANDBY_FILE_MANAGEMENT
- MAX_EVENTS_RECORDED
- MAX_SERVERS
- MAX_SGA
- PRESERVE_COMMIT_ORDER
- RECORD_APPLIES_DDL
- RECORD_SKIP_DDL
- RECORD_SKIP_ERRORS

The Data Guard broker directly controls the database initialization parameters listed in the slide. After defining a Data Guard broker configuration, you should use DGMGRL or Enterprise Manager Cloud Control to manage your configuration. You should not use SQL commands to manage the databases, because you could cause a conflict with the broker.

The broker also uses configurable property settings to manage how redo apply and SQL apply are started. Therefore, the following SQL statements are managed automatically by the broker:

- ALTER DATABASE RECOVER MANAGED STANDBY DATABASE
- ALTER DATABASE START LOGICAL STANDBY APPLY IMMEDIATE

Using the Command-Line Interface of the Data Guard Broker

```
DGMGRL> connect sysdg/password
Connected.
DGMGRL> show configuration
Configuration - DRSolution
  Protection Mode: MaxPerformance
  Databases:
  boston   - Primary database
      bostonFS - Far Sync
          london   - Physical standby database
          london2  - Logical standby database
      londonFS - Far Sync (inactive)

Fast-Start Failover: DISABLED

Configuration Status:
SUCCESS
```


DGMGRL Commands

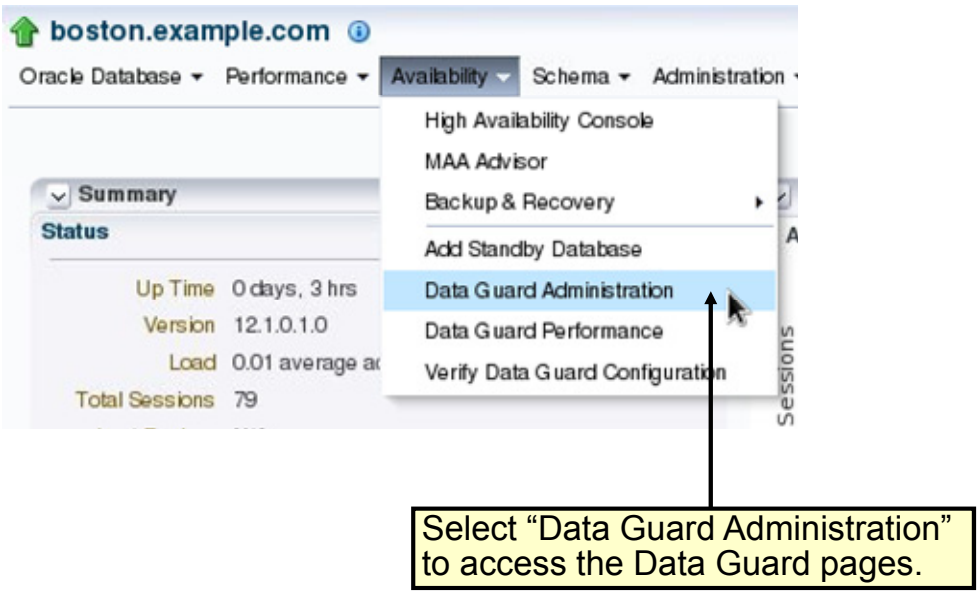
The following commands are available in DGMGRL (the Data Guard CLI). Many of these commands have additional arguments that are not described here. See *Oracle Data Guard Broker* for detailed information.

- `ADD DATABASE`: Adds a new standby database to the existing broker configuration
- `ADD FAR_SYNC`: Adds an existing Far Sync instance to an Oracle Data Guard broker configuration
- `CONNECT`: Connects to the specified database using the specified username
- `CONVERT DATABASE`: Converts the specified database to either a snapshot standby database or a physical standby database
- `CREATE CONFIGURATION`: Creates a broker configuration and adds a primary database to that configuration
- `DISABLE CONFIGURATION`: Disables broker management of a configuration so that the configuration and all of its databases are no longer managed by the broker
- `DISABLE DATABASE`: Disables broker management of the named standby database

- `DISABLE FAR_SYNC`: Disables broker management of a Far Sync instance
- `DISABLE FAST_START FAILOVER`: Disables fast-start failover
- `DISABLE FAST_START FAILOVER CONDITION`: Allows a user to remove conditions for which a fast-start failover should be performed
- `EDIT CONFIGURATION` (Property): Changes the value of a property for the broker configuration
- `EDIT CONFIGURATION` (Protection Mode): Changes the current protection mode setting for the broker configuration
- `EDIT CONFIGURATION` (RENAME): Changes the configuration name
- `EDIT CONFIGURATION RESET` (Property): Resets the specified configuration property to its default value
- `EDIT DATABASE` (Property): Changes the value of a property for the named database
- `EDIT DATABASE` (Rename): Changes the name used by the broker to refer to the specified database
- `EDIT DATABASE` (State): Changes the state of the specified database
- `EDIT DATABASE RESET` (Property): Resets the specified property for the named database to its default value
- `EDIT FAR_SYNC`: Changes the name, state, or properties of a Far Sync instance
- `EDIT FAR_SYNC RESET` (Property): Resets the specified property for the named Far Sync instance to its default value
- `EDIT INSTANCE` (AUTO PFILE): Sets the name of the initialization parameter file for the specified instance
- `EDIT INSTANCE` (Property): Changes the value of a property for the specified instance
- `EDIT INSTANCE RESET` (Property): Resets an instance-specific property for the specified instance(s) to its default value
- `ENABLE CONFIGURATION`: Enables broker management of the broker configuration and all of its databases
- `ENABLE DATABASE`: Enables broker management of the specified database
- `ENABLE FAR_SYNC`: Enables broker management of the specified Far Sync instance
- `ENABLE FAST_START FAILOVER`: Enables the broker to automatically fail over from the primary database to a target standby database
- `ENABLE FAST_START FAILOVER CONDITION`: Allows a user to add conditions for which a fast-start failover should be performed
- `EXIT`: Exits the Data Guard command-line interface
- `FAILOVER`: Performs a database failover operation in which the standby database to which DGMGRL is currently connected fails over to the role of primary database
- `HELP`: Displays online help for the Data Guard command-line interface
- `QUIT`: Quits the Data Guard command-line interface
- `REINSTATE DATABASE`: Reinstates the database after a failover
- `REMOVE CONFIGURATION`: Removes the broker configuration and ends broker management of its members

- `REMOVE DATABASE`: Removes the specified standby database from the broker configuration
- `REMOVE FAR_SYNC`: Removes a Far Sync instance from an Oracle Data Guard broker configuration
- `REMOVE INSTANCE`: Removes an instance from the broker configuration
- `SHOW CONFIGURATION`: Displays information about the broker configuration
- `SHOW DATABASE`: Displays information about the specified database
- `SHOW FAR_SYNC`: Shows information about a Far Sync instance
- `SHOW FAST_START FAILOVER`: Displays all fast-start failover–related information
- `SHOW INSTANCE`: Displays information about the specified instance
- `SHUTDOWN`: Shuts down a currently running Oracle database
- `SQL`: Allows you to enter SQL statements from the Data Guard command-line interface (DGMGRL)
- `START OBSERVER`: Starts the observer
- `STARTUP`: Starts an Oracle instance with the same options as SQL*Plus, including mounting and opening a database
- `STOP OBSERVER`: Stops the observer
- `SWITCHOVER`: Performs a switchover operation in which the current primary database becomes a standby database, and the specified standby database becomes the primary database
- `VALIDATE DATABASE`: Performs a comprehensive set of database checks prior to a role change

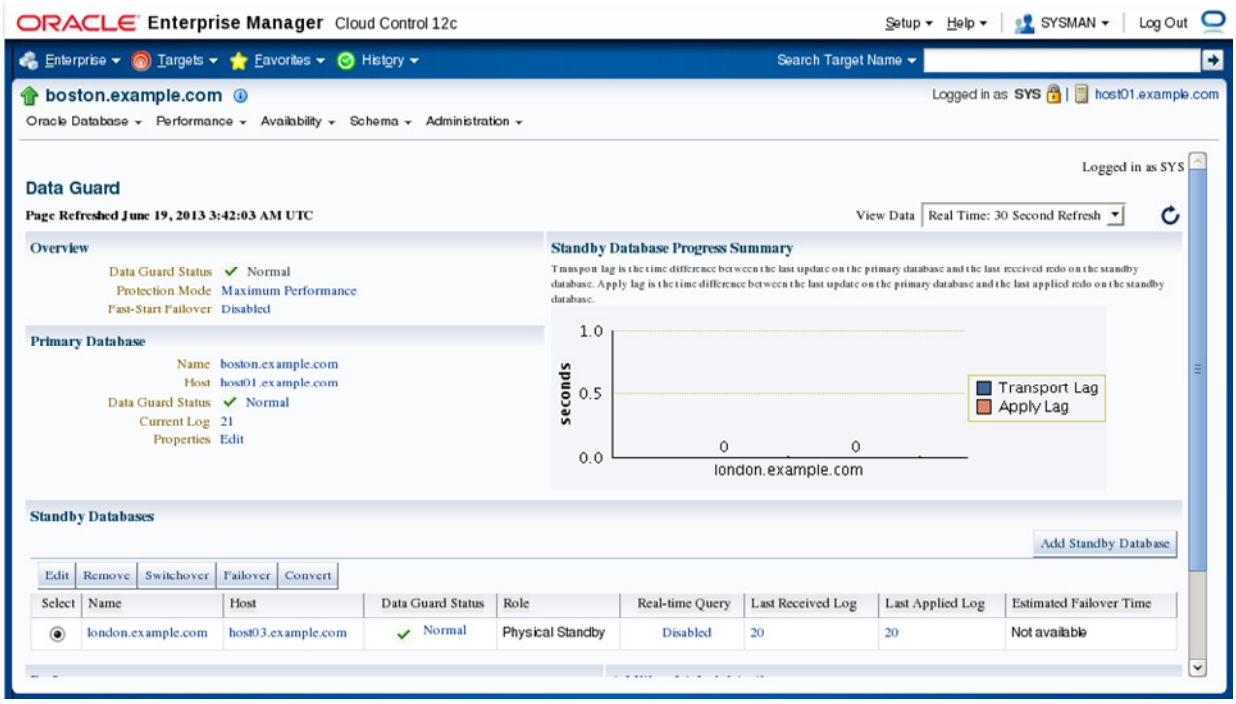
Using Oracle Enterprise Manager Cloud Control



To access the Data Guard features in Enterprise Manager Cloud Control:

1. Click the Targets tab to go to the Targets page.
2. Click Databases to go to the Databases page, where you can see a list of all discovered databases, including the primary database.
3. Click the primary database to go to the primary database home page.
4. Click Availability.
5. Click “Data Guard Administration” to open the Data Guard Overview page.

Data Guard Overview Page



On the Data Guard Overview page, you can:

- View the protection mode and access the page to edit the protection mode
- View a summary showing the amount of data that the standby database has not received
- View information about the primary database
- View or access pages to change information for the standby databases:
  - Add a standby database to the broker configuration
  - Change the state or properties
  - Discontinue Data Guard broker control
  - Switch the role from standby to primary
  - Transition the standby database to the role of the primary database
- Access pages to view performance information for the configuration and status of online redo log files for each standby database
- Perform a verification process on the Data Guard configuration

Benefits of Using Enterprise Manager

- Enables you to manage your configuration by using a familiar interface and event-management system
- Automates and simplifies the complex operations of creating and managing standby databases through the use of wizards
- Performs all Oracle Net Services configuration changes that are necessary to support redo transport services and log apply services
- Provides a verify operation to ensure that redo transport services and log apply services are configured and functioning properly
- Enables you to select a new primary database from a set of viable standby databases

ORACLE

Managing your Data Guard configuration with Oracle Enterprise Manager Cloud Control provides the following benefits:

- Enables you to manage your configuration using the familiar Enterprise Manager interface and event-management system
- Provides a wizard that automates the complex tasks involved in creating a broker configuration
- Provides the Add Standby Database Wizard to guide you through the process of adding more databases
- Performs all Oracle Net Services configuration changes that are necessary to support redo transport services and log apply services across the configuration
- Provides a verify operation to ensure that redo transport services and log apply services are configured and functioning properly
- Enables you to select a new primary database from a set of viable standby databases when you need to initiate a role change for a switchover or failover operation

Quiz

Which tools are used to interface with the Data Guard Broker?

a. DGMGRL
b. SRVCTL
c. Enterprise Manager
d. CRSCTL

Answer: a, c

Quiz

It is necessary to manage standby databases with the Data Guard Broker framework.

a. True
b. False

Answer: b

Summary

In this lesson, you should have learned how to:

- The Data Guard broker architecture
- Data Guard broker components
- Benefits of the Data Guard broker
- Data Guard broker configurations
- How to use Enterprise Manager to manage your Data Guard configuration
- How to invoke DGMGRL to manage your Data Guard configuration

9

Creating a Data Guard Broker Configuration

Objectives

After completing this lesson, you should be able to use DGMGRL commands to:

- Create a Data Guard broker configuration
- Manage the Data Guard broker configuration

Data Guard Broker: Requirements

- Oracle Database Enterprise Edition
- Single-instance or multi-instance environment
- `COMPATIBLE` parameter: Set to 12.1 or later for primary and standby databases to take advantage of new 12.1 features (Optional).
- Oracle Net Services network files: Must be configured for the primary database and any existing standby databases or Far Sync instances. Enterprise Manager Cloud Control configures files for new standby databases.
- `GLOBAL_DBNAME` attribute: Set to a concatenation of `db_unique_name_DGMGRL.db_domain`

To use the Data Guard broker, you must comply with the following requirements:

- Use the Enterprise Edition of Oracle Database.
- Use a single-instance or multi-instance environment.
- Set the `COMPATIBLE` initialization parameter to `12.1` to take advantage of new Oracle Database 12*c* Release 1 (12.1) features.
- Enterprise Manager automatically configures the Oracle Net Services network files when it creates a standby database. If you configure an existing standby database in the broker configuration, you must configure the network files. You must use TCP/IP.
- To enable the Data Guard broker to restart instances during the course of broker operations, a service with a specific name must be statically registered with the local listener of each instance. The value of the `GLOBAL_DBNAME` attribute must be set to a concatenation of `db_unique_name_DGMGRL.db_domain`.

Data Guard Broker: Requirements

- DG_BROKER_START initialization parameter: Set to TRUE
- Primary database: ARCHIVELOG mode
- All databases: MOUNT or OPEN mode
- DG_BROKER_CONFIG_FILE*n*: Configured for shared access for any RAC databases
- LOG_ARCHIVE_DEST_n parameters: Must be cleared on any instance when using SERVICE attribute before creating a broker configuration

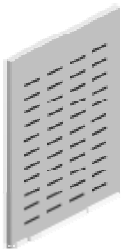
- Set the DG_BROKER_START initialization parameter to TRUE. This starts the DMON process.

  Note: When you use Enterprise Manager to create your configuration, this parameter is automatically set to TRUE.
- The primary database must be in ARCHIVELOG mode.
- Any database that is managed by the broker (including, for a RAC database, all instances of the database) must be mounted or open. The broker cannot start an instance.
- If any database in your configuration is a RAC database, you must configure the DG_BROKER_CONFIG_FILE*n* initialization parameters for that database so that they point to the same shared files for all instances of that database. You cannot use the default values for these parameters.

  Note: The shared files could be files on a cluster file system or on raw devices, or the files could be stored using Automatic Storage Management (ASM).
- As of Oracle Database 12*c* Release 1 (12.1), for all databases to be added to a broker configuration, any LOG_ARCHIVE_DEST_n parameters that have the SERVICE attribute set, but not the NOREGISTER attribute, must be cleared.

Data Guard Broker and the SPFILE

- You must use a server parameter file (SPFILE) for initialization parameters.
- Using the SPFILE enables the Data Guard broker to keep its configuration file and the database SPFILE consistent.
- If you use the broker, use Enterprise Manager Cloud Control or DGMGRL to update database parameter values.



To ensure that the broker can update the values of parameters in both the database instance and the configuration file, you must use the persistent server parameter file (SPFILE) to control static and dynamic initialization parameters. Use of the SPFILE gives the broker a mechanism that enables it to reconcile property values that you have selected when using the broker with any related initialization parameter values that are recorded in the SPFILE. In addition, the SPFILE permits persistent Data Guard settings so that Data Guard continues to work even after the broker is disabled.

When you set definitions or values for database properties in the broker configuration, the broker records the changes in the configuration file and also propagates the changes to the related initialization parameters in the server parameter file in the Data Guard configuration.

When the configuration is enabled, the broker keeps the database property values in the Data Guard configuration file consistent with the values of the database initialization parameters in the SPFILE.

Even when the configuration is disabled, you can update database property values through the broker. The broker retains the property settings (without validating the values) and updates the database initialization parameters in the SPFILE and the in-memory settings the next time you enable the broker configuration.

For dynamic initialization parameters, the broker keeps the value of the database parameter consistent in the System Global Area (SGA) for the instance, in the Data Guard configuration files, and in the SPFILE. For static initialization parameters, the value in the SGA may differ from what is in the configuration files and in the SPFILE. Typically, the broker reconciles the differences by updating all parameter and property values the next time the database instance is stopped and restarted.

Note: When using the broker (with Enterprise Manager or DGMGRL), do not attempt to manually set the parameters that the broker controls. If you set them manually, either you render your configuration inoperable or the broker simply takes the next opportunity to reset the parameter to the recorded setting. If you want to change a parameter value, you must change it by using one of the broker interfaces.

Data Guard Monitor: Configuration File

- The broker configuration file is:
  - Automatically created and named using a default path name and file name when the broker is started
  - Managed automatically by the DMON process
- The configuration file and a copy are created at each managed site with default names:
  - dr1<db_unique_name>.dat
  - dr2<db_unique_name>.dat
- Configuration file default locations are operating system–specific:
  - Default location for UNIX and Linux: ORACLE_HOME/dbs
  - Default location for Windows: ORACLE_HOME\database
- Use DG_BROKER_CONFIG_FILE*n* to override the default path name and file name.

ORACLE

The DMON process maintains persistent configuration data about all the databases in the broker configuration in a broker configuration file. Every database that is part of the Data Guard broker configuration has two broker configuration files that are maintained and synchronized for each database in the broker configuration. One of the files is in use and the other acts as a backup. The configuration files are binary files and cannot be edited.

When the broker is started for the first time, the configuration files are created and named automatically by using a default name. You can override this default name by setting the DG_BROKER_CONFIG_FILE*n* initialization parameters. You can also change the configuration file names dynamically by issuing the ALTER SYSTEM SQL statement. You must disable the broker configuration and stop the DMON process before changing the names.

The configuration files contain entries that describe the state and properties of the databases in the configuration. For example, the files record the databases that are part of the configuration, the roles and properties of each of the databases, and the state of each of the databases in the configuration. Two files are maintained so that there is always a record of the last-known valid state of the configuration. The broker uses the data in the configuration file to configure and start the databases, control each database's behavior, and provide information to DGMGRL and Enterprise Manager.

Note: The broker configuration files can reside on disks having any sector size up to 4 KB.

Data Guard Broker: Log Files

- The broker log files contain information recorded by the DMON process.
- There is one file for each instance in the broker configuration.
- Broker log files are created in the same directory as the alert log and are named drc<$ORACLE_SID>.log.

The DMON process records operational and status information in the broker log file for each instance in the Data Guard broker configuration.

Creating a Broker Configuration

1. Clear existing `LOG_ARCHIVE_DEST_n` entries on every instance that references network locations.
2. Invoke DGMGRL and connect to the primary database.
3. Define the configuration, including a profile for the primary database.
4. Add standby databases to the configuration.
5. Add Far Sync instances to the configuration.
6. Enable the configuration, including the databases.
7. Define routing rules for the configuration.

ORACLE

Detailed steps for creating a broker configuration will be shown in the following slides.

Clear Redo Transport Network Locations on Primary

- Clear existing LOG_ARCHIVE_DEST_n entries on each instance that references network locations.

```
SQL> select name,value from v$parameter where value like
   '%SERVICE%';
NAME                 VALUE
-------------------- ---------------------------------
log_archive_dest_2   SERVICE=bostonFS SYNC REOPEN=15
                     valid_for=(ONLINE_LOGFILES,
                     PRIMARY_ROLE) db_unique_name =
                     bostonFS
log_archive_dest_3   SERVICE=london2 SYNC REOPEN=15
                     valid_for=(ONLINE_LOGFILES,
                     PRIMARY_ROLE) db_unique_name =
                     london2
SQL> alter system log_archive_dest_2='' scope=both;
SQL> alter system log_archive_dest_3='' scope=both;
```

ORACLE

As of Oracle Database 12*c* Release 1 (12.1), no database can be added to the Data Guard broker configuration if the database instance has a LOG_ARCHIVE_DEST_n parameter using the SERVICE attribute. For an existing Data Guard environment created with SQL, this includes the primary database, physical standby databases, logical standby databases, and Far Sync instances.

You must clear any remote redo transport destinations on the primary database that do not have the NOREGISTER attribute, before a configuration can be created. Otherwise, the following error message is returned when you attempt to create the configuration:

```
ORA-16698: LOG_ARCHIVE_DEST_n parameter set for object to be added
Failed.
```

Connecting to the Primary Database with DGMGRL

- Connecting to the primary database on the local system

```
DGMGRL> connect sysdg/password
```

- Connecting to the primary database remotely using the password file

```
DGMGRL> connect sysdg/password@boston
```

- Adding a database user to the password file

```
SQL> grant sysdg to dguser;
```

ORACLE

Similar to SQL*Plus, DGMGRL can connect using operating system authentication (not shown in the above slide), local user authentication, and remote user authentication using the password file. The generic syntax is as follows:

DGMGRL> CONNECT username/password[@connect-identifier]

The username and password must be valid for the configuration member to which you are trying to connect. The username you specify must have the `SYSDG` or `SYSDBA` privilege. Do not include `AS SYSDG` or `AS SYSDBA` on the `CONNECT` command. DGMGRL first attempts an `AS SYSDG` connection. If that fails, it will then attempt an `AS SYSDBA` connection. Additional database account users can be added to the password file by granting them the SYSDG privilege. The following query can show who has the `SYSDG` privilege:

```
SQL> select * from v$pwfile_users;
USERNAME            SYSDB SYSOP SYSAS SYSBA SYSDG SYSKM     CON_ID
------------------- ----- ----- ----- ----- ----- ----- ----------
SYS                 TRUE  TRUE  FALSE FALSE FALSE FALSE          0
SYSDG               FALSE FALSE FALSE FALSE TRUE  FALSE          1
SYSBACKUP           FALSE FALSE FALSE TRUE  FALSE FALSE          1
SYSKM               FALSE FALSE FALSE FALSE FALSE TRUE           1
```

Defining the Broker Configuration and the Primary Database Profile

```
DGMGRL> CREATE CONFIGURATION 'DRSolution' AS PRIMARY
        DATABASE IS 'boston' CONNECT IDENTIFIER IS
        boston;
Configuration "DRSolution" created with primary database
   "boston"

DGMGRL> show configuration
Configuration - DRSolution

  Protection Mode: MaxPerformance
  Databases:
  boston - Primary database

Fast-Start Failover: DISABLED

Configuration Status:
DISABLED
```

ORACLE

Execute the `CREATE CONFIGURATION` command to define the broker creation and create a profile for the primary database. The following parameters must be specified:

- **`configuration-name`:** User-specified name for the configuration
- **`database-name`:** Used by the broker to reference the primary database. You must use the value of the `DB_UNIQUE_NAME` initialization parameter for the database name.
- **`connect-identifier`:** The value you specify for the connect identifier is a fully specified connect descriptor or a name to be resolved by an Oracle Net Services naming method. The broker uses this value to communicate with the other databases defined in the configuration. The `DGConnectIdentifier` database property is set to the connect identifier value.

Adding a Standby Database to the Configuration

```
DGMGRL> add database 'london' as connect identifier is
    london;
Database "london" added

DGMGRL> show configuration
Configuration - DRSolution

  Protection Mode: MaxPerformance
  Databases:
  boston   - Primary database
    london   - Physical standby database

Fast-Start Failover: DISABLED

Configuration Status:
DISABLED
```


Use the `ADD DATABASE` DGMGRL command to define the standby database and create a broker configuration profile. The database name specified must be the same as the value of the `DB_UNIQUE_NAME` initialization parameter. The connect identifier is used by Oracle Net Services to access the database from all other databases in the configuration.

The `AS CONNECT IDENTIFIER` clause is optional. If you do not specify this clause, the broker will search the `LOG_ARCHIVE_DEST_`*n* initialization parameters on the primary database for an entry that corresponds to the database being added.

The broker uses the specified `connect-identifier` to communicate with the specified database from other databases. Therefore, you must ensure that the `connect-identifier` can be used to address the specified database from all databases in your configuration. For example, if TNS is used as the naming method, you must ensure that the `tnsnames.ora` file on every database and instance that is part of the configuration contains an entry for the connect identifier. The connect identifier must resolve to the same connect descriptor.

Note: The broker will determine the standby type whenever you add an existing standby. In older versions of the Oracle database, such as version 10.2, you had to specify "`MAINTAINED AS [ PHYSICAL | LOGICAL ]`" when you added an existing standby.

Adding a Far Sync to the Configuration

```
DGMGRL> add far_sync 'bostonFS' as connect identifier is
   bostonFS;
far sync instance "bostonFS" added

DGMGRL> show configuration
Configuration - DRSolution

  Protection Mode: MaxPerformance
  Databases:
  boston   - Primary database
    bostonFS - Far Sync (inactive)
    london   - Physical standby database

Fast-Start Failover: DISABLED

Configuration Status:
DISABLED
```


Use the `ADD FAR_SYNC` DGMGRL command to define the Far Sync and create a broker configuration profile. The Far Sync name specified must be the same as the value of the `DB_UNIQUE_NAME` initialization parameter. The connect identifier is used by Oracle Net Services to access the database from all other databases in the configuration.

Enabling the Configuration

```
DGMGRL> ENABLE CONFIGURATION;
Enabled.

DGMGRL> SHOW CONFIGURATION
Configuration - DRSolution

  Protection Mode: MaxPerformance
  Databases:
  boston   - Primary database
    bostonFS - Far Sync (inactive)
    london   - Physical standby database
    london2  - Logical standby database

Fast-Start Failover: DISABLED

Configuration Status:
SUCCESS
```


After defining the databases in the configuration, you enable the configuration and its databases by executing the `ENABLE CONFIGURATION` DGMGRL command.

Broker Support for Complex Redo Routing

By default, a primary database sends its redo to every possible redo transport destination in a broker configuration.

- The `RedoRoutes` broker property allows more complex routing to be defined.
  - Supports cascaded configurations
  - Supports Far Sync configurations
  - Supports real-time cascading and without real-time cascading
  - Supports rules dependent on which database is the current primary
  - Supports the new Fast Sync configuration

ORACLE

By default, a primary database sends the redo that it generates to every other redo transport destination in the configuration. You can use the `RedoRoutes` property with Oracle Data Guard 12*c* to create a more complex redo transport topology, such as one in which a physical standby database or a Far Sync forwards redo received from the primary database to one or more destinations, or one in which the redo transport mode used for a given destination depends on which database is in the primary role.

Defining RedoRoutes by Using DGMGRL

- The RedoRoutes property is set to a character string that contains one or more redo routing rules.

```
DGMGRL> EDIT DATABASE database-name
        SET PROPERTY 'RedoRoutes' = '(redo routing rule 1)(redo routing rule n)';
```

- Each rule contains one or more redo sources and one or more redo destinations, separated by a colon.
- The redo source field must contain the LOCAL keyword or a comma-separated list of DB_UNIQUE_NAME values.
- The redo destination field must contain the keyword ALL or a comma-separated list of database names, each of which can be followed by an optional redo transport attribute.

```
(redo source : redo destination) (LOCAL : redo destination ATTRIBUTE)
(redo source : redo destination, redo destination ATTRIBUTE)
```


The RedoRoutes property is set to a character string that contains one or more redo routing rules. Each rule contains one or more redo sources and one or more redo destinations. A redo routing rule becomes active when one of the redo sources in the rule is in the primary role. This results in redo from the primary database being sent to every redo destination in that rule. A redo routing rule contains a redo source field and a redo destination field, separated by a colon:

```
(redo source : redo destination)
```

The redo source field must contain the LOCAL keyword or a comma-separated list of DB_UNIQUE_NAME values:

```
{LOCAL | db_unique_name_1,[,db_unique_name_n]}
```

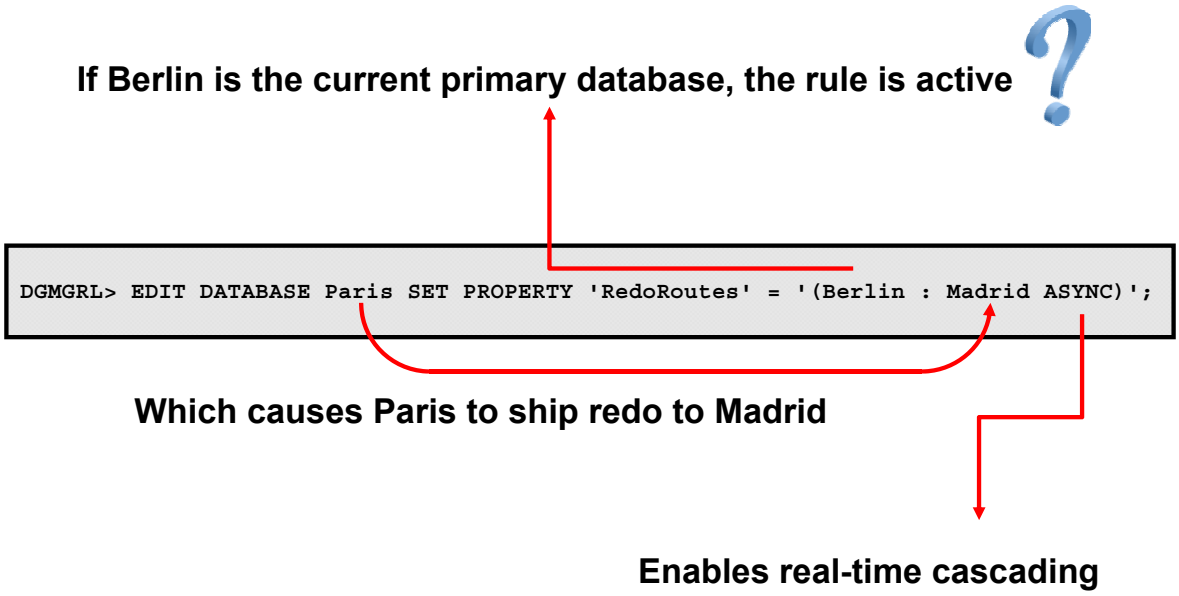
You cannot set the RedoRoutes property on a logical or snapshot standby database.

RedoRoutes Usage Guidelines

- The RedoRoutes property has a default value of NULL, which is treated as (LOCAL : ALL).
- A redo routing rule is active if its redo source field specifies the current primary database.
- If a rule is active, primary database redo is sent by the database at which the rule is defined to each destination specified in the redo destination field of that rule.
- The ASYNC redo transport attribute must be explicitly specified for a cascaded destination to enable real-time cascading to that destination.
- The RedoRoutes property cannot be set on a logical or snapshot standby database.

The RedoRoutes property has a default value of NULL, which is treated as (LOCAL : ALL). A redo routing rule is active if its redo source field specifies the current primary database. If a rule is active, primary database redo is sent by the database at which the rule is defined to each destination specified in the redo destination field of that rule. The ASYNC redo transport attribute must be explicitly specified for a cascaded destination to enable real-time cascading to that destination. The RedoRoutes property cannot be set on a logical or snapshot standby database.

How to Read Redo Routing Rules



The diagram in the slide provides an example of interpreting a redo routing rule. The command syntax specifies Paris as the database unique name for which the `RedoRoutes` property is being set, Berlin as the redo source, and Madrid as the redo destination. The `ASYNC` attribute is explicitly specified. If Berlin (redo source) is the current primary database, the rule is active. An active rule would cause Paris (database unique name) to ship redo to Madrid (redo destination). The redo transport attribute can be set to `SYNC`, `FASTSYNC`, or `ASYNC`. The `AYSNC` setting used in the example is required to enable real-time cascading. The `SYNC` and `FASTSYNC` keywords are only applicable when the database that is doing the shipping is a primary database.

Far Sync Example with `RedoRoutes`

An example Far Sync configuration for a Maximum Availability broker configuration:

- Primary database in Boston, MA.
- Far Sync in Cambridge, MA.
- Physical standby in Chicago, IL.
- Far Sync in Shaumburg, IL. (for role reversal)

```
DGMGRL> EDIT DATABASE Boston SET PROPERTY 'RedoRoutes' = '(LOCAL : Cambridge
SYNC)';
DGMGRL> EDIT FAR_SYNC Cambridge SET PROPERTY 'RedoRoutes' = '(Boston : Chicago
ASYNC)';
DGMGRL> EDIT DATABASE Chicago SET PROPERTY 'RedoRoutes' = '(Local : Shaumburg
SYNC)';
DGMGRL> EDIT FAR_SYNC Shaumburg SET PROPERTY 'RedoRoutes' = '(Chicago : Boston
ASYNC)';
```


After Data Guard broker has been configured for a primary database, Far Sync, physical standby, and a second Far Sync for role reversal, the configuration will appear as follows:

```
DGMGRL> SHOW CONFIGURATION;
Configuration - The SUPER cluster
  Protection Mode: MaxAvailability
  Databases:
    Boston  - Primary database
        Cambridge  - Far Sync
            Chicago  - Physical standby database
                Shaumburg - Far Sync (Inactive)
Fast-Start Failover: DISABLED
Configuration Status:
SUCCESS
```

Notice that in the output of the show configuration command, indentation is used to show that a database or Far Sync instance is receiving redo from a source location. The use of RedoRoutes overrides the `LogXptMode` Data Guard property.

Changing Database Properties and States

- To alter the state of only the primary database:

```
DGMGRL> EDIT DATABASE boston SET STATE='TRANSPORT-OFF';
```

- To alter the state of a physical or logical standby database:

```
DGMGRL> EDIT DATABASE boston SET STATE='APPLY-OFF';
```

- To alter a configurable database property:

```
DGMGRL> EDIT DATABASE boston SET PROPERTY
   LogXptMode='SYNC';
```


Configurable database properties can be viewed and dynamically updated by using the `EDIT DATABASE SET PROPERTY` DGMGRL command. Use the `SHOW DATABASE VERBOSE` command to obtain a complete list of database properties.

The `EDIT DATABASE SET STATE` DGMGRL command is used to change the state of the primary database and standby databases. When the broker configuration is enabled, the databases are in one of four states:

- **`TRANSPORT-ON`** (applicable only to the primary database): Redo transport services transmit redo data to the standby databases when the primary database is open in read/write mode.
- **`TRANSPORT-OFF`** (applicable only to the primary database): Redo transport services are stopped on the primary database.
- **`APPLY-ON`** (applicable only to a physical or logical standby database): Redo Apply is started on the physical standby database when it is mounted or in open read-only mode. SQL Apply is started on a logical standby database when it is opened and the logical standby database guard is on.
- **`APPLY-OFF`** (applicable only to a physical or logical standby database): Redo Apply is stopped on a physical standby database. SQL Apply is not running on a logical standby database.

Managing Redo Transport Services by Using DGMGRL

Specify database properties to manage redo transport services:

Property	Purpose
AlternateLocation	Specifies an alternate disk location to store the archived redo log files in the standby when the location specified by the `StandbyArchiveLocation` configurable property fails
ApplyLagThreshold	Generates a warning status for a logical or physical standby when the member's apply lag exceeds the value specified by the property
ArchiveLagTarget	Limits the amount of data that can be lost and, effectively, increases the availability of the standby database or Far Sync instance by forcing a log switch after the amount of time you specify (in seconds) elapses
Binding	Specifies whether the redo destination is `MANDATORY` or `OPTIONAL`
DGConnectIdentifier	Specifies the connection identifier the broker uses when making connections to a configuration member

ORACLE

The properties that the Data Guard broker uses can be categorized into configuration properties that control the behavior of the broker configuration, monitorable (read-only) properties for information, and configurable properties. The properties listed in the slide are configurable properties related to redo transport services specifically.

For a complete list and detailed explanation of each broker property, see the documentation titled *Oracle Data Guard Broker.*

Managing Redo Transport Services by Using DGMGRL

Specify database properties to manage redo transport services:

Property	Purpose
LogArchiveFormat	Specifies the format for filenames of archived redo log files using a database ID (%d), thread (%t), sequence number (%s), and resetlogs ID (%r)
LogArchiveMaxProcesses	Specifies the initial number of archiver processes (ARCn) that are invoked
LogArchiveMinSucceedDest	Controls when online redo log files are available for reuse provided archiving has succeeded to a minimum number of destinations
LogShipping	Specifies the `ENABLE` and `DEFER` values for the `LOG_ARCHIVE_DEST_STATE_n` initialization parameter of the database or Far Sync instance that is sending redo data
LogXptMode	Specifies the redo transport services on each configuration member to `SYNC`, `ASYNC`, or `FASTSYNC`

ORACLE

The properties that the Data Guard broker uses can be categorized into configuration properties that control the behavior of the broker configuration, monitorable (read-only) properties for information, and configurable properties. The properties listed in the slide are configurable properties related to redo transport services specifically.

For a complete list and detailed explanation of each broker property, see the documentation titled *Oracle Data Guard Broker*

Managing Redo Transport Services by Using DGMGRL

Specify database properties to manage redo transport services:

Configurable Property	Purpose
MaxConnections	Specifies how many ARCn processes will be used in parallel to transmit redo data from a single archived redo log on the database or Far Sync instance to the archived redo log at the remote site
MaxFailure	Specifies the maximum number of contiguous archiving failures before the redo transport services stop trying to transport archived redo log files to the standby database
NetTimeout	Specifies the number of seconds the LGWR waits for Oracle Net Services to respond to a LGWR request and is used to bypass the long connection timeout in TCP/IP
RedoCompression	Specifies whether redo data is transmitted to a standby database or Far Sync instance in compressed or uncompressed form

ORACLE

The properties that the Data Guard broker uses can be categorized into configuration properties that control the behavior of the broker configuration, monitorable (read-only) properties for information, and configurable properties. The properties listed in the slide are configurable properties related to redo transport services specifically.

For a complete list and detailed explanation of each broker property, see the documentation titled *Oracle Data Guard Broker*.

Managing Redo Transport Services by Using DGMGRL

Specify database properties to manage redo transport services:

Configurable Property	Purpose
ReopenSecs	Specifies the minimum number of seconds before the archiver process (ARCn, foreground, or log writer process) should try again to access a previously failed destination
StandbyArchiveLocation	Specifies the location of archived redo log files arriving from a redo source
TransportDisconnectedThreshold	Used to generate a warning status for a logical, physical, or snapshot standby, or a Far Sync instance when the last communication from the primary database exceeds the value specified in seconds
TransportLagThreshold	Used to generate a warning status for a logical, physical, or snapshot standby, or a Far Sync instance when the member's transport lag exceeds the value specified in seconds

ORACLE

The properties that the Data Guard broker uses can be categorized into configuration properties that control the behavior of the broker configuration, monitorable (read-only) properties for information, and configurable properties. The properties listed in the slide are configurable properties related to redo transport services specifically.

For a complete list and detailed explanation of each broker property, see the documentation titled *Oracle Data Guard Broker*

Managing the Redo Transport Service by Using the LogXptMode Property

- The redo transport service must be set up for the chosen data protection mode.
- Use the LogXptMode property to set the redo transport services:
  - ASYNC: Sets the ASYNC and NOAFFIRM attributes of LOG_ARCHIVE_DEST_*n*
  - SYNC: Sets the SYNC and AFFIRM attributes of LOG_ARCHIVE_DEST_*n*
  - FASTSYNC: Sets the SYNC and NOAFFIRM attributes of LOG_ARCHIVE_DEST_*n*

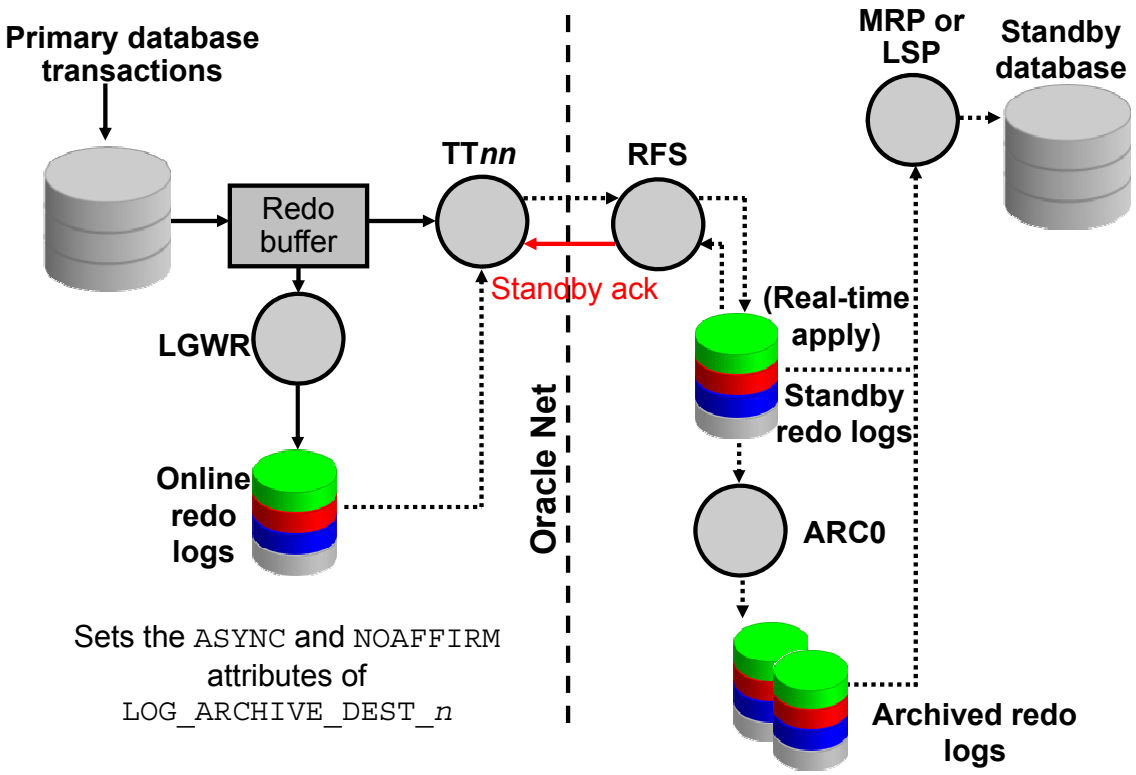
Use the Data Guard broker LogXptMode property to set redo transport services.

- **ASYNC:** Configures redo transport services by setting the ASYNC and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter
- **SYNC:** Configures redo transport services by setting the SYNC and AFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter
- **FASTSYNC:** Configures redo transport services by setting the SYNC and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter

Definitions of LOG_ARCHIVE_DEST_*n* Attributes

- **ASYNC:** Redo data that is generated by a transaction need not have been received at a destination that has this attribute before the transaction can commit.
- **SYNC:** Redo data that is generated by a transaction must have been received by every enabled destination that has this attribute before the transaction can commit.
- **AFFIRM and NOAFFIRM:** Control whether redo transport services use synchronous or asynchronous disk I/O to write redo data to the archived redo log files
  - **AFFIRM:** Specifies that a redo transport destination acknowledges received redo data after writing it to the standby redo log
  - **NOAFFIRM:** Specifies that a redo transport destination acknowledges received redo data before writing it to the standby redo log

Setting LogXptMode to ASYNC

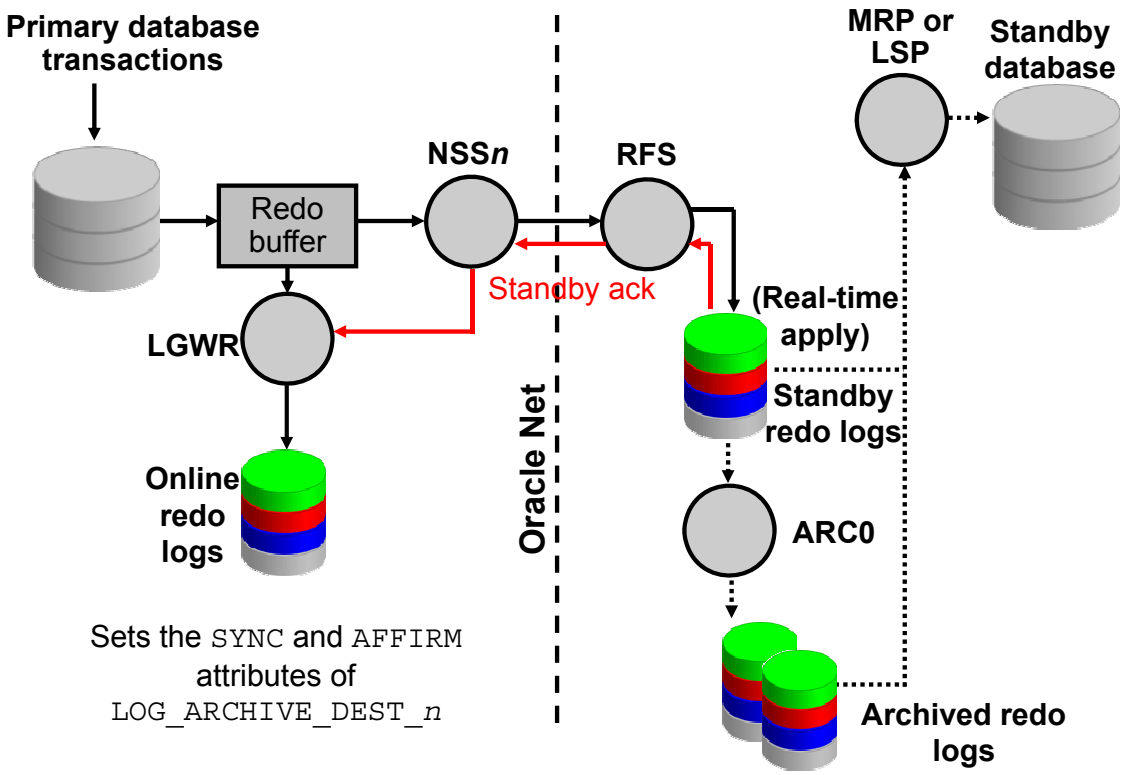


When you set the LogXptMode property to ASYNC, the broker configures redo transport services for this standby database by using the ASYNC and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter. ASYNC mode enables a moderate level of data protection for the primary database with a lower performance impact than SYNC mode.

Standby redo log files are required for ASYNC mode.

In the diagram in the slide, the solid line represents a synchronous operation (writing locally to the primary database online redo log files). The broken lines represent asynchronous operations with respect to the transaction commit on the primary database.

Setting LogXptMode to SYNC

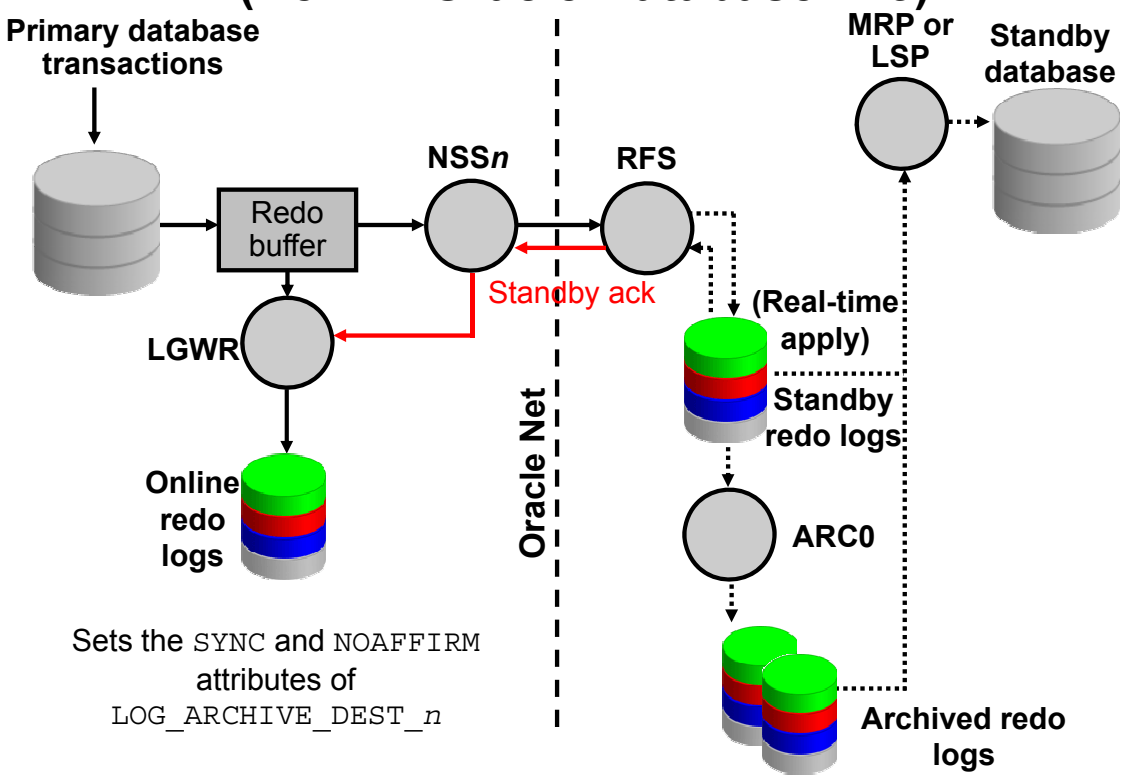


When you set the LogXptMode database property to SYNC, the broker configures redo transport services for this standby database by using the SYNC and AFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter. This mode enables the highest level of data protection to the primary database but also incurs the highest performance overhead.

Standby redo log files are required for SYNC mode.

In the diagram in the slide, the solid line represents a synchronous operation (writing locally to the primary database online redo log files). The broken lines represent asynchronous operations with respect to the transaction commit on the primary database.

Setting LogXptMode to FASTSYNC (New in Oracle Database 12*c*)



When you set the LogXptMode database property to FASTSYNC, the broker configures redo transport services for this standby database by using the SYNC and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_*n* initialization parameter. This mode is new for Oracle Database 12*c*.

Standby redo log files are required for FASTSYNC mode.

In the diagram in the slide, the solid line represents a synchronous operation (writing locally to the primary database online redo log files). The broken lines represent asynchronous operations with respect to the transaction commit on the primary database.

Disabling Broker Management of the Configuration or Standby Database

- Disable broker management of the configuration:

```
DGMGRL> DISABLE CONFIGURATION;
```

- Disable broker management of a standby database:

```
DGMGRL> DISABLE DATABASE 'london';
```

- Disable broker management of a Far Sync:

```
DGMGRL> DISABLE FAR_SYNC 'londonFS';
```


You can disable broker management of a configuration or a specific standby database. You may want to disable broker management to perform maintenance or to temporarily prevent automated actions for testing.

Disabling broker management does not affect the underlying Data Guard configuration. Rather, it removes the ability for you to manage the configuration by using DGMGRL or Enterprise Manager Cloud Control.

Use the `DISABLE CONFIGURATION` DGMGRL command to disable broker management of the configuration. Use the `DISABLE DATABASE` DGMGRL command to disable broker management of the named standby database.

Note: You must use the `DISABLE CONFIGURATION` command to disable broker management of a primary database.

Removing the Configuration or Standby Database

- Remove a standby database from the configuration:

```
DGMGRL> REMOVE DATABASE 'london2' [PRESERVE
   DESTINATIONS];
```

- Remove a Far Sync from the configuration

```
DGMGRL> REMOVE FAR_SYNC 'londonFS';
```

- Remove the configuration:

```
DGMGRL> REMOVE CONFIGURATION [PRESERVE DESTINATIONS];
```


Use the `REMOVE DATABASE` command to delete the named standby database from the broker configuration.

Use the `REMOVE CONFIGURATION` command to delete the configuration. When you execute the `REMOVE CONFIGURATION` command, the broker configuration is removed, all database profiles are removed, the Data Guard broker configuration files is removed, and broker management of all databases in the configuration is terminated. Execution of this command does not affect the underlying databases. You cannot remove the configuration when fast-start failover is enabled.

The `PRESERVE DESTINATIONS` syntax is available for both the `REMOVE DATABASE` and `REMOVE CONFIGURATIONS` commands. By default, broker settings for `LOG_ARCHIVE_DEST_n` and `LOG_ARCHIVE_CONFIG` initialization parameters are removed when the `REMOVE` commands are issued, but `PRESERVE DESTINATIONS` maintains these initialization parameters. A common usage for this is to allow the Data Guard broker to perform all of the setup for the `LOG_ARCHIVE_DEST_n` initialization parameters and then remove the broker when it has finished.

Quiz

How many Data Guard broker configuration files are created for each database unique name?

a. One
b. Two
c. Three
d. Four

Answer: b

Summary

In this lesson, you should have learned how to use DGMGRL commands to:

- Create a Data Guard broker configuration
- Manage the Data Guard broker configuration

Practice 9: Overview

This practice covers the following topics:

- Setting the `DG_BROKER_START` initialization parameter
- Creating the broker configuration
- Enabling the broker configuration